

# FPT UNIVERSITY

## AI Capstone Project

---

### AI Deep Learning Framework for Vietnamese Multi-Speaker Transcription and Identification in Noisy Environments

| Group and member information |                                                          |
|------------------------------|----------------------------------------------------------|
| Group Members                | Tran Nguyen Quang Khang - Leader - SE183747              |
|                              | Nguyen Duy Phuong - Member - SE183477                    |
|                              | Truong Minh Sang - Member - SE184204                     |
|                              | Ho Khanh Duy - Member - SE184539                         |
| Supervisor                   | Huynh Van Thong - ThongHV4<br>Nguyen Hong Hai - HongNH51 |
| Capstone Project code        | AIP491-SP26AI91                                          |

HCMC, April 2026

# Contents

|                        |           |
|------------------------|-----------|
| <b>List of Figures</b> | <b>iv</b> |
|------------------------|-----------|

|                       |           |
|-----------------------|-----------|
| <b>List of Tables</b> | <b>vi</b> |
|-----------------------|-----------|

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>1 Introduction</b>                                                  | <b>1</b>  |
| 1.1 Background . . . . .                                               | 1         |
| 1.2 Objective . . . . .                                                | 2         |
| 1.3 Problem Statement . . . . .                                        | 3         |
| 1.4 Scope & Limitations . . . . .                                      | 5         |
| <b>2 Project Management Plan</b>                                       | <b>7</b>  |
| 2.1 Team Structure and Roles . . . . .                                 | 7         |
| 2.2 Communication Plan . . . . .                                       | 7         |
| <b>3 Literature Review</b>                                             | <b>9</b>  |
| 3.1 Overview . . . . .                                                 | 9         |
| 3.2 State-of-the-Art Models and Existing Approaches . . . . .          | 10        |
| 3.3 Vietnamese Speech Processing Context . . . . .                     | 11        |
| 3.4 Knowledge Gap . . . . .                                            | 13        |
| 3.5 Datasets Relevant to This Project . . . . .                        | 13        |
| <b>4 Methodology</b>                                                   | <b>15</b> |
| 4.1 Inference Pipeline: Real-world Deployment . . . . .                | 15        |
| 4.1.1 End-to-End Pipeline . . . . .                                    | 15        |
| 4.2 Diarization & Transcription Pipeline: Technical Overview . . . . . | 15        |
| 4.2.1 Overall Pipeline Overview . . . . .                              | 15        |
| 4.3 Dataset Overview . . . . .                                         | 22        |
| 4.4 Self- labeled Datasets . . . . .                                   | 24        |
| 4.4.1 Process formation idea . . . . .                                 | 24        |
| 4.4.2 What this idea was based on . . . . .                            | 25        |
| 4.4.3 General characteristics of the dataset . . . . .                 | 25        |
| 4.4.4 Machine part . . . . .                                           | 25        |
| 4.4.5 Detailed description of the machine part . . . . .               | 27        |
| 4.4.6 Manual labeling . . . . .                                        | 28        |
| 4.5 Synthetic Datasets . . . . .                                       | 34        |
| 4.5.1 Introduction . . . . .                                           | 34        |
| 4.5.2 Raw Data Sources . . . . .                                       | 34        |
| 4.5.3 Forced Alignment . . . . .                                       | 35        |
| 4.5.4 Creating the Enrollment Set . . . . .                            | 39        |
| 4.5.5 Meeting Simulation Pipeline . . . . .                            | 40        |
| 4.5.6 Experimental Configurations and Parameters . . . . .             | 49        |
| 4.5.7 Output Labeling Formats . . . . .                                | 50        |
| 4.5.8 Dataset Splitting Strategy . . . . .                             | 52        |
| 4.5.9 Dataset Quality Validation . . . . .                             | 54        |
| 4.5.10 Overall Dataset Statistics . . . . .                            | 57        |
| 4.6 DiariZen Model: Overview and Architecture . . . . .                | 57        |
| 4.6.1 Overview and Basic Model Information . . . . .                   | 57        |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 4.6.2    | Technical Parameters to Clarify . . . . .                       | 60        |
| 4.6.3    | PIT Multilabel BCE: DiariZen . . . . .                          | 60        |
| 4.6.4    | Training and Test Data . . . . .                                | 61        |
| 4.6.5    | Three-Phase Fine-tuning Strategy (Gradual Unfreezing) . . . . . | 61        |
| 4.6.6    | Head Replacement and Multilabel Head Initialization . . . . .   | 62        |
| 4.6.7    | Two-Level Validation: Crop-level and File-level DER . . . . .   | 64        |
| 4.7      | Pyannote Model: Overview and Architecture . . . . .             | 64        |
| 4.7.1    | Pipeline Overview and Architecture . . . . .                    | 64        |
| 4.7.2    | Technical Constraints to Clarify . . . . .                      | 66        |
| 4.7.3    | Powerset Cross-Entropy: Pyannote . . . . .                      | 66        |
| 4.7.4    | Training and Test Data . . . . .                                | 67        |
| 4.7.5    | Pyannote Pipeline During Training and Inference . . . . .       | 69        |
| 4.8      | PhoWhisper-large: Overview and Architecture . . . . .           | 69        |
| 4.8.1    | Research Background . . . . .                                   | 69        |
| 4.8.2    | Model Architecture . . . . .                                    | 69        |
| 4.8.3    | Fine-tuning Strategy: Partial Unfreezing . . . . .              | 70        |
| 4.8.4    | Unicode NFC Normalization . . . . .                             | 71        |
| 4.8.5    | Training Data . . . . .                                         | 72        |
| 4.8.6    | Fine-tuning PhoWhisper-large . . . . .                          | 74        |
| 4.8.7    | Loss Function: Cross-Entropy . . . . .                          | 75        |
| 4.8.8    | Preprocessing Pipeline: Manifest Builder . . . . .              | 76        |
| 4.9      | Introduction to Qwen 2.5-1.5B-Instruct . . . . .                | 76        |
| 4.9.1    | Characteristics & Selection Rationale . . . . .                 | 76        |
| 4.9.2    | Qwen Summarization Pipeline . . . . .                           | 77        |
| 4.10     | Diarization Evaluation Metrics . . . . .                        | 78        |
| 4.10.1   | Diarization Error Rate (DER) . . . . .                          | 78        |
| 4.10.2   | Missed Speech (Miss) . . . . .                                  | 79        |
| 4.10.3   | False Alarm (FA) . . . . .                                      | 79        |
| 4.10.4   | Speaker Confusion (Conf) . . . . .                              | 80        |
| 4.10.5   | Overlap Detection Metrics: Precision, Recall, F1 . . . . .      | 80        |
| 4.10.6   | Diagnostic Quantities . . . . .                                 | 81        |
| 4.11     | Word Error Rate (WER): PhoWhisper . . . . .                     | 81        |
| 4.12     | Qwen Pipeline Evaluation Metrics . . . . .                      | 82        |
| <b>5</b> | <b>System Design and Implementation</b>                         | <b>82</b> |
| 5.1      | Overview . . . . .                                              | 82        |
| 5.2      | Design Principles and Configurability . . . . .                 | 83        |
| 5.3      | UI-Backend Integration and Output Organization . . . . .        | 84        |
| 5.4      | Main Interface Functions . . . . .                              | 85        |
| 5.4.1    | Input Area and Execution Modes . . . . .                        | 85        |
| 5.4.2    | Workspace for Transcript and Summary Inspection . . . . .       | 86        |
| 5.4.3    | Run History and Reopening Previous Sessions . . . . .           | 87        |
| <b>6</b> | <b>Results and Discussion</b>                                   | <b>89</b> |
| 6.1      | DiariZen Results . . . . .                                      | 89        |
| 6.1.1    | Training Progress . . . . .                                     | 89        |
| 6.1.2    | Threshold Sweep Results: DiariZen . . . . .                     | 90        |
| 6.1.3    | Overall Results on Synthetic data . . . . .                     | 92        |

|          |                                                                |            |
|----------|----------------------------------------------------------------|------------|
| 6.1.4    | Noise Robustness Evaluation: Synthetic Data . . . . .          | 93         |
| 6.1.5    | Results by Speaker Count Synthetic Data . . . . .              | 94         |
| 6.1.6    | Results on Self-labeled Set (Primary Evaluation Set) . . . . . | 95         |
| 6.2      | Pyannote Results . . . . .                                     | 96         |
| 6.2.1    | Training Progress . . . . .                                    | 96         |
| 6.2.2    | Threshold Sweep: Pyannote . . . . .                            | 96         |
| 6.2.3    | Overall Results on Synthetic Data . . . . .                    | 97         |
| 6.2.4    | Noise Robustness Evaluation: Synthetic Data . . . . .          | 98         |
| 6.2.5    | Results by Speaker Count: Synthetic Data . . . . .             | 99         |
| 6.2.6    | Results on Self-labeled Set (Primary Evaluation Set) . . . . . | 100        |
| 6.3      | PhoWhisper Results . . . . .                                   | 101        |
| 6.3.1    | PhoWhisper: Training Loss Visualization . . . . .              | 101        |
| 6.3.2    | Evaluation Method: Diagnostic Decomposition . . . . .          | 103        |
| 6.3.3    | Overall Results: Base → Stage 1 → Stage 2 . . . . .            | 104        |
| 6.3.4    | Results by Domain . . . . .                                    | 105        |
| 6.3.5    | Result Analysis . . . . .                                      | 105        |
| 6.3.6    | Technical Notes . . . . .                                      | 106        |
| 6.4      | Qwen Pipeline Evaluation Results . . . . .                     | 106        |
| 6.4.1    | Evaluation Methodology . . . . .                               | 106        |
| 6.4.2    | Automated Evaluation Results . . . . .                         | 107        |
| 6.4.3    | LLM Judge Results (Qwen 7B) . . . . .                          | 108        |
| 6.4.4    | Performance Benchmark . . . . .                                | 108        |
| 6.5      | Discussion . . . . .                                           | 108        |
| 6.5.1    | General Technical Notes . . . . .                              | 108        |
| 6.5.2    | DiariZen Discussion . . . . .                                  | 109        |
| 6.5.3    | Pyannote Discussion . . . . .                                  | 112        |
| 6.5.4    | Comparison DiariZen and Pyannote . . . . .                     | 115        |
| 6.5.5    | PhoWhisper: Discussion . . . . .                               | 119        |
| 6.5.6    | Qwen Discussion . . . . .                                      | 121        |
| <b>7</b> | <b>Conclusion</b>                                              | <b>123</b> |
|          | <b>References</b>                                              | <b>124</b> |

## List of Figures

|    |                                                                                                                                                      |    |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1  | Comparison of manual and AI-assisted meeting documentation workflows. . . . .                                                                        | 1  |
| 2  | Overview of the proposed system pipeline. . . . .                                                                                                    | 2  |
| 3  | Main sources of difficulty in Vietnamese multi-speaker meeting transcription, including tonal speech, background noise, and speaker overlap. . . . . | 3  |
| 4  | Simplified overview of the redesigned overlap-aware pipeline used in the proposed system. . . . .                                                    | 4  |
| 5  | System scope and boundary of the proposed research prototype, including the main in-scope modules and current out-of-scope limitations. . . . .      | 5  |
| 6  | From single-speaker speech recognition to speaker-aware meeting transcription. . . . .                                                               | 9  |
| 7  | Proposed Integrated Framework for Vietnamese Multi-speaker Transcription. . . . .                                                                    | 12 |
| 8  | End-to-end pipeline model. . . . .                                                                                                                   | 15 |
| 9  | Overall Pipeline Overview. . . . .                                                                                                                   | 16 |
| 10 | Phase 1 Input Preparation and Backend Inference. . . . .                                                                                             | 17 |
| 11 | Phase 2 Shared Segment Generation and Tiering. . . . .                                                                                               | 18 |
| 12 | Phase 3 Enrollment-Aware Speaker Initialization. . . . .                                                                                             | 19 |
| 13 | Phase 4 Embedding Collection and Initial Assignment. . . . .                                                                                         | 20 |
| 14 | Phase 5 Missing-Track Rescue and Propagation. . . . .                                                                                                | 21 |
| 15 | Phase 6: Post-Processing and Export. . . . .                                                                                                         | 21 |
| 16 | Phase 7 PhoWhisper Vietnamese Automatic Speech Recognition. . . . .                                                                                  | 22 |
| 17 | Phase 8 Summarization and Post-Processing Using Qwen. . . . .                                                                                        | 22 |
| 18 | Overall interface of the tool after loading the audio and transcript. . . . .                                                                        | 30 |
| 19 | Selecting a region on the waveform and adjusting the start/end times. . . . .                                                                        | 31 |
| 20 | Transcript table before manual correction. . . . .                                                                                                   | 31 |
| 21 | Transcript table after manual correction. . . . .                                                                                                    | 31 |
| 22 | Speaker label before manual correction. . . . .                                                                                                      | 32 |
| 23 | Speaker label after manual correction. . . . .                                                                                                       | 32 |
| 24 | Segment structure before merging or splitting. . . . .                                                                                               | 33 |
| 25 | Segment structure after merging or splitting. . . . .                                                                                                | 33 |
| 26 | Folder structure after export. . . . .                                                                                                               | 34 |
| 27 | Source data noise. . . . .                                                                                                                           | 35 |
| 28 | Example of forced alignment results with word-level labels for speaker VIVOSPK01. . . . .                                                            | 37 |
| 29 | Meeting simulation pipeline, Part 1. . . . .                                                                                                         | 40 |
| 30 | Meeting simulation pipeline, Part 2. . . . .                                                                                                         | 41 |
| 31 | Meeting simulation pipeline, Part 3. . . . .                                                                                                         | 42 |
| 32 | Actual overlap validation results for the TRAIN and VALID sets. . . . .                                                                              | 45 |
| 33 | Examples of overlapping speaker segments in the RTTM file. . . . .                                                                                   | 46 |
| 34 | Waveform comparison between a clean meeting and a noisy realistic meeting. . . . .                                                                   | 48 |
| 35 | RTTM label file (top) and JSON metadata statistics (bottom) for a simulated 2-speaker meeting with 10% overlap. . . . .                              | 51 |
| 36 | Distribution of the TRAIN and VAL sets across overlap levels and acoustic conditions. . . . .                                                        | 53 |
| 37 | Dataset directory structure. . . . .                                                                                                                 | 54 |
| 38 | Output on the VAL and TRAIN sets. . . . .                                                                                                            | 56 |
| 39 | Modified EEND Module Architecture in DiariZen . . . . .                                                                                              | 58 |
| 40 | Conformer block architecture . . . . .                                                                                                               | 59 |
| 41 | WavLM . . . . .                                                                                                                                      | 60 |
| 42 | PIT multilabel BCE loss in DiariZen . . . . .                                                                                                        | 60 |

|    |                                                                                                                                                                  |     |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| 43 | Three-phase gradual unfreezing fine-tuning strategy. . . . .                                                                                                     | 62  |
| 44 | Modified pyannote . . . . .                                                                                                                                      | 65  |
| 45 | Powerset segments . . . . .                                                                                                                                      | 66  |
| 46 | Powerset cross-entropy loss in Pyannote . . . . .                                                                                                                | 67  |
| 47 | PhoWhisper-large Encoder-Decoder Architecture . . . . .                                                                                                          | 70  |
| 48 | LoRA Principle and Unfreeze Positions in PhoWhisper . . . . .                                                                                                    | 71  |
| 49 | Cross-Entropy Loss . . . . .                                                                                                                                     | 76  |
| 50 | 3-stage pipeline with Smart-Skip mechanism. . . . .                                                                                                              | 77  |
| 51 | General DER Formula . . . . .                                                                                                                                    | 78  |
| 52 | Timeline illustration of the three DER error types . . . . .                                                                                                     | 79  |
| 53 | Miss% Formula . . . . .                                                                                                                                          | 79  |
| 54 | FA% Formula . . . . .                                                                                                                                            | 80  |
| 55 | Confusion% Formula . . . . .                                                                                                                                     | 80  |
| 56 | Overlap Precision, Recall, and F1-score Formulas . . . . .                                                                                                       | 80  |
| 57 | WER Formula . . . . .                                                                                                                                            | 81  |
| 58 | Overall Streamlit interface of the Vietnamese meeting-processing system. . . . .                                                                                 | 83  |
| 59 | Sidebar configuration panel with language, theme, and pipeline settings. . . . .                                                                                 | 84  |
| 60 | UI-backend architecture of the Streamlit-based meeting-processing system. . . . .                                                                                | 85  |
| 61 | Run tab for selecting the input source and execution mode. . . . .                                                                                               | 86  |
| 62 | Workspace view for transcript inspection and meeting-summary review. . . . .                                                                                     | 87  |
| 63 | Saved-session history for reopening and reviewing previous runs. . . . .                                                                                         | 88  |
| 64 | DiariZen Crop-level Validation DER by epoch. . . . .                                                                                                             | 89  |
| 65 | DiariZen fine-tuned: Overlap Precision/Recall/F1 and Predicted Segments by thresh-<br>old. Precision increases, Recall decreases as threshold increases. . . . . | 90  |
| 66 | DiariZen: Overlap Precision/Recall/F1 and Predicted Segments overall on Synthetic<br>Data by checkpoint. . . . .                                                 | 93  |
| 67 | DiariZen Overlap F1 and Predicted Segments by class on test_labeled. . . . .                                                                                     | 95  |
| 68 | Pyannote Crop-level Validation DER by epoch. . . . .                                                                                                             | 96  |
| 69 | Pyannote fine-tuned Threshold sweep on all representative samples. . . . .                                                                                       | 97  |
| 70 | Pyannote Overlap Precision/Recall/F1 and Predicted Segments overall on Synthetic Data<br>. Baseline F1=0.088, fine-tuned F1=0.555. . . . .                       | 98  |
| 71 | Pyannote DER components by speaker count on Synthetic Data. . . . .                                                                                              | 100 |
| 72 | Pyannote Overlap F1 and Predicted Segments by class on test_labeled. . . . .                                                                                     | 101 |
| 73 | Stage 1 Checkpoint-804 Epoch 1. . . . .                                                                                                                          | 101 |
| 74 | Stage 1 Checkpoint-1608 Epoch 2. . . . .                                                                                                                         | 102 |
| 75 | Stage 2 Checkpoint-965 Epoch 1. . . . .                                                                                                                          | 102 |
| 76 | Combined All Stages. . . . .                                                                                                                                     | 103 |
| 77 | WER progression across fine-tuning stages (test set, 1,655 samples). . . . .                                                                                     | 104 |
| 78 | WER by domain: Base / Stage 1 / Stage 2. . . . .                                                                                                                 | 105 |
| 79 | Automated evaluation results across five datasets. . . . .                                                                                                       | 107 |
| 80 | Train loss comparison of two models. . . . .                                                                                                                     | 116 |
| 81 | Validation loss comparison of two models . . . . .                                                                                                               | 116 |
| 82 | Crop-level validation DER comparison of two models. . . . .                                                                                                      | 117 |
| 83 | Pyannote (Best model). . . . .                                                                                                                                   | 117 |
| 84 | DiariZen (Best model). . . . .                                                                                                                                   | 118 |

## List of Tables

|    |                                                                                                  |    |
|----|--------------------------------------------------------------------------------------------------|----|
| 1  | Problem Definition of the Proposed Vietnamese Multi-Speaker Speech System . . . . .              | 4  |
| 2  | Project Scope and Current Limitations. . . . .                                                   | 6  |
| 3  | Team task allocation. . . . .                                                                    | 7  |
| 4  | Overview of communication frequency and patterns. . . . .                                        | 7  |
| 5  | Main communication and coordination tools used in the project. . . . .                           | 8  |
| 6  | Comparative Analysis of Component Models. . . . .                                                | 11 |
| 7  | Comparative Analysis: Commercial Platforms vs. Proposed Integrated Framework. . . .              | 12 |
| 8  | Speech datasets relevant to the proposed Vietnamese multi-speaker transcription project.         | 14 |
| 9  | Dataset usage overview in the proposed system. . . . .                                           | 23 |
| 10 | Public Vietnamese speech corpora used as source data for synthetic data generation. .            | 23 |
| 11 | YouTube dataset summary for Self-labeled data. . . . .                                           | 24 |
| 12 | Technical specifications of the forced alignment process. . . . .                                | 36 |
| 13 | Alignment quality validation criteria. . . . .                                                   | 39 |
| 14 | Actual alignment quality statistics after cleaning. . . . .                                      | 39 |
| 15 | Estimated time allocation ratio for the simulated meeting timeline. . . . .                      | 43 |
| 16 | Actual overlap validation results for the TRAIN set (399 meetings, 210.34 h). . . . .            | 44 |
| 17 | Actual overlap validation results for the VAL set (63 meetings, 33.19 h). . . . .                | 44 |
| 18 | Room simulation parameters. . . . .                                                              | 47 |
| 19 | Default system configuration parameters. . . . .                                                 | 49 |
| 20 | Configuration parameters for train. . . . .                                                      | 49 |
| 21 | Configuration parameters for val. . . . .                                                        | 50 |
| 22 | Evaluation purposes and tools for each labeling format. . . . .                                  | 51 |
| 23 | Actual statistics for the TRAIN set, 18 speakers. . . . .                                        | 52 |
| 24 | Actual statistics for the VAL set, 40 speakers. . . . .                                          | 52 |
| 25 | General statistics of the training and validation datasets. . . . .                              | 57 |
| 26 | Self-labeled data distribution. . . . .                                                          | 61 |
| 27 | Synthetic Data distribution data_mix. . . . .                                                    | 62 |
| 28 | Detailed fine-tuning configuration: DiariZen (BUT-FIT/diarizen-wavlm-large-s80-md-v2). . . . .   | 63 |
| 29 | Detailed fine-tuning configuration: Pyannote (pyannote/speaker-diarization-community-1). . . . . | 68 |
| 30 | PhoWhisper-large Architecture Components. . . . .                                                | 70 |
| 31 | Partial Unfreezing / LoRA Configuration. . . . .                                                 | 71 |
| 32 | Data Distribution by Domain and Quality Bucket. . . . .                                          | 72 |
| 33 | Self-labeled data distribution. . . . .                                                          | 72 |
| 34 | Synthetic Data distribution. . . . .                                                             | 73 |
| 35 | Detailed fine-tuning configuration for PhoWhisper-large. . . . .                                 | 74 |
| 36 | Segment-aware Chunking Rules. . . . .                                                            | 76 |
| 37 | Qwen 2.5-1.5B-Instruct Deployment Configuration. . . . .                                         | 77 |
| 38 | Block Segmentation Parameters. . . . .                                                           | 78 |
| 39 | Crop-level DER and File-level DER. . . . .                                                       | 81 |
| 40 | Definition of 3 WER Error Types. . . . .                                                         | 81 |
| 41 | Automated Evaluation Metrics. . . . .                                                            | 82 |
| 42 | LLM Judge Criteria (Qwen 7B). . . . .                                                            | 82 |
| 43 | DiariZen Fine-tuned DER (%) by threshold on test_labeled set (★ = selected threshold).           | 91 |

|    |                                                                                                             |     |
|----|-------------------------------------------------------------------------------------------------------------|-----|
| 44 | DiariZen Fine-tuned DER / Miss / FA / Conf by threshold on Synthetic Data (★ = selected threshold). . . . . | 92  |
| 45 | DiariZen Overall Results on Synthetic Data Set by Checkpoint (thr=0.25). . . . .                            | 93  |
| 46 | DiariZen DER / Miss / FA / Conf by Condition and Model (thr=0.25). . . . .                                  | 94  |
| 47 | DiariZen DER / Miss / FA / Conf by Speaker Count on Synthetic Data (thr=0.25). . . . .                      | 94  |
| 48 | DiariZen: DER / Miss / FA / Conf on test_labeled Set (best model, thr=0.25). . . . .                        | 95  |
| 49 | Pyannote Overall Results on Synthetic Data by Checkpoint. . . . .                                           | 97  |
| 50 | Pyannote DER / Miss / FA / Conf by Condition and Model. . . . .                                             | 99  |
| 51 | Pyannote DER / Miss / FA / Conf by Speaker Count on Synthetic Data. . . . .                                 | 99  |
| 52 | Pyannote DER / Miss / FA / Conf on test_labeled Set (best model). . . . .                                   | 100 |
| 53 | Global WER Comparison of Base / Stage 1 / Stage 2 (1,655 test samples). . . . .                             | 104 |
| 54 | WER Comparison of base model and checkpoints. . . . .                                                       | 105 |
| 55 | Runtime Performance RTX 3090 Ti, FP16. . . . .                                                              | 105 |
| 56 | Automated Evaluation Results. . . . .                                                                       | 107 |
| 57 | Overall Formula Automated Evaluation. . . . .                                                               | 107 |
| 58 | LLM Judge Results Using Qwen 7B. . . . .                                                                    | 108 |
| 59 | VRAM and Tokens/s Benchmark. . . . .                                                                        | 108 |
| 60 | Comparison DiariZen vs Pyannote (Best model). . . . .                                                       | 119 |
| 61 | Five implemented mechanisms for reducing hallucination. . . . .                                             | 120 |
| 62 | Summary of achievements and limitations by system component. . . . .                                        | 123 |

# 1 Introduction

## 1.1 Background

This project addresses the problem of converting noisy Vietnamese multi-speaker recordings into structured and usable text. In meetings, academic discussions, online calls, interviews, and internal reviews, information is often still recorded manually or extracted by listening back to long audio files. This process is slow and error-prone. Important details may be missed, and statements may be assigned to the wrong speaker.

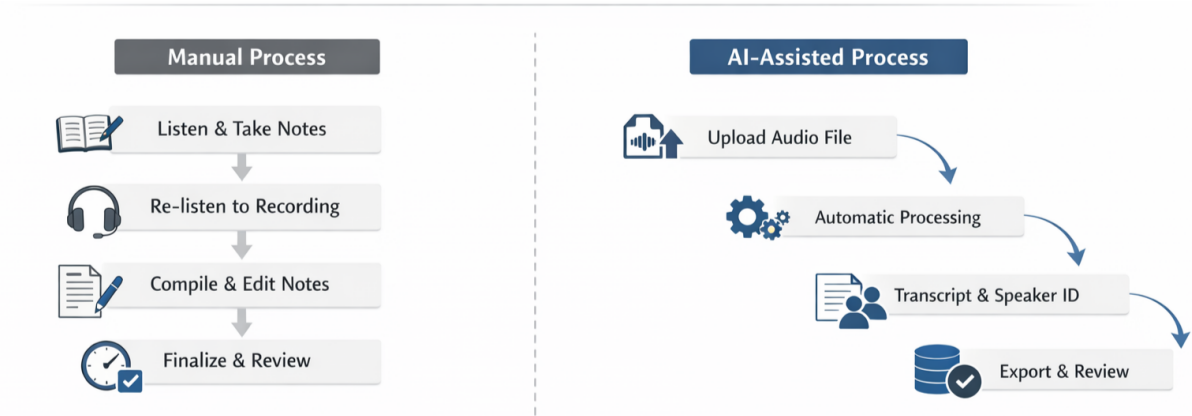


Figure 1: Comparison of manual and AI-assisted meeting documentation workflows.

The task becomes more difficult in real Vietnamese conversations. Speakers may interrupt each other, speak quickly, overlap in short segments, or change speaking style depending on context. Recordings may also contain background noise, reverberation, keyboard sounds, music, low-quality microphones, or packet loss from online calls. These factors reduce the reliability of both speaker diarization and automatic speech recognition [1, 2, 3].

Vietnamese adds further difficulty because it is a tonal language. Noise, overlap, and poor recording quality can affect tone information and reduce recognition accuracy. Regional accents, speaking speed variation, informal expressions, and Vietnamese-English code-switching also make the task harder than clean single-speaker transcription [4].

A useful meeting transcript must contain not only the recognized words, but also the speaker labels and timestamps. In other words, the system must answer what was said, who said it, and when it was said [1, 2]. This is especially important in overlap regions, where more than one speaker may be active at the same time. If the diarization stage misses these regions, the error can affect transcription, speaker attribution, and summarization.

The project was selected because existing speech-to-text systems are often not sufficient for noisy Vietnamese meeting audio. Clean-speech ASR systems cannot fully handle realistic conversations with overlap, noise, and multiple speakers. Therefore, this work treats meeting transcription as an integrated pipeline that combines diarization, speaker identification, ASR, and summarization.

From an application perspective, the system can reduce manual note-taking effort and make meeting content easier to review, search, and export. From a research perspective, the project is meaningful because Vietnamese multi-speaker transcription under noisy and overlap-prone conditions remains

challenging. It requires the integration of several tasks that are often handled separately, including overlap-aware diarization, speaker representation, speaker identification, and timestamped ASR.

The project also considers both synthetic and real recordings. Synthetic meeting-style data helps control speaker count, overlap ratio, and noise level, while real recordings expose problems that may not appear in controlled data. This combination allows the system to be evaluated under both experimental and practical conditions.

## 1.2 Objective

The overall objective of this project is to design, implement, and evaluate a deep learning framework for Vietnamese multi-speaker transcription and speaker identification in noisy environments. The framework is intended to process real-world meeting recordings and generate structured outputs that are useful for documentation, review, and later analysis.

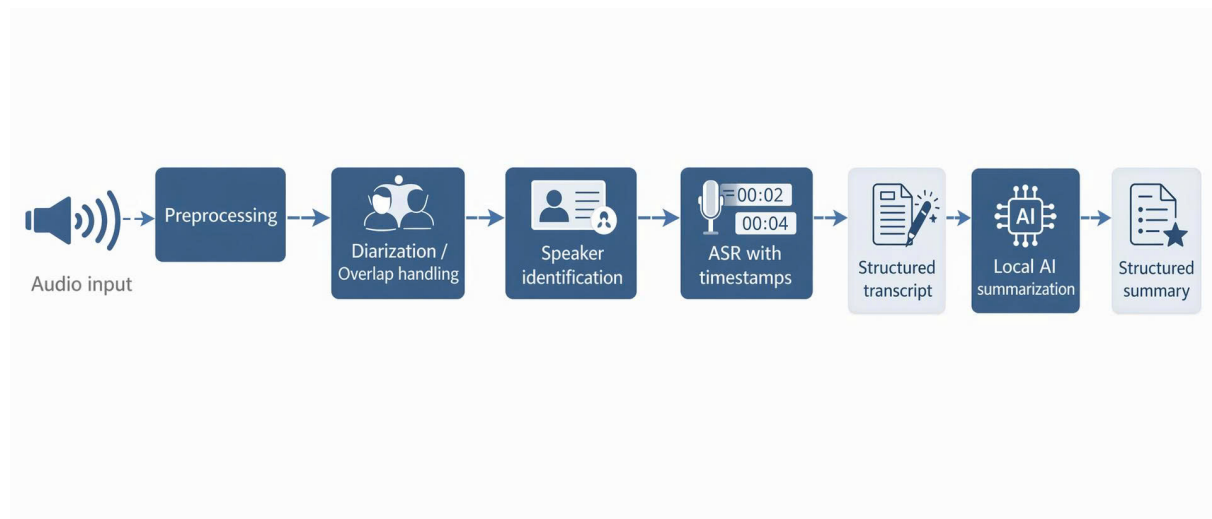


Figure 2: Overview of the proposed system pipeline.

The specific objectives of the project are as follows:

1. To develop an audio preprocessing pipeline that standardizes input recordings and improves signal quality through operations such as resampling, normalization, denoising, and dereverberation, so that downstream modules can operate more reliably in noisy meeting conditions.
2. To build or adapt a speaker diarization module capable of determining who spoke when in Vietnamese multi-speaker conversations, including difficult cases with short speaker turns and overlapping speech.
3. To integrate a speaker identification mechanism that compares speech segments with enrolled voice samples in order to recognize known participants; for speakers without enrollment data, the system should assign consistent anonymous speaker labels.
4. To implement an automatic speech recognition module for converting Vietnamese speech into text with timestamps, while maintaining robustness to accent variation, spontaneous speaking style, noise, and code-switching between Vietnamese and English.

5. To combine diarization and transcription outputs into a structured transcript format containing at least the start time, end time, speaker label, recognized text, and confidence-related information, so that the result can be stored, reviewed, and edited efficiently.
6. To support optional post-processing and summarization by using natural language processing or large language models to produce concise meeting summaries, key discussion points, and action items from the generated transcript.
7. To develop a prototype application interface that allows users to upload or record audio, monitor processing progress, inspect transcript segments, edit speaker labels or text, and export the final result in practical formats.
8. To evaluate the effectiveness of the proposed framework using appropriate quantitative metrics. The diarization component will be assessed with measures such as Diarization Error Rate (DER) and its subcomponents, the transcription component will be assessed with Word Error Rate (WER) and Character Error Rate (CER), the speaker identification component will be analyzed using embedding similarity or identification consistency, and the optional summarization component will be evaluated using content coverage and summary quality measures.

### 1.3 Problem Statement

In real meeting environments, spoken conversation is difficult to convert into accurate structured text. Multiple speakers may speak in quick succession, interrupt each other, or talk at the same time. Background noise, room reverberation, and microphone variation further reduce audio quality. For Vietnamese, the problem is harder because tonal information, regional accents, informal expressions, and mixed-language speech can affect recognition accuracy.

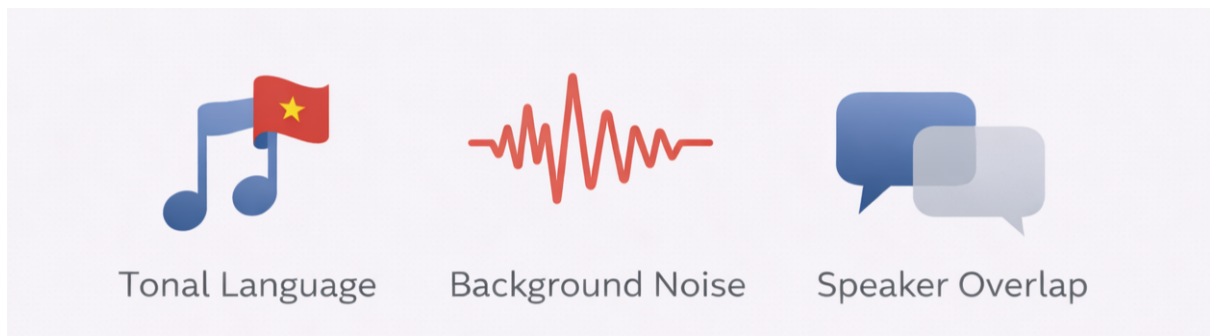


Figure 3: Main sources of difficulty in Vietnamese multi-speaker meeting transcription, including tonal speech, background noise, and speaker overlap.

The project targets the transformation of unstructured Vietnamese meeting audio into structured output. The input is a meeting-scale audio recording collected from a physical or online environment. The expected output is a transcript with time boundaries, speaker labels, and recognized text. In later stages, this output can also support summarization and action-item extraction.

The main technical problem is error propagation across the pipeline. If overlapping speech is missed or speaker boundaries are incorrect, later modules receive incomplete or wrongly segmented audio. This can lead to missing words, incorrect speaker labels, fragmented utterances, and incomplete summaries. Therefore, overlap handling and speaker attribution must be treated as core parts of the system.

The proposed system is designed as an integrated multi-stage pipeline. It detects speech activity, segments audio into speaker-attributed regions, identifies enrolled speakers when possible, assigns anonymous labels to unknown speakers, and generates timestamped text for each segment. This means the problem is not only speech-to-text, but speaker-aware transcription under noisy and overlap-prone conditions.

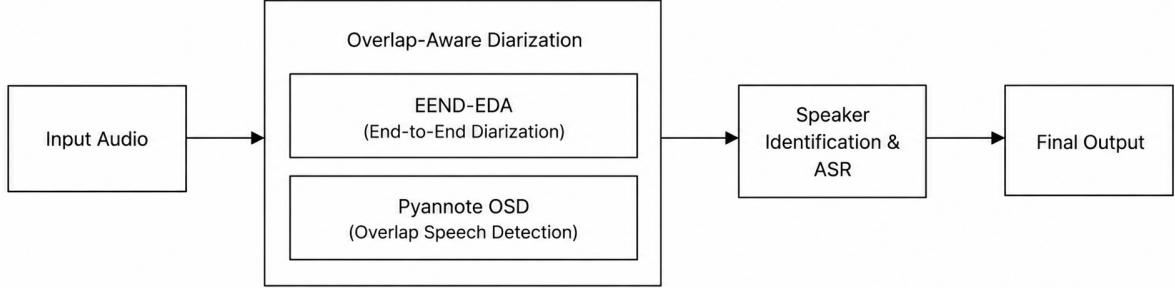


Figure 4: Simplified overview of the redesigned overlap-aware pipeline used in the proposed system.

Speaker attribution is another important part of the problem. The system is not limited to anonymous speaker segmentation. When enrollment recordings are available, it attempts to assign speech segments to known speakers. When enrollment data is unavailable, it still preserves the structure of the conversation by assigning anonymous speaker identifiers. This makes the system more suitable for real meeting scenarios, where some participants may be known while others are not.

The project also considers data realism. Synthetic data is useful for controlling overlap ratio, speaker count, noise level, and segment distribution. Real recordings are needed to test the system under less predictable conditions. Therefore, the project uses both controlled data and real-world Vietnamese recordings to evaluate the pipeline.

Table 1: Problem Definition of the Proposed Vietnamese Multi-Speaker Speech System

| Aspect              | Description                                                                                                                                 |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Input               | Noisy Vietnamese multi-speaker meeting audio recorded in physical or on-line environments.                                                  |
| Core challenges     | Overlapping speech, background noise, reverberation, speaker attribution, and Vietnamese accent variability.                                |
| Target output       | A structured transcript containing timestamps, speaker labels, text content, and overlap tags when applicable.                              |
| Identification mode | If enrollment audio is available, the system assigns known speaker identities; otherwise, it assigns anonymous speaker IDs.                 |
| System type         | A modular deep learning research pipeline integrating overlap-aware diarization, speaker embedding and identification, and timestamped ASR. |

## 1.4 Scope & Limitations

The scope of this project is the design and implementation of an experimental deep learning framework for Vietnamese multi-speaker transcription and identification in noisy environments. The system focuses on recorded meeting-style audio rather than arbitrary general audio. Its main functions include audio preprocessing, speaker diarization, speaker identification, timestamped ASR, transcript generation, and optional summarization.

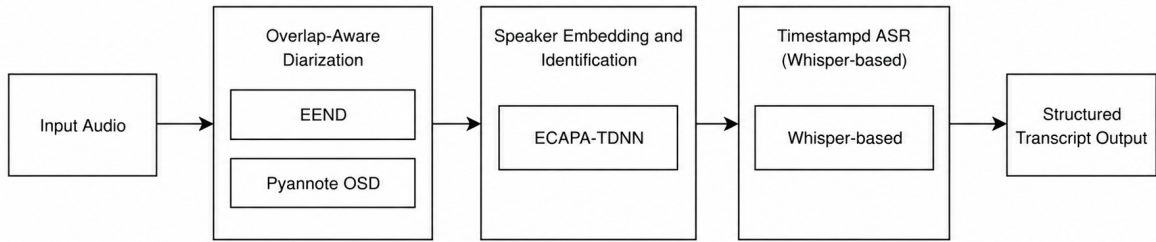


Figure 5: System scope and boundary of the proposed research prototype, including the main in-scope modules and current out-of-scope limitations.

The project focuses on full-pipeline integration instead of isolated model testing. Diarization results are used to support speaker-aware ASR, while speaker identification helps assign known or anonymous labels to speech segments. This modular design makes the system easier to inspect, debug, and improve component by component.

The data scope includes both synthetic and collected recordings. Synthetic meeting-style data is used to create controlled conditions with different speaker counts, overlap ratios, and noise levels. Real recordings are used to evaluate the system under more practical conditions. However, the data does not fully represent all Vietnamese accents, speaking styles, microphone conditions, and acoustic environments.

A major strength of the project is that it directly addresses overlapping speech, which is one of the main causes of errors in meeting transcription. The system is also designed to support speaker-aware output through enrollment samples, making it more useful for repeated meetings or known-team scenarios.

However, the project is not a production-grade meeting intelligence platform. It is better understood as a research prototype for offline processing and evaluation. Real-time streaming, large-scale serving, and fully hardened deployment are outside the current validated scope.

Several limitations remain. First, heavy overlapping speech is still difficult, especially when multiple speakers talk at the same time for long periods. Second, available Vietnamese data is still limited compared with English resources, especially for overlap-rich diarization and speaker-aware ASR.

Third, regional accents and Vietnamese-English code-switching may reduce system performance if they are not well represented in the data. Fourth, speaker identification depends on the quality and availability of enrollment audio. If enrollment samples are missing or poor in quality, the system can only assign anonymous speaker labels.

Finally, because the system is modular, errors can still propagate between stages. Incorrect diarization can affect speaker matching, ASR, timestamp quality, and summarization. The proposed pipeline reduces some of these problems by handling speaker structure explicitly, but it does not remove inter-stage dependency completely.

Overall, the project scope is appropriate for a research prototype. It covers the full path from noisy Vietnamese meeting audio to structured speaker-aware transcription, while its main limitations are related to data coverage, accent diversity, heavy overlap, enrollment quality, and offline-oriented evaluation.

Table 2: Project Scope and Current Limitations.

| Category          | Included in Scope                                                                                                | Outside Scope / Current Limitation                                                                  |
|-------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Audio processing  | Offline processing of Vietnamese meeting-style audio in noisy environments.                                      | Real-time streaming deployment is not the main validated target of the current system.              |
| Speaker handling  | Speaker diarization and speaker identification with support for enrolled speakers and anonymous fallback labels. | Speaker naming accuracy depends on the availability and quality of enrollment audio.                |
| Overlap handling  | Explicit overlap-aware processing through DiariZen and pyannote.audio-based diarization backends [2, 5].         | Performance under heavy real-world overlap is improved but not yet fully reliable.                  |
| Language coverage | Vietnamese-oriented speech processing with attention to practical meeting conditions.                            | Generalization across all regional accents and mixed-language speech remains limited.               |
| System design     | Modular research pipeline with separable components for diarization, identification, and ASR.                    | Large-scale production serving and fully hardened deployment are outside the current project scope. |

## 2 Project Management Plan

### 2.1 Team Structure and Roles

The project team consists of four members, who equally share responsibilities to ensure a balanced workload and effective collaboration throughout the project. Each member contributes to all phases of the project, from research to implementation, ensuring mutual involvement and accountability as detailed in Table 3.

Table 3: Team task allocation.

| Member | Main Task                            | Specific Responsibilities                                                                         |
|--------|--------------------------------------|---------------------------------------------------------------------------------------------------|
| Sang   | Tool Development and Manual Labeling | Find and build the labeling tool; write supporting scripts for labeling; manually label the data. |
| Duy    | Documentation and Manual Labeling    | Write the report and technical documentation; manually label the data together with Sang.         |
| Phuong | Synthetic Data and ASR               | Create the synthetic dataset; work on the ASR component and the Qwen model.                       |
| Khang  | Pipeline and Diarization             | Build the main processing pipeline; work on the two diarization models: DiariZen and Pyannote.    |

### 2.2 Communication Plan

The communication plan ensures efficient and consistent interaction between team members and the project supervisor. Communication is maintained through regular updates, discussions, and feedback sessions to monitor progress and resolve technical challenges. The communication patterns are summarized in Table 4.

Table 4: Overview of communication frequency and patterns.

| Frequency       | Details                                                                                                   |
|-----------------|-----------------------------------------------------------------------------------------------------------|
| Daily Check-ins | Team members provide brief updates and address any immediate issues via Discord or Zalo.                  |
| Weekly Meetings | Detailed progress reviews and advisory sessions are conducted with the project supervisor in Google Meet. |

To support collaboration, the team utilizes the tools described in Table 5.

Table 5: Main communication and coordination tools used in the project.

| <b>Tool</b>  | <b>Main Purpose</b>                       | <b>Frequency</b>   | <b>Details of Use</b>                                                                                                                                        |
|--------------|-------------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Discord      | Daily team discussion                     | Daily / as needed  | Used for day-to-day technical discussion, quick coordination between members, task updates, and short problem solving exchanges during implementation.       |
| GitHub       | Code sharing and version control          | Continuous         | Used to share source code, manage version history, track changes in scripts and configurations, and coordinate collaborative development among team members. |
| Google Drive | Shared storage                            | Continuous         | Used to store shared datasets, experiment results, report materials, and other working documents that need to be accessed by all members.                    |
| Zalo         | Communication and notifications           | As needed          | Used for direct communication, announcements, reminders, and information exchange with the supervisor and among team members when a fast response is needed. |
| Google Meet  | Communication with the project supervisor | Weekly / as needed | Used for weekly progress reports and formal consultations with the supervisor.                                                                               |

This structured communication ensures alignment within the team and allows timely intervention from the supervisor when required, facilitating the successful execution of the project.

## 3 Literature Review

### 3.1 Overview

Speech technology has developed significantly over the past decade. Earlier systems were usually designed for clean, single-speaker speech, where one person spoke at a time and the recording conditions were relatively controlled. In that setting, the main goal was to convert speech into text as accurately as possible. This task was already difficult, but the assumptions were simpler. The system did not need to decide who was speaking, whether two speakers were active at the same moment, or how to preserve the structure of a real conversation.

Meeting transcription is more difficult because it removes those simplifying assumptions. In a meeting, speech is not produced as a neat sequence of isolated utterances. Speakers interrupt one another, speak briefly, overlap in short regions, or respond while another speaker is still active. In addition, the recording environment is often noisy. A meeting can contain keyboard sounds, fan noise, packet loss from online calls, background speech, and room reverberation. As a result, a model that performs well on clean single-speaker audio does not automatically perform well on conversational recordings.

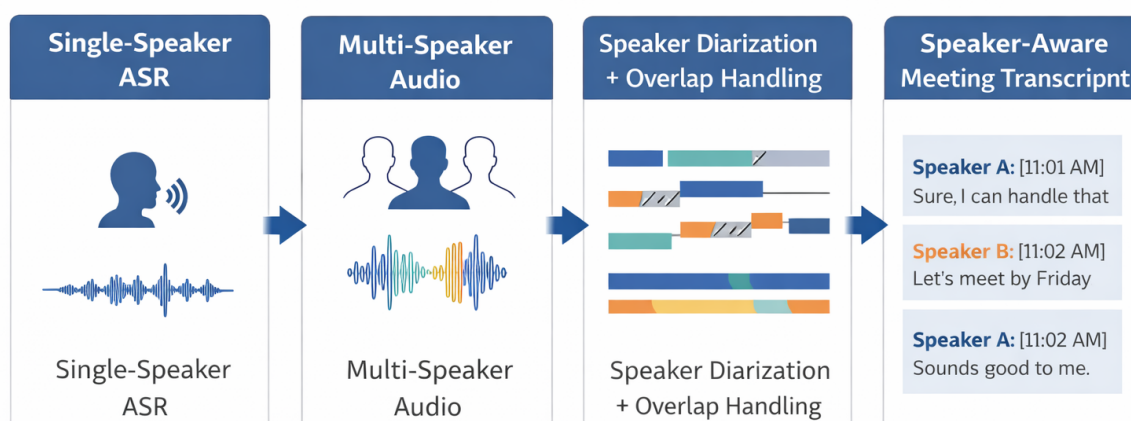


Figure 6: From single-speaker speech recognition to speaker-aware meeting transcription.

For this reason, modern meeting transcription systems are no longer built around ASR alone. They usually combine several modules, including preprocessing, speaker diarization, speaker embeddings or speaker identification, overlap handling, and speech recognition with timestamps. In practical use, these modules must work together. If diarization fails, the transcript may still contain words, but the speaker labels may be wrong. If overlap is missed, some speech content may be omitted. If segmentation is unstable, the final transcript becomes fragmented and harder to review.

The literature relevant to this study is not limited to one family of models. It covers several connected research areas that together support speaker-aware meeting transcription. The most important of these areas are speaker diarization, automatic speech recognition, and overlap handling. They are often presented separately in the literature, but in real systems they interact directly. That interaction is one of the reasons why meeting transcription remains a difficult problem.

In the Vietnamese setting, the difficulty increases further. Vietnamese meeting audio is affected not only by overlap and background noise, but also by tonal variation, regional accents, and frequent insertion of English terms in technical or workplace conversations. Therefore, this literature review not only summarizes existing speech technologies in general. It also examines how well these technologies match the specific requirements of Vietnamese multi-speaker meeting transcription.

### **3.2 State-of-the-Art Models and Existing Approaches**

#### **System Overview.**

The system is designed as a three-stage processing chain: speaker diarization, automatic speech recognition (ASR), and local AI summarization.

#### **Stage 1: Speaker Diarization.**

In the first stage, the system analyzes the incoming audio signal to determine who spoke when, producing a speaker timeline across the recording. The goal of this step is not to recognize speech content, but to segment the conversation into speaker-labeled utterances that serve as the foundation for all subsequent processing stages.

#### **Stage 2: Speaker-Aware ASR.**

Once diarization results are available, the system proceeds to the ASR stage, converting each diarized audio segment into text. By leveraging diarization output, the resulting transcript captures not only the spoken content but also the corresponding timestamps and speaker labels. This is especially important in multi-speaker conversation scenarios, where the final transcript must reflect not just what was said, but who said it.

#### **Stage 3: Local AI Summarization.**

Building on the speaker-structured transcript, the system performs local LLM-based summarization. A locally hosted language model is used to normalize the transcript, extract key points, and generate a final summary. Running the model locally makes the system more suitable for real-world deployment requirements while keeping the entire processing flow within a single unified pipeline from raw audio input to final output.

#### **End-to-End Flow.**

The overall flow can be summarized as:

Audio Input → Speaker Diarization → Speaker-Aware ASR → Local Summary

The final output is not merely a speaker-labeled transcript, but a structured summary that makes conversational content easier to read, follow, and use for downstream information extraction tasks.

Table 6: Comparative Analysis of Component Models.

| Stage                      | Representative Model                                 | Strength                                                                                | Limitation                                                                          |
|----------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>Speaker Diarization</b> | pyannote.audio                                       | Modular, RTTM-compatible, strong multi-speaker baseline                                 | Sensitive to overlap and threshold tuning                                           |
| <b>ASR</b>                 | Whisper / PhoWhisper                                 | Multilingual, timestamped output; PhoWhisper optimized for Vietnamese                   | Speaker structure not preserved natively; quality depends on diarization boundaries |
| <b>Local AI Summary</b>    | Qwen2.5-1.5B-Instruct                                | Runs locally, good instruction following, practical for normalization and summarization | Summary quality bounded by transcript quality; may miss nuance in long contexts     |
| <b>Integrated Pipeline</b> | Diarization to Speaker-Aware ASR to Local AI Summary | End-to-end: speaker timeline, labeled transcript, and final summary                     | Errors propagate across stages; weak upstream components reduce overall reliability |

### 3.3 Vietnamese Speech Processing Context

Vietnamese speech processing presents several challenges that are not fully captured by general multilingual benchmarks. The first challenge is lexical tone. Vietnamese is a tonal language, so phonetic identity is closely tied to pitch contour. When audio quality degrades or when two voices overlap, tonal information becomes less reliable, and recognition errors increase. This means that noisy or overlapped speech is particularly harmful for Vietnamese ASR.

The second difficulty is accent diversity. Vietnamese speech varies substantially across northern, central, and southern regions, and this variation affects pronunciation, speaking rhythm, and acoustic realization. A model trained mostly on one accent group may not generalize evenly to another. This is one reason why adaptation to Vietnamese data remains important even when large multilingual models are available.

The third difficulty is code-switching. In workplace conversations, technical meetings, and informal online discussions, Vietnamese speakers often insert English words, abbreviations, and borrowed expressions. This type of mixed speech is common in real usage, but it is not always well represented in older clean read-speech corpora. As a result, a model that performs well on read Vietnamese may still be unstable in realistic meetings.

Current Vietnamese speech research and development already shows progress in this direction. PhoWhisper is an important example because it adapts the Whisper family to Vietnamese and emphasizes more suitable Vietnamese speech coverage. Industrial platforms in Vietnam also show that speech technology has practical demand in the local context. In earlier project materials, systems such as FPT.AI, Viettel AI, and VAIS were presented as examples of existing Vietnamese or regional solutions with different strengths. These systems demonstrate that Vietnamese ASR and speech services are already active in both academic and applied settings.

Table 7: Comparative Analysis: Commercial Platforms vs. Proposed Integrated Framework.

| System / Platform | Deployment Focus                               | Overlap Resilience                                                              | Key Limitation in Meeting Context                                                                        |
|-------------------|------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>FPT.AI</b>     | Virtual assistants and automated call centers. | Optimized for sequential, turn-based dialogue.                                  | May be less suitable for spontaneous speaker interruptions if explicit overlap handling is not provided. |
| <b>Viettel AI</b> | Enterprise-grade and government solutions.     | May require controlled acoustic conditions depending on the deployment setting. | Primarily designed for formal speech with controlled acoustic conditions.                                |
| <b>VAIS</b>       | Automated meeting minutes and media.           | Post-processing based speaker separation.                                       | Modular architecture may lead to error propagation from diarization to ASR.                              |

At the same time, most existing Vietnamese speech resources and products still perform best in cleaner conditions than in overlap-rich meetings. Many available datasets are dominated by read speech, benchmark speech, or carefully collected single-speaker material. Those resources are useful, but they do not fully represent the conversational structure of spontaneous meetings. This is the main reason why Vietnamese multi-speaker meeting transcription remains a more open problem than plain Vietnamese speech recognition.

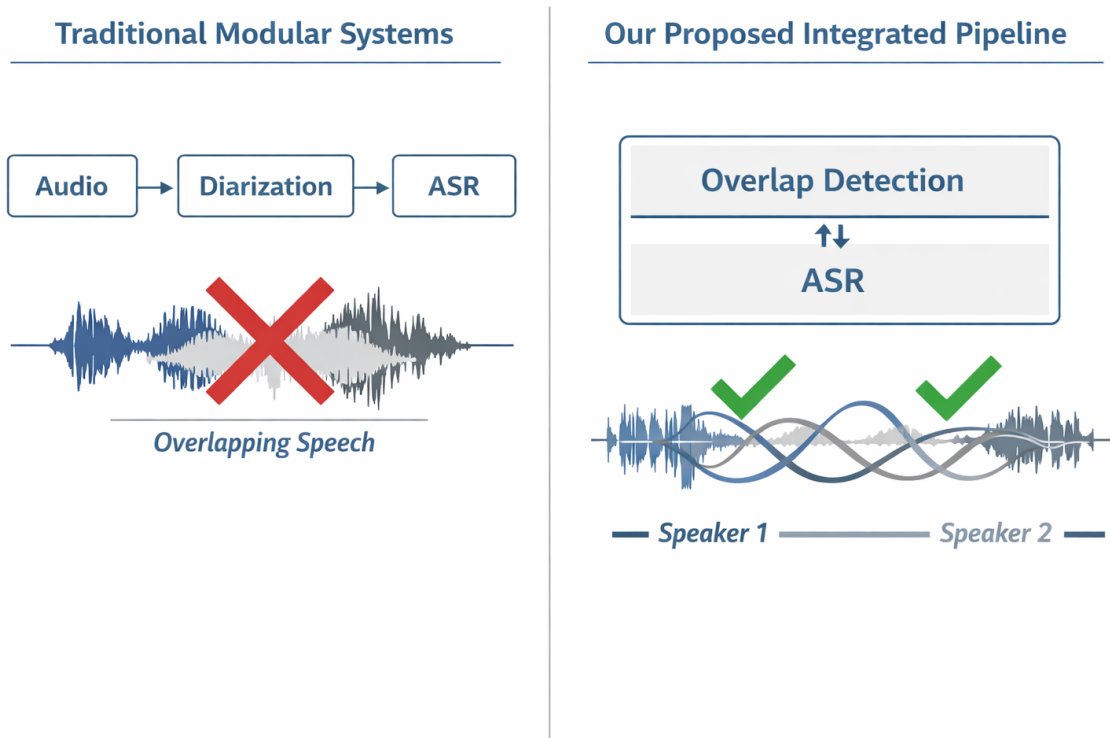


Figure 7: Proposed Integrated Framework for Vietnamese Multi-speaker Transcription.

### 3.4 Knowledge Gap

The literature shows that strong components already exist. There are powerful ASR models. There are strong diarization models. There are also methods for speaker embedding, target-speaker detection, and overlap-aware inference. However, these components do not automatically form a reliable solution for noisy Vietnamese meeting audio.

The first gap is language-specific. Many strong models were developed and tested in settings where training data is richer, cleaner, or less varied than Vietnamese meeting recordings. When the target shifts to Vietnamese speech with regional accents, tone sensitivity, office noise, and code-switching, performance becomes less stable. A model may still work, but not as reliably as it does in benchmark conditions.

The second gap is architectural. In many existing systems, ASR and diarization are still handled as separate modules. This modular design is practical, but it creates error propagation. If overlap is missed, ASR may lose part of the content. If speaker boundaries are unstable, the final transcript may be attributed to the wrong speaker. In other words, the quality of one stage directly influences the others. This is especially serious in meeting transcription, where correctness is not measured only by text accuracy but also by speaker attribution and time structure.

The third gap is data-related. Public Vietnamese datasets such as VIVOS, VLSP, and other common speech resources are valuable, but they are not built specifically for overlap-heavy multi-speaker meetings. They do not fully capture natural interruption patterns, short backchannels, or speaker competition in the same time interval. Because of this, a project that targets Vietnamese meeting transcription cannot rely entirely on public corpora if it wants to study overlap handling in a realistic way.

This leads to the central gap addressed by the present work. There are still few publicly available frameworks that are clearly optimized for Vietnamese meeting-style audio from raw recordings to structured transcripts with speaker labels, timestamps, and overlap-aware processing. The existing literature provides strong pieces. The missing part is their integration under the acoustic and linguistic conditions of Vietnamese multi-speaker conversation. That is why this project proposes a specialized integrated framework rather than a single isolated model.

### 3.5 Datasets Relevant to This Project

Several public datasets are relevant to Vietnamese speech processing, but their suitability for the present task is uneven. VIVOS is one of the best known Vietnamese speech corpora and has been widely used in ASR research. It is open, relatively clean, and useful for controlled experimentation [6]. However, it is fundamentally a read-speech corpus, not a meeting corpus. It provides clear speech material, but it does not naturally contain multi-speaker interaction or realistic overlap.

LSVSC resources are also important because they have supported Vietnamese speech benchmarks and community evaluation [7]. They are useful for model development and benchmarking, especially in ASR-related tasks. However, like many benchmark-oriented corpora, they do not fully represent spontaneous overlap-rich conversational recordings.

Common Voice is another useful public resource because it expands data coverage and helps multilingual or low-resource speech research [8]. Still, it also leans more toward single-speaker or independently recorded utterances than real multi-speaker meetings. As a result, it is more appropriate

for ASR adaptation than for direct overlap-meeting diarization research.

More recent Vietnamese resources move closer to conversational speech. Some datasets focus on broader accent coverage. Others reflect domain-specific or conversational settings better than traditional read corpora. These developments are important because they show that Vietnamese speech resources are improving. Even so, they still do not fully replace a dedicated overlap-heavy meeting corpus with diarization labels, speaker structure, and controlled overlap conditions.

This limitation explains why synthetic data becomes necessary in the present project. If public datasets do not provide the right combination of Vietnamese speech, multiple speakers, overlap, noise, and speaker annotations, then synthetic meeting generation is a reasonable design choice. It allows the project to control overlap ratio, speaker combinations, and acoustic conditions while still building on Vietnamese speech material. In that sense, Synthetic data is not used as a replacement for real data, but as a controlled supplement to address a specific resource gap.

Table 8: Speech datasets relevant to the proposed Vietnamese multi-speaker transcription project.

| <b>Dataset</b>                      | <b>Language</b> | <b>Speech Type</b>                         | <b>Overlap Support</b> | <b>Main Usefulness / Limitation</b>                                                                                                                                      |
|-------------------------------------|-----------------|--------------------------------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VIVOS                               | Vietnamese      | Read speech                                | No                     | Useful as a clean Vietnamese speech resource, but not suitable for natural multi-speaker meetings                                                                        |
| LSVSC                               | Vietnamese      | Benchmark speech                           | No                     | Useful for ASR evaluation, but not designed for overlap-heavy conversation                                                                                               |
| Common Voice                        | Multilingual    | Short utterances                           | No                     | Helpful for ASR adaptation, but not suitable for diarization-focused meeting tasks                                                                                       |
| Conversational Vietnamese resources | Vietnamese      | Conversational speech                      | Limited                | Closer to real usage, but still limited for overlap-rich meetings                                                                                                        |
| Synthetic meeting data              | Vietnamese      | Simulated meetings                         | Yes                    | Suitable for controlled experiments on overlap, speaker count, and noise                                                                                                 |
| Speech Noise Dataset [9]            | Not specified   | Clean/noisy speech + noise-only recordings | No                     | Useful for speech enhancement, noise reduction, and robust ASR under noisy conditions, but not suitable for speaker diarization or overlap-focused meeting transcription |

## 4 Methodology

### 4.1 Inference Pipeline: Real-world Deployment

#### 4.1.1 End-to-End Pipeline

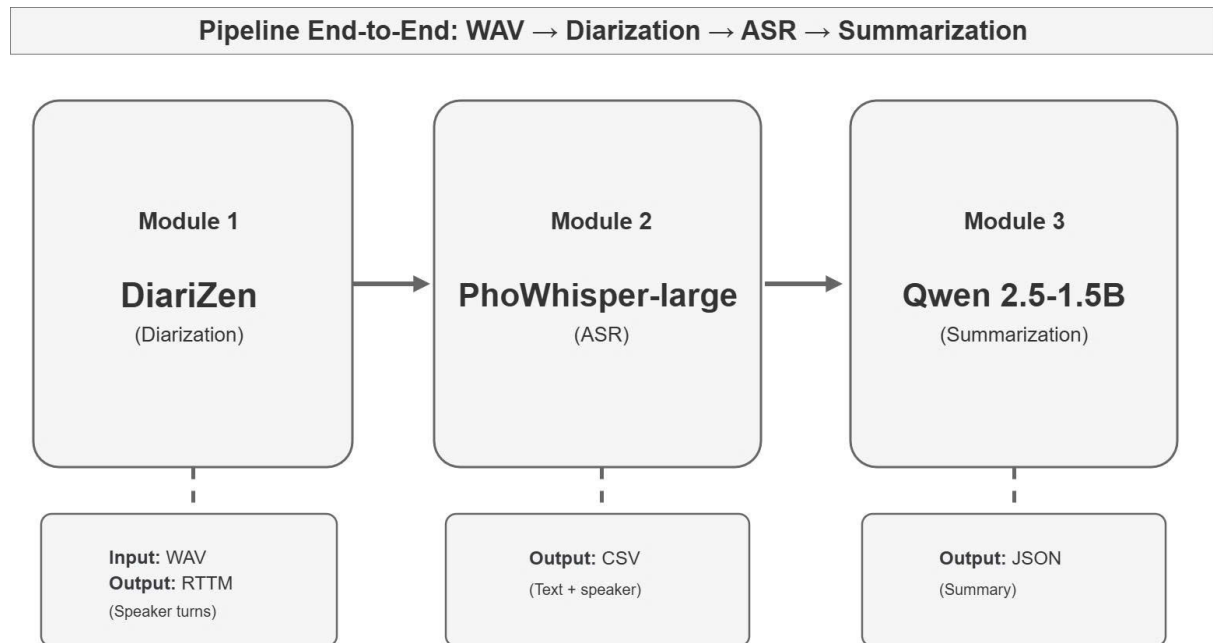


Figure 8: End-to-end pipeline model.

### 4.2 Diarization & Transcription Pipeline: Technical Overview

#### 4.2.1 Overall Pipeline Overview

The pipeline is an end-to-end system that takes raw audio (with optional speaker enrollment) as input and produces a final RTTM file, a speaker-aware transcript, and a conversation summary as output. It consists of eight sequential phases:

The diarization stage (Phases 1-6) answers the question of who spoke when. Starting from raw audio, the system runs a backend inference model (DiariZen or Pyannote) to build a speaker activity map (Phase 1), generates and tiers candidate segments (Phase 2), initializes speaker centroids via AHC clustering or enrollment (Phase 3), assigns segments to centroids (Phase 4), rescues unresolved tracks via propagation (Phase 5), and finally cleans the timeline and exports the RTTM file (Phase 6). The transcription stage (Phase 7) uses the RTTM segments as input boundaries for PhoWhisper to produce a speaker-labeled Vietnamese transcript. The summarization stage (Phase 8) then passes the transcript to Qwen for normalization, block-level summarization, and final summary generation.

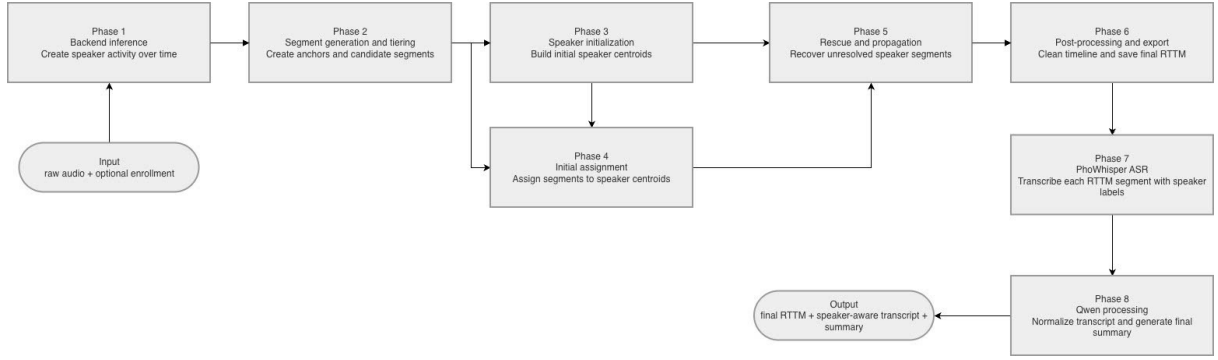


Figure 9: Overall Pipeline Overview.

### Phase 1: Input Preparation and Backend Inference.

This phase receives the audio input, the expected number of speakers, and enrollment data (if available), then selects a diarization backend. In the DiariZen branch, the system slides a fixed-size window over the audio, runs the model to produce a frame-level speaker activity probability matrix, and then smooths it with a median filter before storing the results as chunks containing temporal positions, activity probabilities, and frame-resolution information [5, 10]. In the pyannote branch, the process is analogous: the segmentation backend generates a time-based speaker activity representation to be used in subsequent phases [2]. At the end of this phase, the pipeline does not yet have final speaker labels, but it has a speaker activity map over time.

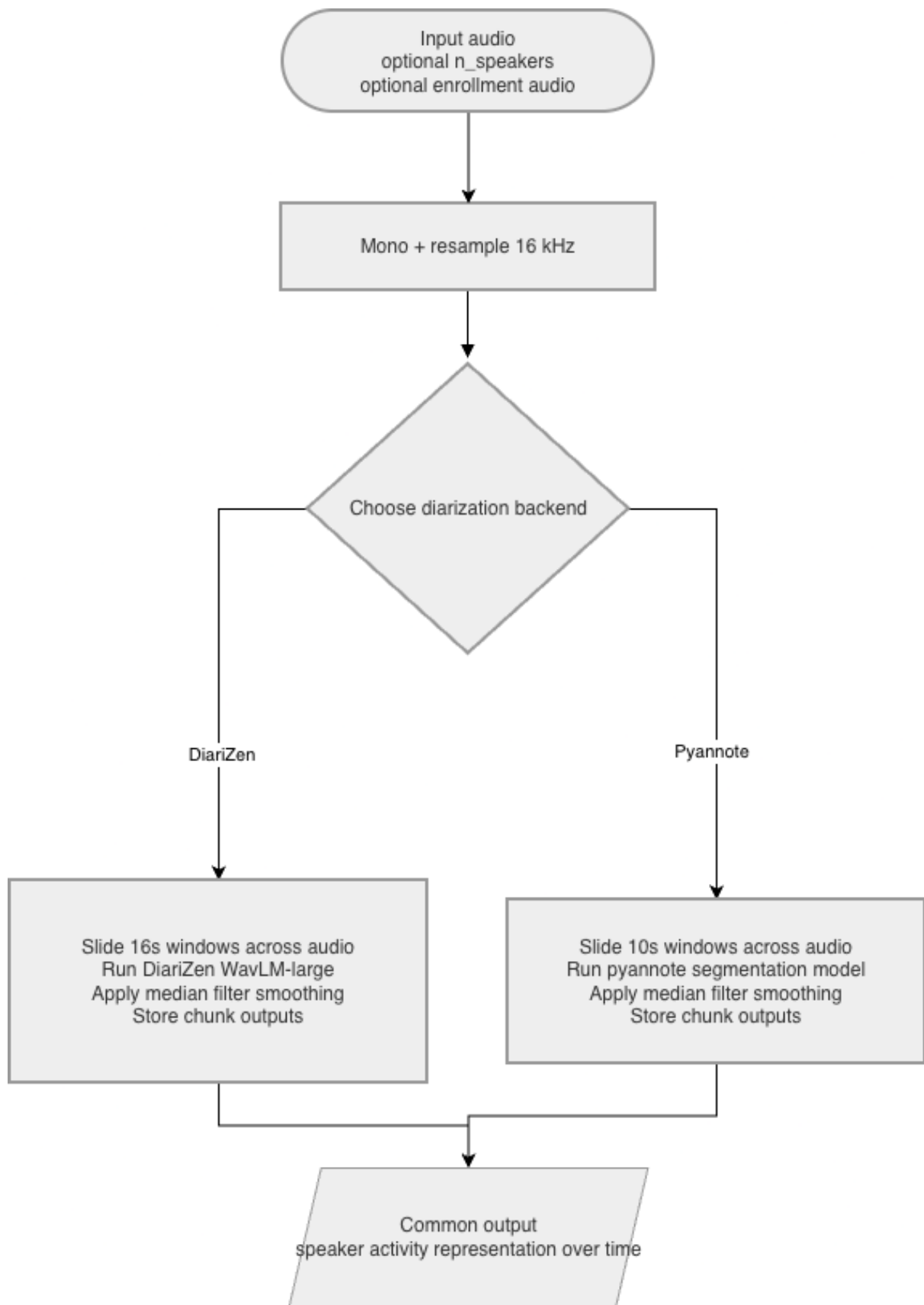


Figure 10: Phase 1 Input Preparation and Backend Inference.

## Phase 2: Shared Segment Generation and Tiering.

From the speaker activity output, the system applies per-frame thresholding and merges consecutive active frames into activity segments. This is a critical intermediate block because it produces two distinct input streams: a stream of high-confidence anchors used to initialize speaker centroids, and a stream of candidate segments for later assignment. For the candidate segments, the system further evaluates the cleanliness of each segment and classifies them as Primary, Aux, or Fallback based on clean span, usable duration, and confidence level. Therefore, this phase does not merely create segments; it decides which segments are clean enough to build cluster centroids and which should only be used at the labeling stage.

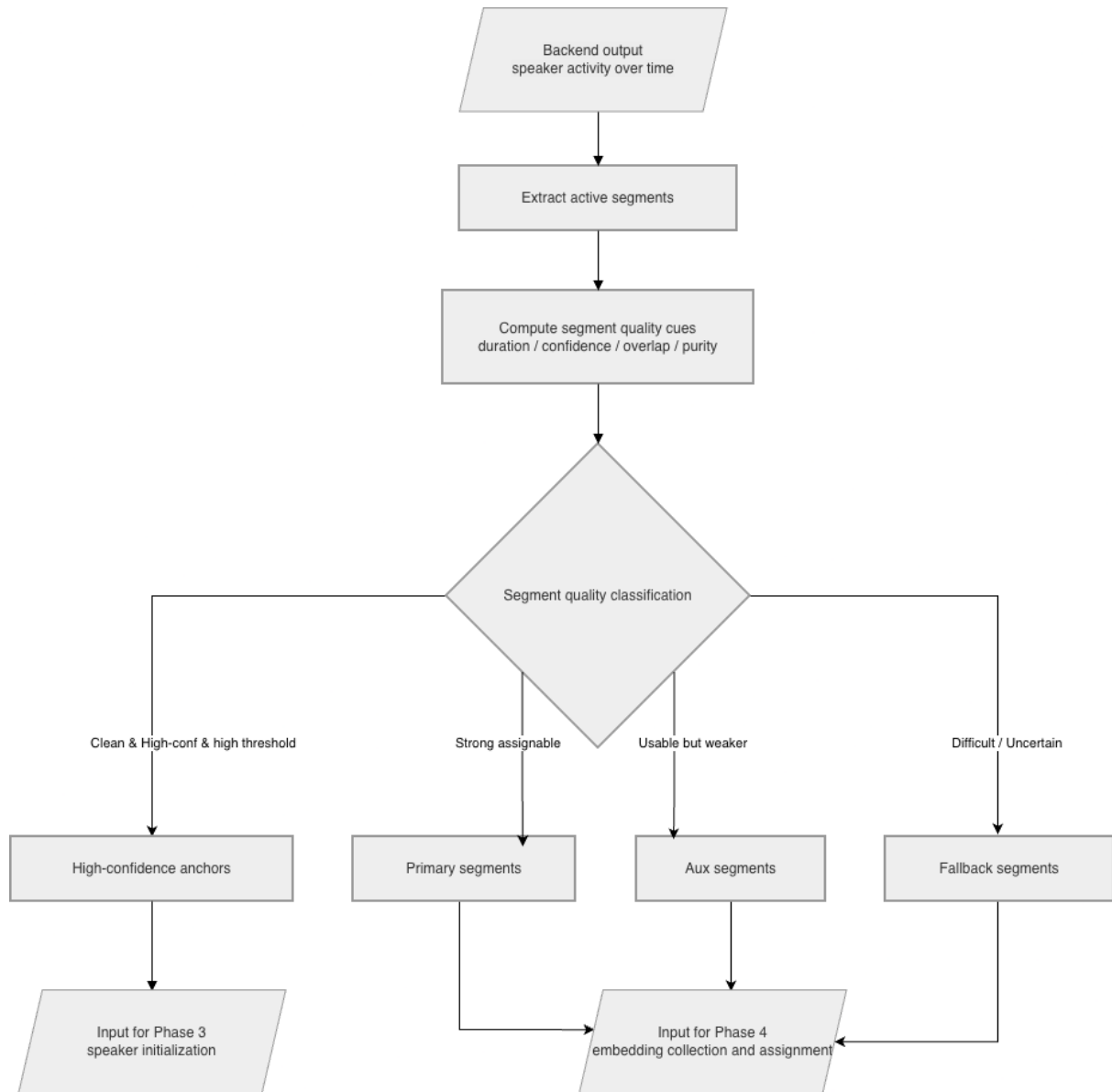


Figure 11: Phase 2 Shared Segment Generation and Tiering.

### Phase 3: Enrollment-Aware Speaker Initialization.

This phase builds the initial speaker centroids. If valid enrollment data is available, the system extracts embeddings from each speaker's sample audio to create enrollment centroids; if the enrollment covers enough speakers it can be used directly, otherwise it is combined with bootstrap centroids de-

rived from actual audio. If no enrollment is provided or the enrollment quality is insufficient, the system takes the high-confidence anchors from the model output, retains only clean segments, extracts embeddings, and clusters them using Agglomerative Hierarchical Clustering (AHC) [11]. After the model (such as EEND-EDA or other feature-vector extractors) produces embeddings, which are vectors representing voice characteristics: AHC groups them in a bottom-up fashion: it starts by treating each embedding as its own cluster, then iteratively merges the two most similar clusters until a stopping criterion is met [12]. Cannot-link constraints are applied between different tracks within the same chunk to prevent incorrect merges. The embeddings are then combined with weighting and normalized to produce the initial centroids for each speaker.

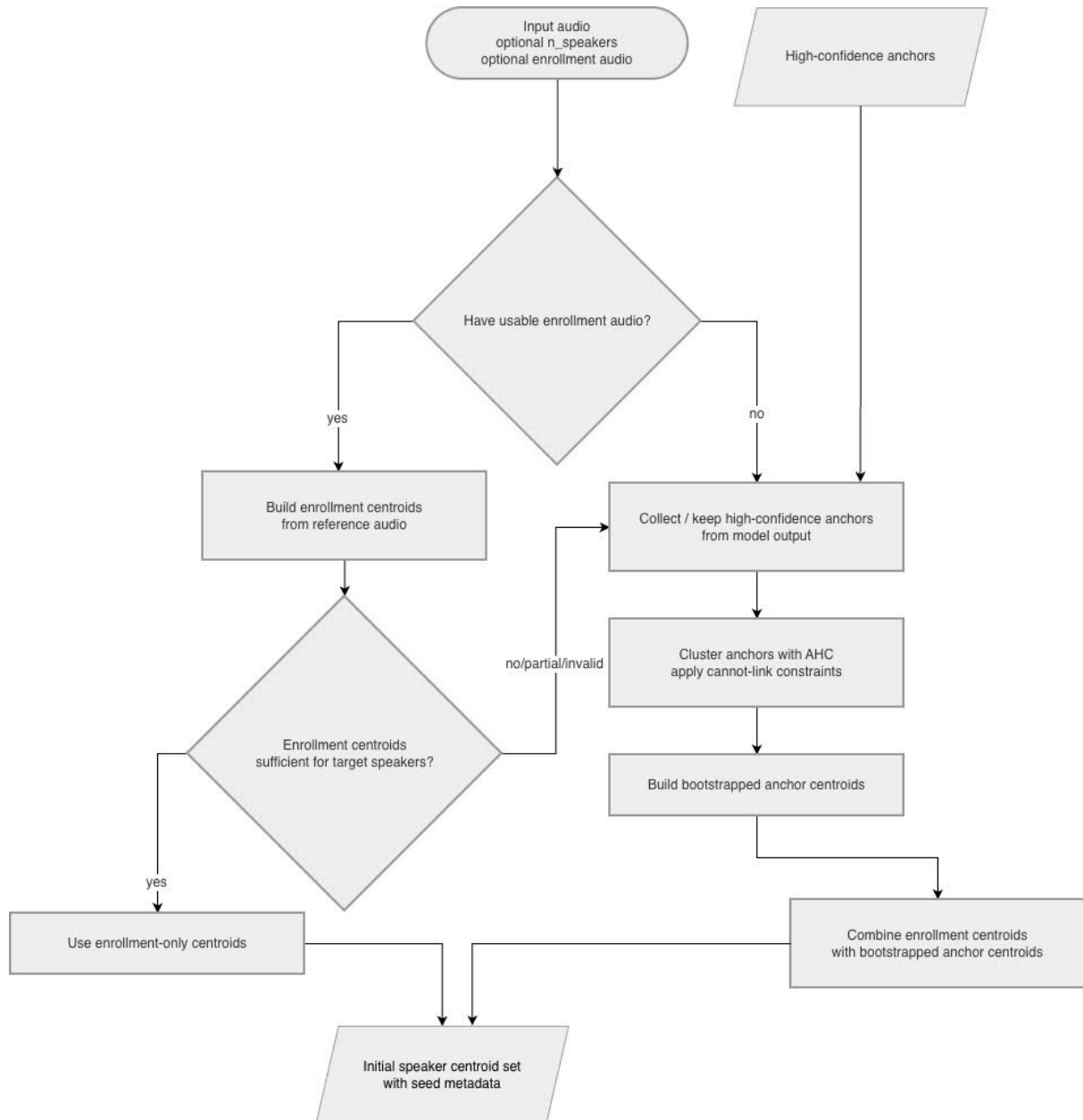


Figure 12: Phase 3 Enrollment-Aware Speaker Initialization.

#### Phase 4: Embedding Collection and Initial Assignment.

With the initial centroids in place, the pipeline scans the model output again to collect candidate segments broadly, classifies them into Primary / Aux / Fallback tiers, and extracts embeddings for each usable segment. Each embedding is compared against all centroids using a similarity measure; if it exceeds the tier-specific threshold, the segment is assigned a speaker label; otherwise it is held back for rescue. The system then checks for collision in overlapping tracks mapped to the same speaker, attempting to reassign the weaker track or discard it if no sufficiently confident alternative exists. From that provisional assignment set, the pipeline runs centroid purification to remove weak samples and recomputes cleaner centroids before performing a final local reassignment at the per-chunk-track level.

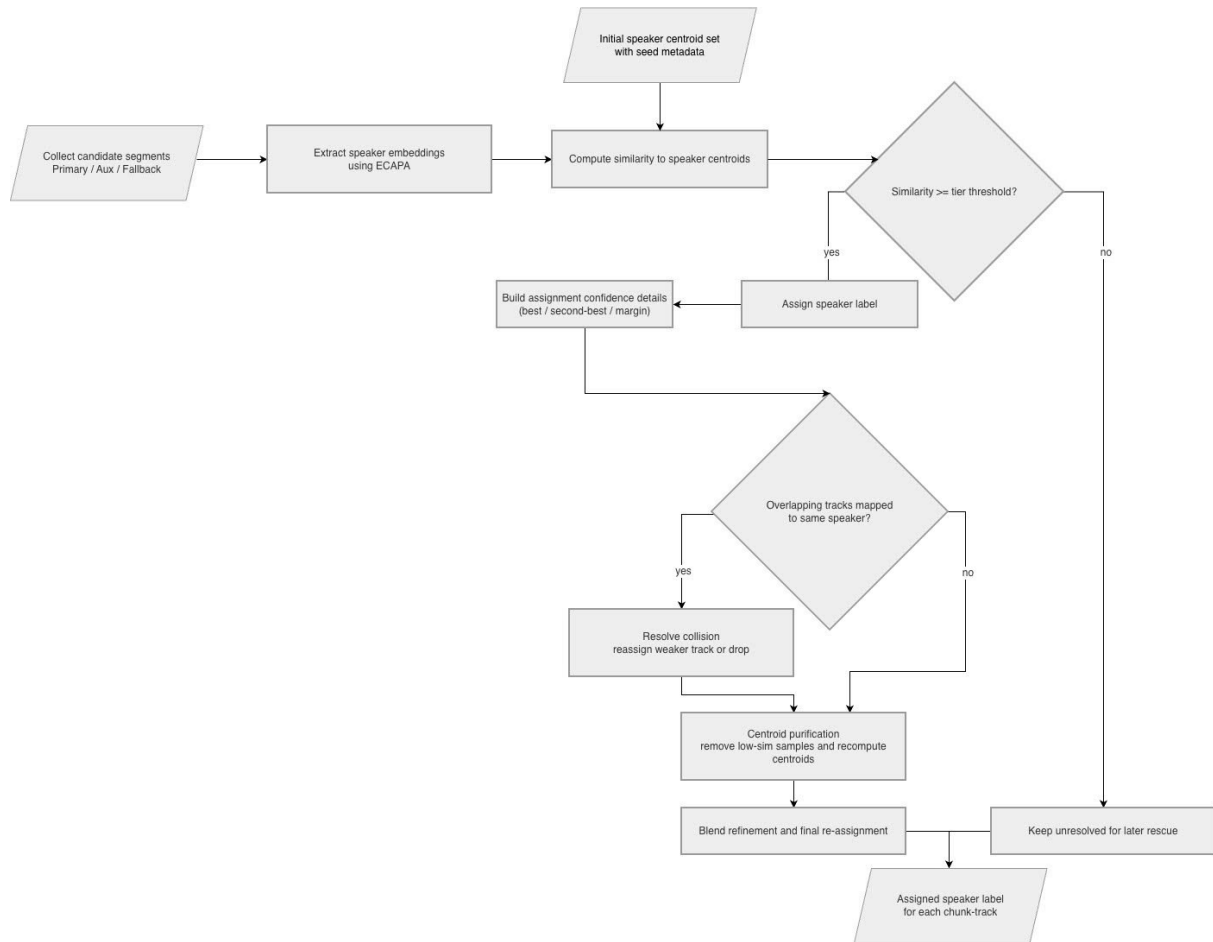


Figure 13: Phase 4 Embedding Collection and Initial Assignment.

### Phase 5: Missing-Track Rescue and Propagation.

This phase handles unresolved or missing segments after the initial assignment. The system first attempts fragment rescue for short or difficult segments by expanding each fragment, examining local sub-windows, re-extracting embeddings, and making one additional assignment attempt. If tracks remain unassigned, the pipeline applies neighbor propagation across nearby chunks; if the local signal is still insufficient, it falls back to global propagation to spread labels over a broader context. At the end of this phase the system has built a raw hypothesis, a speaker timeline with timestamps; that is considerably more complete than the result after the assignment phase alone.

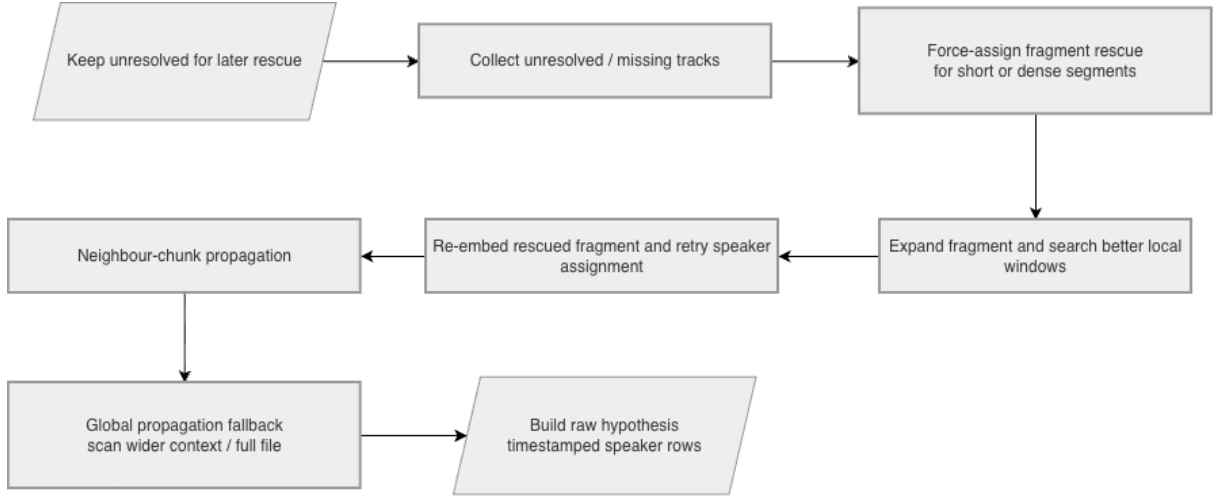


Figure 14: Phase 5 Missing-Track Rescue and Propagation.

### Phase 6: Post-Processing and Export.

This is the final phase, dedicated to cleaning and stabilizing the results before exporting them as RTTM. The pipeline applies a post-silence filter to remove segments with low energy or an excessively low speech ratio, then merges short gaps belonging to the same speaker and locks stable single-speaker regions. If enrollment data is present, internal labels of the form SPK\_i are remapped to the actual speaker names from speaker\_seed\_info. Finally, the system saves both a raw RTTM and a final hypothesis RTTM [13], the latter serving as input to the subsequent ASR phase.

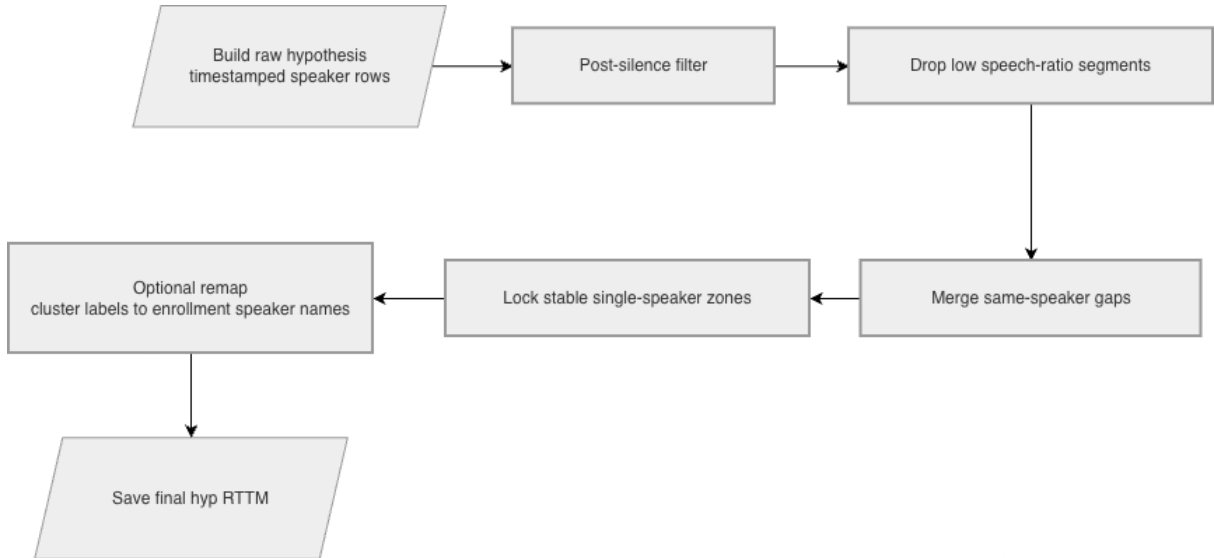


Figure 15: Phase 6: Post-Processing and Export.

### Phase 7: Vietnamese Automatic Speech Recognition Using PhoWhisper..

PhoWhisper receives the raw audio together with the final RTTM generated by the diarization stage [4]. It parses the RTTM into speaker segments, filters out very short or silent segments, extracts the exact audio slice for each valid segment, and performs Vietnamese ASR using the best PhoWhisper check-

point. The output is a structured, speaker-aware transcript in which each record retains the segment timing, speaker label, duration, and transcription text.

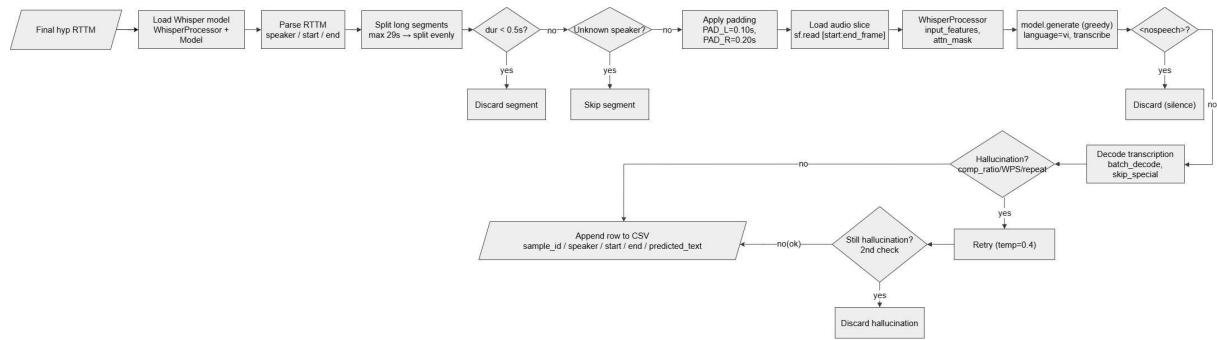


Figure 16: Phase 7 PhoWhisper Vietnamese Automatic Speech Recognition.

### Phase 8 Qwen: Summarization and Post-Processing.

Qwen [14] operates on the speaker-aware transcript produced by PhoWhisper rather than on raw audio. The transcript is first ordered chronologically and divided into manageable blocks. Each block is then normalized to reduce ASR artifacts while preserving speaker identity and semantic content. Qwen then generates block-level summaries, and a reducer combines them into a coherent final summary for the entire conversation. The results are exported in structured formats such as JSON and Markdown.

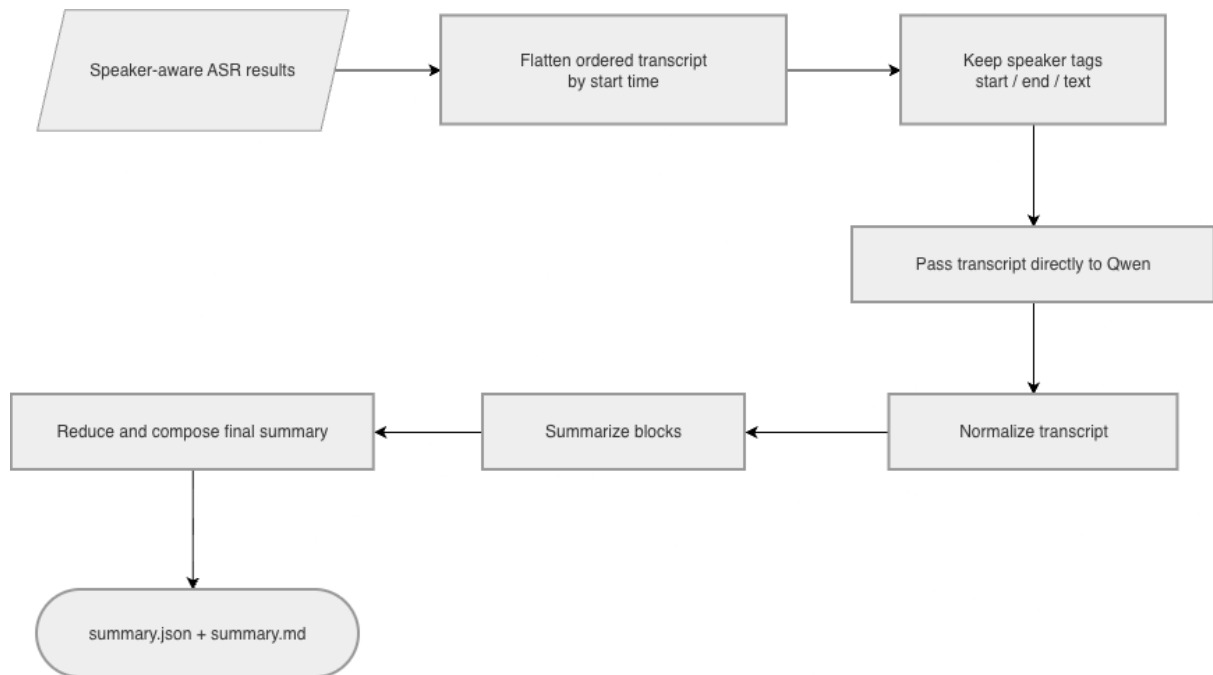


Figure 17: Phase 8 Summarization and Post-Processing Using Qwen.

## 4.3 Dataset Overview

This study uses multiple data sources for different parts of the pipeline. Public speech datasets were used for speech recognition adaptation and speaker-related representation learning, while synthetic

and collected meeting-style recordings were used for diarization training and evaluation. In addition, noisy and augmented meeting audio was included to examine robustness under adverse recording conditions.

Table 9: Dataset usage overview in the proposed system.

| <b>Task</b>      | <b>Purpose</b>                            | <b>Source</b>                         | <b>Usage</b>                  |
|------------------|-------------------------------------------|---------------------------------------|-------------------------------|
| ASR              | Speech recognition training               | VIVOS, Common Voice                   | Train / adapt ASR             |
| Diarization      | Speaker segmentation and overlap handling | Synthetic meetings, self-labeled data | Train / adapt Diarization     |
| Evaluation       | Real-world testing                        | Collected meeting recordings          | Test robustness in real audio |
| Noise robustness | Noisy audio conditions                    | Augmented / noisy meetings            | Evaluate denoising impact     |

Among the public Vietnamese speech corpora used in this study, VIVOS provides the largest number of speakers, with 46 speakers, and serves as the main clean-speech source for synthetic data generation. LSVSC and Common Voice each contribute 6 speakers, supporting additional speaker diversity for training and evaluation. The total duration of 243.19 hours refers to the synthetic speech data generated from these source corpora, not the original duration of VIVOS. In addition, the Speech Noise dataset does not provide speaker information, but contributes 947 noise segments for acoustic augmentation and robustness testing. Together, these datasets were selected to support synthetic data generation for diarization and ASR training and evaluation, while improving speaker coverage, acoustic robustness, and generalization.

Table 10: Public Vietnamese speech corpora used as source data for synthetic data generation.

| <b>Dataset</b>       | <b>Speakers</b> | <b>Synthetic Data Contribution</b>                               | <b>Sampling Rate</b> | <b>Usage</b>                                    |
|----------------------|-----------------|------------------------------------------------------------------|----------------------|-------------------------------------------------|
| VIVOS                | 46              | Part of the 210 h synthetic training set; 33.19 h validation set | 16,000 Hz            | Training and validation for diarization and ASR |
| LSVSC                | 6               | Part of the 210 h synthetic training set                         | 16,000 Hz            | Training for diarization and ASR                |
| Common Voice         | 6               | Part of the 210 h synthetic training set                         | 16,000 Hz            | Training for diarization and ASR                |
| Speech Noise dataset | 0               | 947 noise segments                                               | 16,000 Hz            | Acoustic augmentation and robustness testing    |

The self-labeled YouTube dataset contains 61 labeled sessions with a total duration of 46.16 hours.

Among the five source groups, Chuyen Ho Chuyen Minh contributes the largest amount of labeled audio with 16.97 hours, followed by Conan with 11.27 hours and VIF with 9.29 hours. In terms of usage, Conan is used for training only, while Chuyen Ho Chuyen Minh, Coi mo, Dustin on go, and VIF are used for both training and diarization/ASR evaluation.

Table 11: YouTube dataset summary for Self-labeled data.

| Dataset               | Avg. Speakers | Total Hours | Sampling Rate | Usage                                                           |
|-----------------------|---------------|-------------|---------------|-----------------------------------------------------------------|
| Chuyen Ho Chuyen Minh | 7             | 16.97 h     | 16,000 Hz     | Training (Diarization + ASR),<br>Evaluation (Diarization + ASR) |
| Coi mo                | 5             | 6.23 h      | 16,000 Hz     | Training (Diarization + ASR),<br>Evaluation (Diarization + ASR) |
| Conan                 | 13            | 11.27 h     | 16,000 Hz     | Training (Diarization + ASR)                                    |
| Dustin on go          | 4             | 2.41 h      | 16,000 Hz     | Training (Diarization + ASR),<br>Evaluation (Diarization + ASR) |
| VIF                   | 4             | 9.29 h      | 16,000 Hz     | Training (Diarization + ASR),<br>Evaluation (Diarization + ASR) |

## 4.4 Self- labeled Datasets

### 4.4.1 Process formation idea

The idea of building the data pipeline in this project came from a very practical problem: real-world Vietnamese multi-speaker conversational data is usually not available in a clean, synchronized, and fully labeled form. If only standard datasets are used, the system may perform well under controlled conditions, but that does not necessarily reflect its true performance in real scenarios such as talk shows, interviews, group discussions, or YouTube videos with background noise, background music, multiple overlapping speakers, and cross-talk. Therefore, instead of relying entirely on an existing standard dataset, the process in this project was designed around collecting and processing data directly from real YouTube videos, then combining automatic processing with manual correction to create a dataset suitable for multi-speaker diarization and transcription.

The core idea is not to treat data as merely available input, but as an important part of the research system itself. To evaluate a model properly, the data must be created through a controlled process. From that perspective, the data pipeline in this project is divided into two complementary branches. The first branch is machine processing, used to automatically generate preliminary transcripts, preliminary speaker segmentation, and a standardized data structure. The second branch is manual labeling, used to correct the errors that automatic models still do not handle well, especially in segments with many speakers, background music, overlap, or difficult pronunciation. The combination of these two branches helps the final data achieve both sufficient scale and sufficient quality for training and evaluation.

#### 4.4.2 What this idea was based on

In terms of tools and models, the current pipeline was not created entirely from scratch in the sense of inventing every component independently. Instead, it was built by referring to and combining multiple existing resources from the research community and open-source ecosystems. The diarization part was directly inspired by the pyannote.audio ecosystem, which is an open-source toolkit specialized for speaker diarization and provides pretrained pipelines that can be used directly or further fine-tuned.

The ASR part was based on the Whisper model family. Whisper [3] by OpenAI is a general multilingual speech recognition model trained on a very large scale and capable of speech recognition, translation, and language identification. The version used in the machine pipeline is faster-whisper [15], which is a reimplementation of Whisper using CTranslate2 to accelerate inference and reduce memory usage.

The speech separation from background music was based on Demucs [16], specifically the htdemucs model allows the vocal/speech channel to be separated from the rest of the signal.

The video and subtitle downloading component was based on yt-dlp [17], a popular open-source tool for downloading audio, video, and subtitles from YouTube. The yt-dlp documentation also clearly describes the options for downloading manual subtitles and automatic subtitles.

In addition, intro/outro/advertisement removal uses the SponsorBlock API [18]. This part is not the main focus of the modeling research, but it is a practical preprocessing step to reduce irrelevant content before passing the data into ASR and diarization. The integration of these tools into a single workflow is the project's own design contribution.

#### 4.4.3 General characteristics of the dataset

The real-world dataset in this project was built mainly from YouTube videos such as Conan, VIE Chuyen Ho Chuyen Minh, Coi mo, and Dustin on the Go. After reorganization, there are 61 samples, with the number of speakers in each sample ranging from 3 to 18. The Conan group contains the samples with the highest number of speakers, while groups such as VIE or Dustin on the Go have fewer speakers.

#### 4.4.4 Machine part

##### Overview.

The machine part is the automatic processing branch of the pipeline. The goal of this branch is to transform raw YouTube data into a preliminary version of the dataset that already has transcripts, timestamps, and an initial level of speaker segmentation. In other words, the machine part is not intended to create the final ground truth, but to produce a structured draft of the data that is good enough to be passed to the manual correction stage described later in Section 4.4.6. Without this branch, the entire labeling process would have to be done manually from the beginning, which would be very time-consuming and difficult to scale.

From a high-level perspective, the machine part starts from a YouTube video and ends with a transcript log containing timestamps and preliminary speakers. The whole process consists of multiple

sequential steps: downloading audio and subtitles, normalizing the audio, removing intro, outro, advertisements, separating speech from background music, running diarization to obtain speaker segments, running ASR to obtain transcripts, scanning again for missed regions, and then assigning speakers to each transcript segment based on the largest temporal overlap. The final result is an intermediate-format transcript log that can be loaded into the labeling tool.

### **Input of the machine part.**

The input of the machine part is:

- YouTube video link.
- An optional language setting if the language is to be specified in advance.

The input is a raw YouTube video. From that video, the system automatically downloads the audio and subtitle if available, then continues through the following processing steps.

### **Output of the machine part.**

The main outputs of the machine part include:

- Standardization comparison.
- Full transcript.
- Timestamped transcript, speaker-attributed transcript, and RTTM.
- STM transcript file.

### **Which component is used.**

In the machine part, the models and tools are used quite clearly for different tasks.

yt-dlp is used to download audio and subtitles from YouTube. This is not a deep learning model, but a data acquisition tool for collecting the input.

ffmpeg is used to convert audio into WAV mono 16 kHz format. This is a technical normalization step.

SponsorBlock is used to mark and mute intro, outro, advertisement, or unnecessary interaction segments. This is a coarse content-cleaning step.

Demucs with the htdemucs model is used for separating speech from background music by extracting the vocals stem. This task helps produce a more favorable input signal for the downstream models.

pyannote/speaker-diarization-3.1 is used for diarization, that is, determining who is speaking at which time. According to the official model card, this pipeline automatically downmixes multi-channel audio to mono and resamples to 16 kHz if needed.

faster-whisper large-v3 is used for the main ASR task, namely transcribing audio into text with timestamps. Faster-Whisper is an optimized implementation of Whisper using CTranslate2, with the goal of improving speed and reducing memory usage while maintaining comparable quality.

#### **4.4.5 Detailed description of the machine part**

##### **Downloading audio and subtitles from YouTube.**

This step is used to obtain the raw input data for the entire pipeline. If the audio cannot be obtained, the later steps cannot continue. If subtitles can be obtained, they provide an additional useful textual reference source, although subtitles are not always perfectly accurate. The input is a YouTube link and an optional language setting. The script uses `yt_dlp` with settings to download the best available audio of the video, download manual subtitles if available, download automatic subtitles if regular subtitles are not available, and convert the audio to WAV through `ffmpeg` post-processing.

##### **Normalizing audio to WAV mono 16 kHz.**

The purpose of this step is to unify all audio files into the same technical format, avoiding the case where each file has a different sample rate or number of channels. The input is the raw WAV audio file that was just downloaded. The script calls `ffmpeg` to convert audio to mono, convert the sample rate to 16 kHz using `-ar 16000`, and remove video if any remains using `-vn`. In `pyannote`, audio with a different sample rate can also be automatically resampled during loading, but in this project's pipeline, normalization is still performed proactively in advance so that all later steps operate on the same format.

##### **Muting intro, outro, and sponsor segments.**

This step is used to reduce segments that are not part of the main conversational content, such as intro, outro, advertisements, or teasers. The input is a WAV mono 16 kHz file and the YouTube link used to extract the `video_id`. The pipeline extracts the `video_id` from the URL, sends a query to the SponsorBlock API, obtains the list of segments belonging to categories such as intro, outro, sponsor, interaction, and preview, reads the audio file, sets the amplitude to 0 within those time ranges, and saves the new audio file. An important point is that the script does not remove the segments from the file, but only mutes them, so the time axis of the entire file remains unchanged. The output is a new audio file with the same duration, but with unnecessary segments muted.

##### **Separating speech from background music using Demucs.**

This step is used to emphasize the speech component and reduce the effect of background music or sound effects. The input is the audio file after the intro/outro muting step. The pipeline receives the input audio file, calls `demucs.separate`, uses the `htdemucs` model, waits for `vocals.wav` to be generated, uses this file for the next steps if the process succeeds, and falls back to the previous audio if the process fails.

##### **Diarization using pyannote/speaker-diarization-3.1.**

This step is used to create preliminary speaker time ranges indicating who speaks in which segment. The input is the audio file after cleaning. The pipeline loads the `pyannote/speaker-diarization-3.1` pipeline, moves the model to CUDA if available, runs the model on the audio file, iterates through the output by track, and extracts the start, end, and speaker for each segment. According to the official model card, this pipeline performs speaker diarization on audio and supports automatic downmixing to mono and resampling to 16 kHz at load time.

##### **ASR using faster-whisper large-v3.**

This step generates a preliminary transcript with timestamps from the cleaned audio file and the optional language setting. The pipeline loads `WhisperModel("large-v3")`, uses GPU when available, and otherwise falls back to CPU. It then calls `transcribe(...)` with `beam_size=5`, `condition_on_previous_text=False`, `vad_filter=True`, and VAD parameters including `threshold=0.2`, `min_silence_duration_ms=250`, and `speech_pad_ms=400`. Word-level timestamps are enabled using `word_timestamps=True`. Finally, the pipeline returns the list of ASR segments.

#### **Rescanning long gaps using Whisper large.**

This step is used to reduce missing transcript regions caused by the first ASR pass. The input is the ASR segment list and the audio file. The pipeline iterates through the gaps between two consecutive ASR segments, cuts a region separately if the gap is larger than the 4.0-second threshold, checks whether that region is true silence, runs Whisper large again if it is not true silence, filters out results likely to be junk such as “đăng ký” and “subscribe”, and adds valid text back into the segment list if such text exists. The output is a transcript with fewer missing regions than the first ASR pass.

#### **Assigning speakers to the transcript.**

This step is used to convert a plain transcript into a transcript with speaker attribution. The input is the ASR segment list and the speaker segment list from pyannote. The pipeline takes each transcript segment, compares its temporal overlap with each speaker segment, finds the speaker with the largest overlap, assigns that speaker to the transcript segment, and saves the speaker-attributed transcript. The output is a preliminary speaker-attributed transcript.

### **4.4.6 Manual labeling**

#### **Overview.**

Manual labeling is the branch for checking and correcting the data after the machine part described in Section 4.4.4 has produced the draft. The role of this branch is to fix the parts that the automatic models have not handled correctly, such as incorrect boundaries, wrong speaker assignment, missing words, incorrect recognition of regional words, or unreasonable segment division. In practice, this is the step that determines the final quality of the dataset. If the machine part produces silver labels, then the manual labeling step raises the data to near-gold quality.

#### **Input of the manual labeling part.**

The inputs of this stage are:

- The .wav audio file;
- The preliminary transcript log with timestamps;
- The preliminary speaker labels generated by the machine part.

The VAD Editor tool is designed to load these types of files directly.

#### **Output of the manual labeling part.**

The outputs are:

- Corrected transcript;
- Corrected segment boundaries;
- Corrected speaker labels;
- Audio clips cut by speaker;
- Cleaner dataset to continue normalization, consistency checking, and organization into the dataX/audio, dataX/labeled, and dataX/enrollment structure.

### **Which model is used for which task.**

In the manual labeling part, the main tool is not a single model, but the VAD Editor GUI. However, this GUI also integrates Whisper to support semi-automatic transcription on selected segments. In the current code, the GUI uses `whisper.load_model("large")` when running Auto ASR. This means Whisper here only serves as an assistance tool for the annotator, not as the final ground-truth source. The user still decides the final content of the transcript.

- The GUI is used to view the waveform, edit time, edit speakers, and edit text;
- Whisper in the GUI is used to suggest text for the selected segment;
- The user uses listening and visual inspection to confirm the final result.

### **Detailed description of the manual labeling part.**

#### **Loading data into the interface.**

##### **Purpose.**

To place the audio and preliminary transcript in the same editing workspace.

##### **Input.**

A .wav file and a transcript log file.

##### **Processing.**

The user opens the GUI, loads the audio, and then loads the transcript. The GUI displays the waveform and the segment table. Each transcript line is parsed according to the timestamp format so that the correct start time, end time, and corresponding text are displayed.

##### **Output.**

A working session in which the audio and transcript are synchronized in the interface.

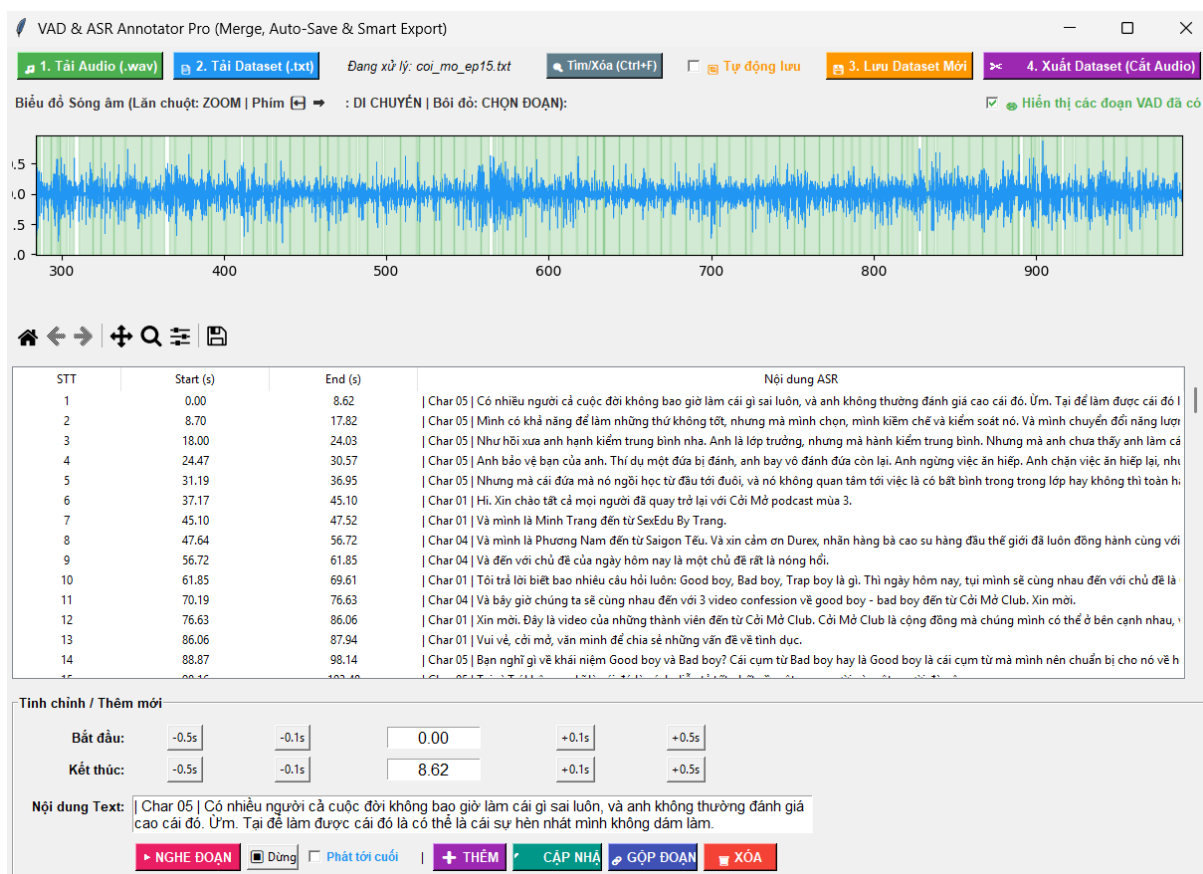


Figure 18: Overall interface of the tool after loading the audio and transcript.

## Checking and correcting time boundaries.

### Purpose.

To adjust the start/end boundaries of each segment more precisely.

### Input.

A segment with preliminary timing generated by the machine part.

### Processing.

The user observes the waveform, selects a segment, listens to the segment again, increases or decreases the start or end time, and, if necessary, directly selects a new region on the waveform. The GUI supports fine-grained time adjustment using small increment/decrement buttons.

### Output.

A segment with more accurate temporal boundaries.

The audio clips exported later may be cut too early, too late, or may overlap with another speaker's utterance.

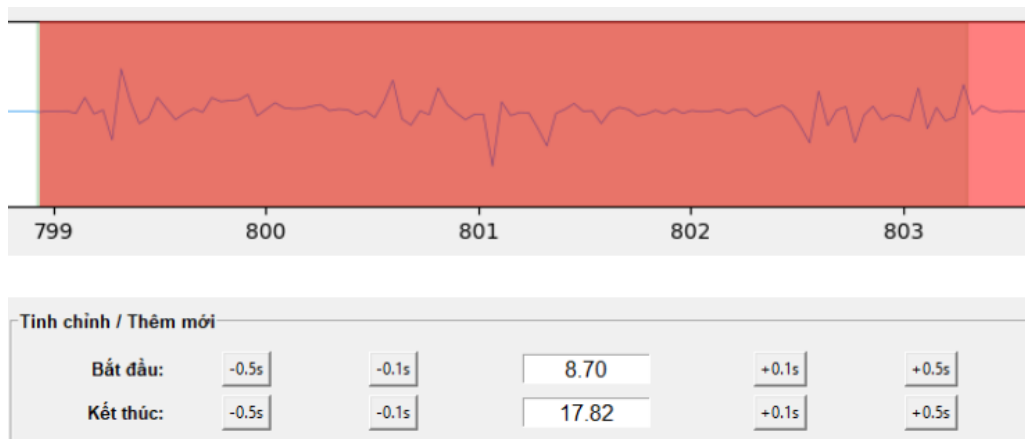


Figure 19: Selecting a region on the waveform and adjusting the start/end times.

## Checking and correcting transcript content.

### Purpose.

To correct wrongly recognized words, missing words, or unnatural text produced by ASR.

### Input.

The preliminary text from the machine part.

### Processing.

The user listens to the audio again, compares it with the displayed transcript, and directly edits the text in the GUI. If the transcript is very inaccurate, the user may delete it and type it again. If the transcript is only slightly wrong, the user edits it directly in place.

### Output.

A transcript that is more accurate and closer to the actual speech.

The data will still contain many ASR errors and will not be reliable enough for WER evaluation or training.

|                |                                                                                                         |
|----------------|---------------------------------------------------------------------------------------------------------|
| Nội dung Text: | Char 05   Thì dĩ nhiên nếu mà chưa có tiêu chuẩn thì họ phải nghĩ tới việc là cứ bao nhiêu quãng vô hết |
|----------------|---------------------------------------------------------------------------------------------------------|

Figure 20: Transcript table before manual correction.

|                |                                                                                                                 |
|----------------|-----------------------------------------------------------------------------------------------------------------|
| Nội dung Text: | Char 05   Thì dĩ nhiên là nếu mà chưa có tiêu chuẩn thì họ phải nghĩ tới cái việc là cứ bao nhiêu quãng vô hết. |
|----------------|-----------------------------------------------------------------------------------------------------------------|

Figure 21: Transcript table after manual correction.

## Correcting speaker labels.

### Purpose.

To ensure that each segment is assigned to the correct speaker.

### **Input.**

The preliminary speaker label generated by the machine part.

### **Processing.**

The user listens to the segment again, observes the context, and changes the speaker if the current speaker is not correct. The GUI supports a list of characters/speaker names for fast operations. In many cases, this step is even more important than correcting the text, because a correct transcript attached to the wrong speaker is still an incorrect label.

### **Output.**

More accurate speaker labels for each segment.

The data will contain speaker attribution errors, which are very harmful for the diarization task.

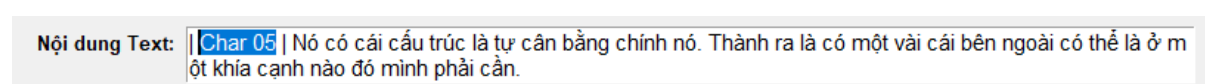


Figure 22: Speaker label before manual correction.

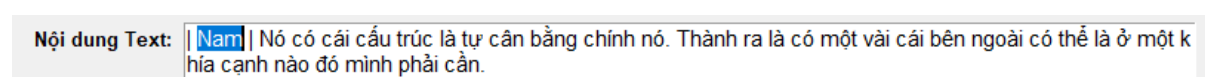


Figure 23: Speaker label after manual correction.

## **Merging or splitting segments again.**

### **Purpose.**

To make the transcript structure more natural and closer to the actual speaking units.

### **Input.**

Existing segments that may be too fragmented or too long.

### **Processing.**

The user selects the segment to modify, merges two consecutive segments if they are actually one sentence, splits one long segment if it contains multiple utterances by selecting a new region on the waveform, and updates the text and speaker after merging or splitting.

### **Output.**

A transcript with a more reasonable segment structure.

The data will be harder to use for training and harder to read during audit.

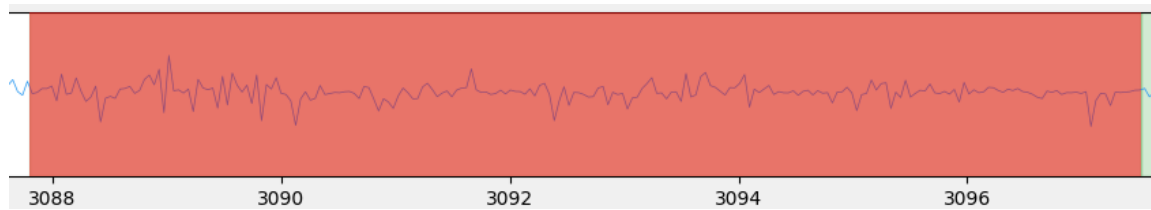


Figure 24: Segment structure before merging or splitting.

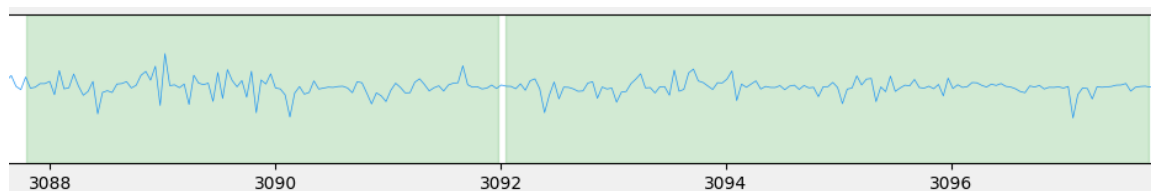


Figure 25: Segment structure after merging or splitting.

## Exporting data after manual correction.

### Purpose.

To convert the corrected transcript and original audio into assets that can be used in the later dataset normalization pipeline.

### Input.

The original audio and the manually corrected transcript.

### Processing.

The GUI exports by reading each segment, cutting audio according to the corrected timestamps, adding a small amount of padding, parsing the speaker from the text, and saving the file into the corresponding speaker directory.

### Output.

Audio files cut by speaker, ready for the dataset normalization step.

The data will still have the global transcript, but will lack speaker-level assets for cross-checking and reuse.

|                                                                                                       |                    |               |            |
|-------------------------------------------------------------------------------------------------------|--------------------|---------------|------------|
|  vocals.rttm         | 3/28/2026 8:53 AM  | RTTM File     | 16 KB      |
|  vocals.stm          | 3/28/2026 8:53 AM  | STM File      | 53 KB      |
|  vocals_annotate.tsv | 3/28/2026 8:53 AM  | TSV File      | 60 KB      |
|  coi_mo_ep5.txt      | 3/28/2026 8:52 AM  | Text Document | 55 KB      |
|  vocals.wav          | 3/12/2026 10:40 AM | WAV File      | 871,805 KB |
|  Trang               | 3/28/2026 9:06 AM  | File folder   |            |
|  Uyen                | 3/28/2026 9:06 AM  | File folder   |            |
|  Duy                 | 3/28/2026 9:06 AM  | File folder   |            |
|  Nam                 | 3/28/2026 9:06 AM  | File folder   |            |
|  Toan                | 3/28/2026 9:06 AM  | File folder   |            |

Figure 26: Folder structure after export.

## 4.5 Synthetic Datasets

### 4.5.1 Introduction

Training and evaluating a speaker diarization system requires a source of multi-speaker conversational audio data with accurate labels at the time-segment level [6, 7, 8]. However, real-world conversational data of this type, especially for Vietnamese, remains limited in quantity and diversity. Most existing Vietnamese speech corpora are primarily designed for automatic speech recognition (ASR) tasks, consisting of isolated sentences from individual speakers, which do not reflect the characteristics of a real-world multi-speaker conversational environment.

To address this issue, we built a synthetic data generation pipeline to create controlled multi-speaker simulated meetings. The pipeline allows flexible adjustment of parameters such as speech overlap ratio, background noise conditions, room acoustics, number of speakers, and meeting duration. The entire source code is implemented in Python, utilizing a modular architecture with centralized configuration parameters, ensuring consistency and reproducibility.

This section presents in detail the entire process of building the training dataset, from collecting raw data, performing forced alignment to create word-level labels, simulating meetings, up to validating the output quality.

### 4.5.2 Raw Data Sources

#### Vietnamese Speech Corpora.

The raw audio data is collected from three public Vietnamese speech corpora, separated by speaker and accompanied by text transcripts. A total of 52 speakers are divided into two non-overlapping groups to generate two distinct datasets. The Train Group (18 speakers) consists of 6 VIVOS speakers (SPK04, 07, 08, 44, 45, 46), 6 LSVSC speakers, and 6 Common Voice speakers. This group is used to generate the training set of approximately 210 hours. The Val Group (40 speakers) consists of the remaining 40 VIVOS speakers (SPK01-46, excluding the 6 speakers in the Train group). This group is

used to generate the validation set of approximately 33 hours.

### Background Noise Data.

To enhance the realism of the synthetic data, we additionally utilize the real-world noise dataset speech-noise-dataset, containing 947 WAV files of common office noise types such as air conditioner hum, keyboard typing, waiting-room chatter, indistinct background conversations, and electronic static [9].

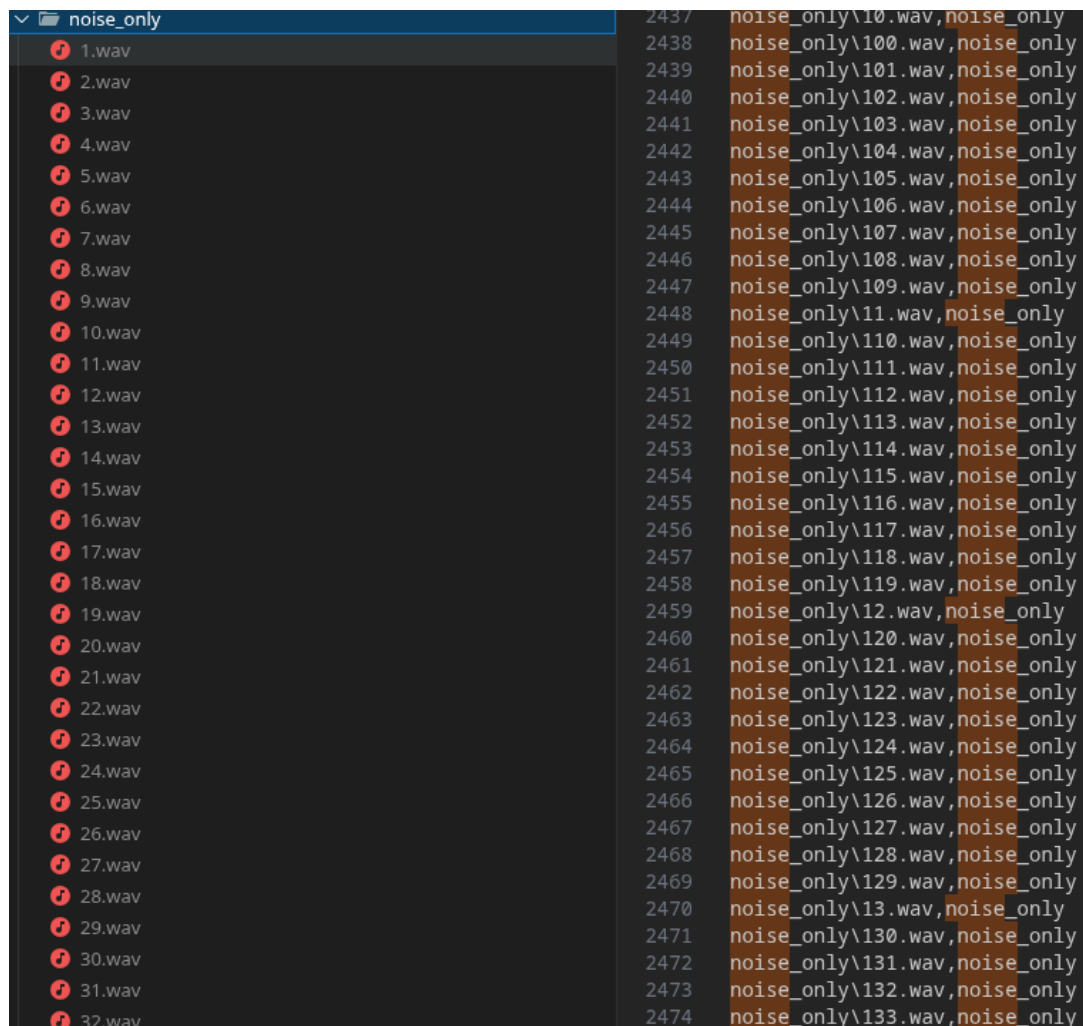


Figure 27: Source data noise.

### 4.5.3 Forced Alignment

#### Objective and Methodology.

In order to generate word-level labels for the synthetic data, it is necessary to accurately determine the start and end times of each word within every audio file. We perform forced alignment, which is the process of forcibly synchronizing audio with a known transcript, using the Wav2Vec2 [19] model for Vietnamese.

The utilized model is nguyenvulebinh/wav2vec2-base-vietnamese-250h, a fine-tuned Wav2Vec2 model fine-tuned on 250 hours of Vietnamese data. This model is downloaded from Hugging Face via the

Transformers library [20].

### **Forced Alignment Workflow.**

The forced alignment process is implemented module and executes the following steps.

#### **Step 1: Load Input Data.**

Load the speaker’s WAV file and the corresponding transcript from the prompts.txt file. The prompts file contains (utterance\_id, transcript) pairs separated by whitespace.

#### **Step 2: Extract CTC Features.**

Pass the 16 kHz resampled audio signal through the Wav2Vec2 feature extractor to obtain a frame-level representation. This is then fed through the model’s encoder to yield a CTC logits matrix of size  $(T \times V)$ , where  $T$  is the number of frames and  $V$  is the vocabulary size [21].

#### **Step 3: Greedy CTC Decode and Mapping to Words.**

Perform greedy decoding on the logits to identify the highest-probability token label at each frame. Consecutive identical tokens and blank tokens are merged. The resulting token sequence is then mapped back to the list of words in the reference transcript in chronological order.

#### **Step 4: Calculate Time Boundaries.**

Based on the frame positions of each token, calculate the start and end times, in seconds, of each word.

#### **Step 5: Export Results.**

Save the output as an individual .json file for each utterance, where each word is represented by the attribute set {word, start, end, duration}.

Table 12: Technical specifications of the forced alignment process.

| Parameter         | Value                                         |
|-------------------|-----------------------------------------------|
| Model             | nguyenvulebinh/wav2vec2-base-vietnamese-250h  |
| Framework         | PyTorch 2.0.1 + Transformers (< 4.40)         |
| Decoding method   | Greedy CTC Decode                             |
| Input sample rate | 16000 Hz (resampled if necessary)             |
| Output format     | JSON: [{word, start, end, duration}] per file |
| Compute device    | GPU CUDA (NVIDIA, CUDA 11.7)                  |

### **Advantages of the Method.**

The method offers several practical advantages. First, it does not require a complex setup and can be installed with a simple pip install, without requiring Conda or the Montreal Forced Aligner. Second, it leverages a Wav2Vec2 model pre-trained on 250 hours of Vietnamese, yielding good results on the read-speech domain. Third, it provides fast processing speed on GPU, typically under 1 second per file. Finally, its accuracy is highly sufficient for conversational-level synthetic data generation tasks.

### **Disadvantages and Limitations.**

This remains a semi-accurate forced alignment method based on CTC probability distributions rather than manual alignment, so boundary precision depends on pronunciation quality and transcript fidelity. In addition, greedy CTC decoding is sub-optimal compared to Viterbi or beam search. The method also struggles with OOV words or large pronunciation deviations from the transcript. For that reason, a post-processing step is required to rectify boundary errors.

```
1  {
2    "file": "VIVOSSPK01_R014.wav",
3    "audio_id": "VIVOSSPK01_R014",
4    "words": [
5      {
6        "word": "ĂN",
7        "start": 0.903,
8        "end": 1.064,
9        "duration": 0.161
10     },
11     {
12       "word": "CẤP",
13       "start": 1.144,
14       "end": 1.305,
15       "duration": 0.161
16     },
17     {
18       "word": "PHÒNG",
19       "start": 1.385,
20       "end": 1.465,
21       "duration": 0.08
22     },
23     {
24       "word": "TƯỢNG",
25       "start": 1.586,
26       "end": 1.746,
27       "duration": 0.16
28     }
29   ]
30 }
```

Figure 28: Example of forced alignment results with word-level labels for speaker VIVOSSPK01.

### Alignment Cleaning.

The raw forced alignment results obtained from greedy decoding on the CTC logits matrix sometimes lack sharpness at word boundaries and can generate logical timeline errors. Specifically, the system may encounter reversed time chronology ( $\text{end} < \text{start}$ ), acoustic overlap where the succeed-

ing word begins before the preceding word concludes, abnormally long gaps between two words, or word durations that are unreasonably short or long compared to active pronunciation.

To reduce these issues and ensure a valid audio timeline, the module executes a strict three-step cleaning procedure.

### **Step 1: Fix Time Inversion Errors.**

The system loops through all words in the alignment results. At positions where the end time precedes the start time ( $\text{end} < \text{start}$ ), the algorithm swaps these two values to restore the linearity of the time axis. This is a preliminary sanity check to prevent subsequent arithmetic calculation errors.

### **Step 2: 20 Iterations of Time-Boundary Optimization.**

This is the core of the cleaning process. Adjusting the time boundaries of one word can cause a ripple effect, directly impacting the spacing or overlap with adjacent words. Therefore, the system does not simply fix the issue once, but executes 20 iterative loops covering the following three rules to encourage a stable timeline and eliminate all anomalies.

1. **Resolve acoustic overlap.** Within a single speaker's utterance, two words cannot be vocalized simultaneously. If the system records a negative distance between the current word and the next word, it means they are overlapping. The system calculates the midpoint of this overlap region, then assigns timestamps so the preceding word ends exactly at Midpoint and the succeeding word starts at Midpoint. This solution bridges the sentence without losing or distorting the total duration.
2. **Compress excessive gaps.** If the silent gap between two words exceeds 0.49 seconds, it usually represents an alignment error where words are overly compressed, leaving a phantom silence. To reconstruct this, the system stretches the boundaries of both adjacent words into that silence. Each side receives half of the excess time limit.
3. **Duration clamping.** Clamp all words into a safe interval of [0.055 s, 1.95 s]. The 55 ms lower bound rejects random fleeting spikes stemming from the CTC, while the 1.95 s upper bound prevents words from being irrationally elongated due to the model misidentifying background noise as word components.

### **Step 3: Absolute Constraints in the Final Pass.**

After the loops converge, the module performs a final consolidation pass applying a looser protective threshold by clamping duration to [0.051 s, 1.99 s]. All localized start and end timestamps undergo hard rounding to 4 decimal places. This operation is the key to reducing floating-point precision, which are capable of generating microscopic overlaps on the scale of microseconds. Thanks to this manipulation, strict subsequent measurement tools such as dscore or JSON parsers will not falsely trigger validation errors.

### **Alignment Quality Validation.**

Following cleaning, the script automatically verifies the quality of each alignment file and exports a report. The validation criteria are described in Table 13.

Table 13: Alignment quality validation criteria.

| Validation criterion      | Warning threshold | Action          |
|---------------------------|-------------------|-----------------|
| Word is too short         | < 50 ms           | Warning, logged |
| Word is too long          | > 2000 ms         | Warning, logged |
| Gap is too large          | > 500 ms          | Warning, logged |
| Residual negative overlap | < 0 ms            | Critical error  |
| File contains 0 words     | 0 words           | Reject file     |

Table 14: Actual alignment quality statistics after cleaning.

| Metric                | VAL (40 VIVOS) | VIVOS (6 spk) | LSVSC (6 spk) | CV (6 spk) |
|-----------------------|----------------|---------------|---------------|------------|
| Total alignment files | 11,656         | 1,649         | 19,527        | 1,942      |
| Total words aligned   | 154,208        | 18,235        | 508,549       | 14,755     |
| Empty files (0 words) | 0              | 0             | 0             | 0          |
| Words < 50 ms         | 0 (0.00%)      | 43 (0.24%)    | 392 (0.08%)   | 60 (0.41%) |
| Words > 2 s           | 0 (0.00%)      | 0 (0.00%)     | 28 (0.01%)    | 2 (0.01%)  |
| Residual overlaps     | 0              | 0             | 0             | 0          |
| Gaps > 500 ms         | 0              | 9             | 2,063         | 61         |
| Average duration      | 0.197 s        | 0.184 s       | 0.180 s       | 0.205 s    |
| Median duration       | 0.180 s        | 0.180 s       | 0.141 s       | 0.181 s    |
| Min duration          | 0.051 s        | 0.020 s       | 0.020 s       | 0.020 s    |
| Max duration          | 1.950 s        | 1.244 s       | 5.789 s       | 2.650 s    |

**Remarks.**

The VAL set shows the cleanest alignment quality after cleaning, with no residual overlaps, long gaps, or abnormal word durations. The LSVSC set retains 2,063 gaps longer than 500 ms, mainly because it contains slow read speech with natural pauses. This does not affect the conversation simulation pipeline because word-level timestamps are reassigned during timeline generation.

**4.5.4 Creating the Enrollment Set**

In closed-set speaker identification scenarios, it is necessary to prepare an enrollment set, which is a small set of reference WAV files representing each speaker. This set is used to extract speaker embeddings during the model evaluation phase, enabling the system to map anonymous speaker labels to known speaker identities.

The module automatically generates the enrollment set using the following procedure:

1. Iterate through each speaker directory in data/val/.
2. Retrieve all WAV files for that speaker.

3. Randomly select, without replacement, a maximum of 10 WAV files.
4. Copy the selected files to the destination directory `data/val/output/enroll/<speaker_id>/`.

Limiting the number to 10 files per speaker ensures balance across speakers and reflects real-world conditions, because in practical deployments users typically provide only a small number of enrollment samples.

#### 4.5.5 Meeting Simulation Pipeline

##### Architectural Overview.

The overall process consists of eight main steps: segment preprocessing, 70/30 segment merging, timeline generation for time allocation, overlap event determination, assigning audio to the timeline, room acoustic simulation, background noise augmentation, and signal mixing, normalization, and label exportation.

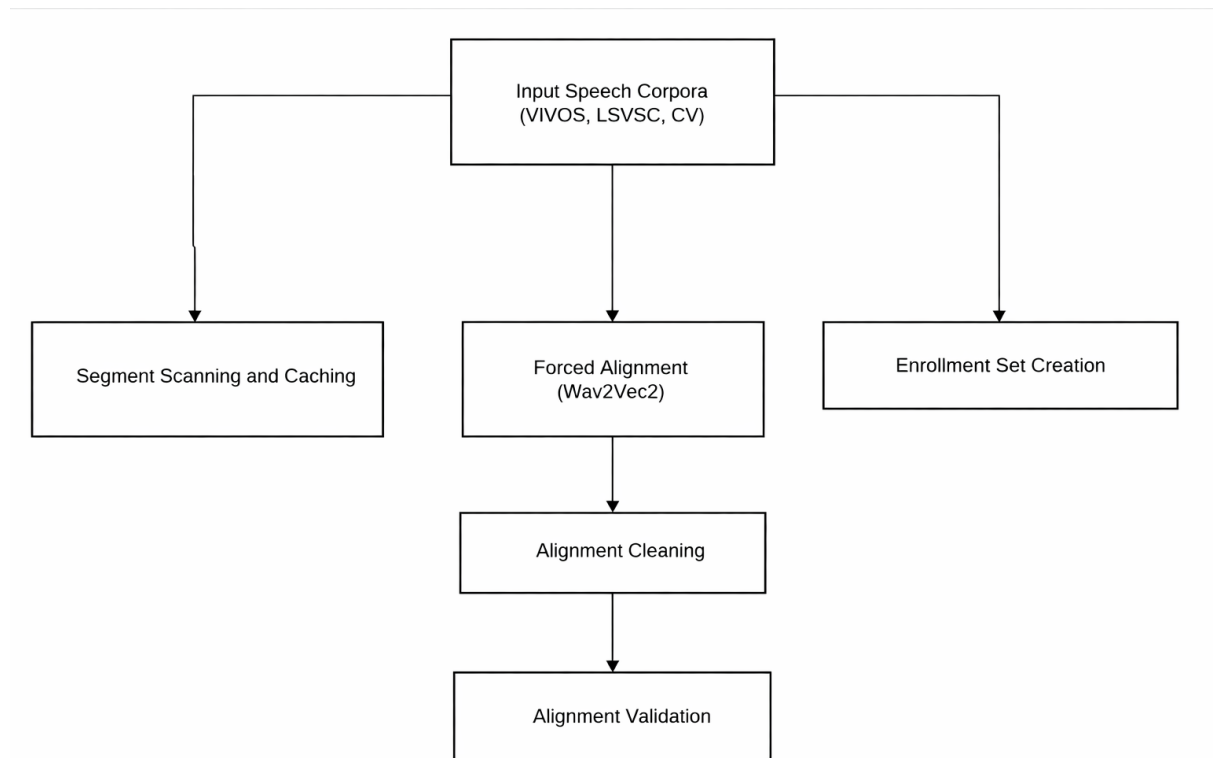


Figure 29: Meeting simulation pipeline, Part 1.

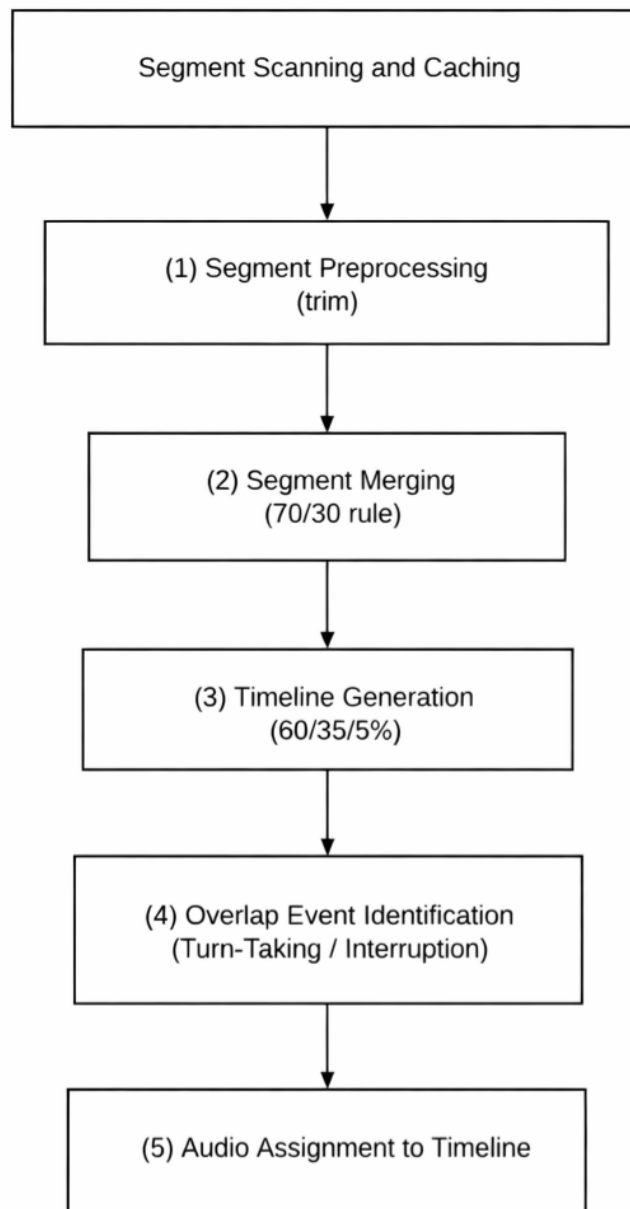


Figure 30: Meeting simulation pipeline, Part 2.

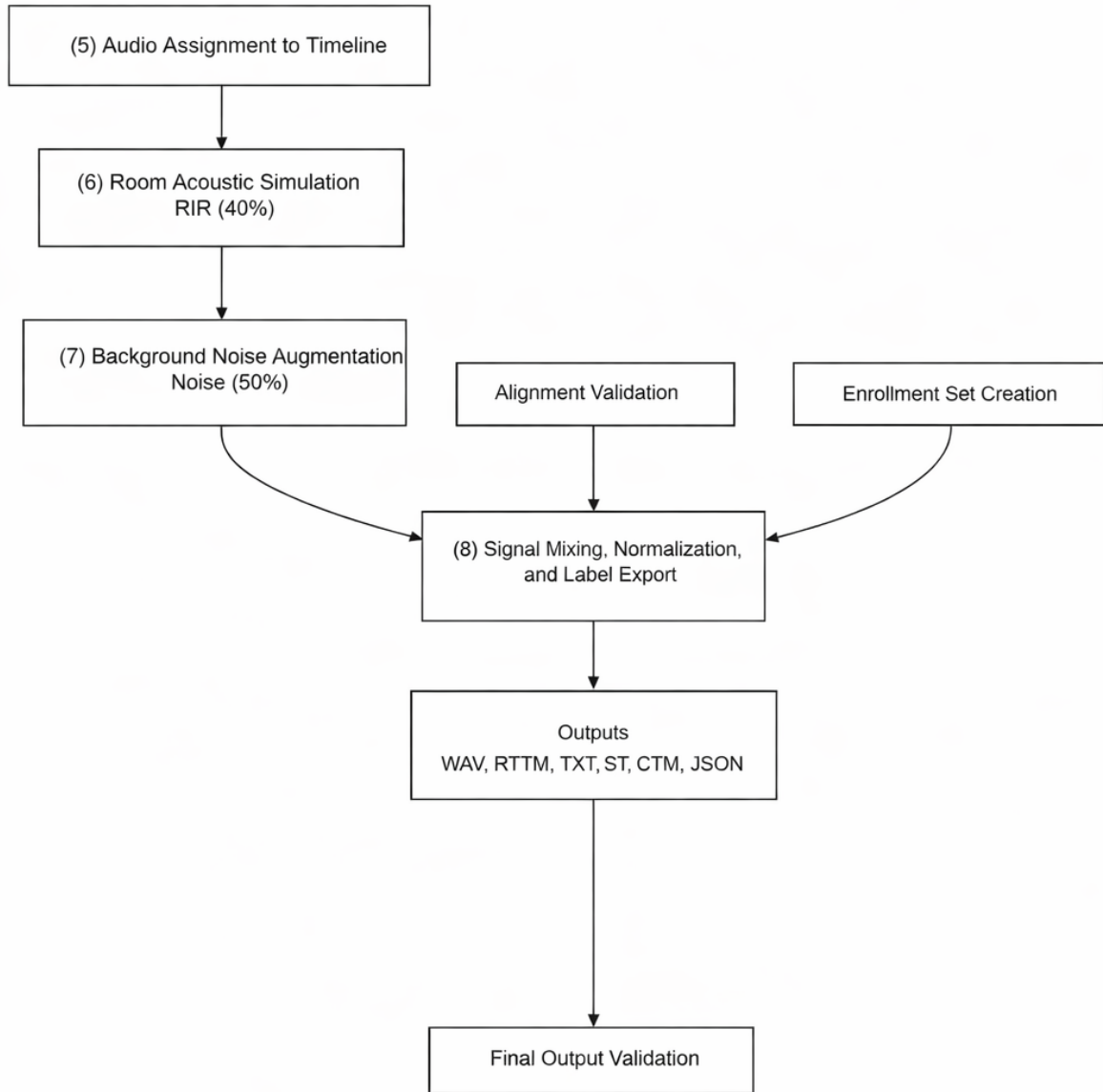


Figure 31: Meeting simulation pipeline, Part 3.

### Segment Preprocessing.

The module preprocesses each audio segment before it enters the simulation pipeline.

### Sample Rate Normalization.

All source data is already in 16 kHz mono, as shown in Table 10, so this step merely checks and typically bypasses resampling. In cases where source data uses a different sample rate, such as 8 kHz or 48 kHz, the module automatically resamples it to 16 kHz.

### Trim Silence.

Remove trailing and leading silence from every file. Silence is defined as frames with an RMS amplitude below the -40 dB threshold, provided the retained segment is at least 50 ms long. This removes background noise and extra silence at both ends of the recording, enhancing input audio quality.

### Segment Pool Management.

The system maintains a tracking list of used segments for each speaker. During meeting generation, the system prioritizes unused segments to prevent content repetition. When all segments for a given speaker are exhausted, the pool resets to continue data generation.

### Segment Merging: The 70/30 Rule.

To naturally simulate how a person speaks several consecutive sentences in a conversation, rather than isolated singular utterances, the system applies a 70/30 segment merging rule.

- **70% of the time:** Merge 2 to 3 consecutive segments from the same speaker into a longer speech block. The pauses between segments are padded with random silence ranging from 100 ms to 300 ms, mimicking natural pauses between sentences. At the junction, a 20 ms crossfade (fade-in/fade-out) is applied to prevent click artifacts.
- **30% of the time:** Retain a single segment, simulating brief utterances in a conversation.

A crucial design aspect is that the pipeline never forcibly truncates audio. If the merged segment ends up being longer than the allocated timeline slot, the system accepts it and updates the end timestamp to the audio's actual length. This guarantees that the entire utterance content is preserved, preventing dropped words that lead to transcript discrepancies.

### Timeline Generation.

The module allocates the meeting time into discrete event slots. Each slot is characterized by a start time, end time, event type, and a list of participating speakers.

The default time allocation ratio is configured according to Table 15.

Table 15: Estimated time allocation ratio for the simulated meeting timeline.

| Event type     | Symbol     | Default ratio | Description                                  |
|----------------|------------|---------------|----------------------------------------------|
| Single speaker | SINGLE_SPK | 60%           | One speaker is talking, others are listening |
| Silence        | SILENCE    | 35%           | Pauses, no one is speaking                   |
| Speech overlap | OVERLAP    | 5%            | Two or more people speaking simultaneously   |

However, when generating data for different overlap levels, from 0% to 30%, this ratio is dynamically adjusted: `silence_ratio` remains fixed at 35%, `overlap_ratio` matches the target level, and `single_speaker_ratio` =  $\max(0.1, 1.0 - \text{silence} - \text{overlap})$ . Tables 16 and 17 present the ratios and actual validation results for each level.

Table 16: Actual overlap validation results for the TRAIN set (399 meetings, 210.34 h).

| Level | single | silence | overlap | actual overlap | actual single | actual silence | Status  |
|-------|--------|---------|---------|----------------|---------------|----------------|---------|
| 0%    | 65.0%  | 35.0%   | 0.0%    | 0.00%          | 84.37%        | 15.63%         | PERFECT |
| 5%    | 60.0%  | 35.0%   | 5.0%    | 5.44%          | 79.55%        | 15.00%         | PASSED  |
| 10%   | 55.0%  | 35.0%   | 10.0%   | 10.75%         | 74.86%        | 14.38%         | PASSED  |
| 15%   | 50.0%  | 35.0%   | 15.0%   | 15.62%         | 70.06%        | 14.32%         | PASSED  |
| 20%   | 45.0%  | 35.0%   | 20.0%   | 22.70%         | 62.13%        | 15.17%         | PASSED  |
| 25%   | 40.0%  | 35.0%   | 25.0%   | 26.56%         | 57.85%        | 15.60%         | PASSED  |
| 30%   | 35.0%  | 35.0%   | 30.0%   | 31.67%         | 53.25%        | 15.08%         | PASSED  |

Table 17: Actual overlap validation results for the VAL set (63 meetings, 33.19 h).

| Level | single | silence | overlap | actual overlap | actual single | actual silence | Status  |
|-------|--------|---------|---------|----------------|---------------|----------------|---------|
| 0%    | 65.0%  | 35.0%   | 0.0%    | 0.00%          | 83.16%        | 16.84%         | PERFECT |
| 5%    | 60.0%  | 35.0%   | 5.0%    | 4.88%          | 79.12%        | 16.00%         | PASSED  |
| 10%   | 55.0%  | 35.0%   | 10.0%   | 10.36%         | 73.44%        | 16.20%         | PASSED  |
| 15%   | 50.0%  | 35.0%   | 15.0%   | 14.35%         | 70.24%        | 15.40%         | PASSED  |
| 20%   | 45.0%  | 35.0%   | 20.0%   | 21.22%         | 62.36%        | 16.42%         | PASSED  |
| 25%   | 40.0%  | 35.0%   | 25.0%   | 25.15%         | 58.96%        | 15.89%         | PASSED  |
| 30%   | 35.0%  | 35.0%   | 30.0%   | 29.83%         | 54.59%        | 15.59%         | PASSED  |

**Remarks.**

All 14 configurations, including 7 Train levels and 7 Val levels, meet the pass criteria with deviation below 3 percentage points. As overlap scales from 0% to 30%, the single-speaker ratio linearly decreases from 65% to 35%, while actual silence fluctuates between 14% and 17%, which is lower than the 35% target because the pipeline prioritizes filling audio slots before silence. The largest overlap deviation occurs at 20% Train, with 22.70% actual overlap versus a target of 20%, corresponding to a 2.70 percentage-point deviation that remains within the acceptable threshold.

```

=== TRAIN ===
=====
OVERLAP VALIDATION REPORT
=====

[OVERLAP_0] - Target: 0%
  Total Meetings: 57
  Total Duration: 29.90 hours
  Actual Overlap: 0.00% ✓ PERFECT
  Actual Single : 84.37%
  Actual Silence: 15.63%

[OVERLAP_5] - Target: 5%
  Total Meetings: 57
  Total Duration: 30.09 hours
  Actual Overlap: 5.44% ✓ PASSED
  Actual Single : 79.55%
  Actual Silence: 15.00%

[OVERLAP_10] - Target: 10%
  Total Meetings: 57
  Total Duration: 30.02 hours
  Actual Overlap: 10.75% ✓ PASSED
  Actual Single : 74.86%
  Actual Silence: 14.38%

[OVERLAP_15] - Target: 15%
  Total Meetings: 57
  Total Duration: 30.20 hours
  Actual Overlap: 15.62% ✓ PASSED
  Actual Single : 70.06%
  Actual Silence: 14.32%

[OVERLAP_20] - Target: 20%
  Total Meetings: 57
  Total Duration: 29.84 hours
  Actual Overlap: 22.70% ✓ PASSED
  Actual Single : 62.13%
  Actual Silence: 15.17%

[OVERLAP_25] - Target: 25%
  Total Meetings: 57
  Total Duration: 29.96 hours
  Actual Overlap: 26.56% ✓ PASSED
  Actual Single : 57.85%
  Actual Silence: 15.60%

[OVERLAP_30] - Target: 30%
  Total Meetings: 57
  Total Duration: 30.34 hours
  Actual Overlap: 31.67% ✓ PASSED
  Actual Single : 53.25%
  Actual Silence: 15.08%
=====

```

TRAIN set

```

=== VAL ===
=====
OVERLAP VALIDATION REPORT
=====

[OVERLAP_0] - Target: 0%
  Total Meetings: 9
  Total Duration: 4.68 hours
  Actual Overlap: 0.00% ✓ PERFECT
  Actual Single : 83.16%
  Actual Silence: 16.84%

[OVERLAP_5] - Target: 5%
  Total Meetings: 9
  Total Duration: 4.78 hours
  Actual Overlap: 4.88% ✓ PASSED
  Actual Single : 79.12%
  Actual Silence: 16.00%

[OVERLAP_10] - Target: 10%
  Total Meetings: 9
  Total Duration: 4.73 hours
  Actual Overlap: 10.36% ✓ PASSED
  Actual Single : 73.44%
  Actual Silence: 16.20%

[OVERLAP_15] - Target: 15%
  Total Meetings: 9
  Total Duration: 4.79 hours
  Actual Overlap: 14.35% ✓ PASSED
  Actual Single : 70.24%
  Actual Silence: 15.40%

[OVERLAP_20] - Target: 20%
  Total Meetings: 9
  Total Duration: 4.77 hours
  Actual Overlap: 21.22% ✓ PASSED
  Actual Single : 62.36%
  Actual Silence: 16.42%

[OVERLAP_25] - Target: 25%
  Total Meetings: 9
  Total Duration: 4.55 hours
  Actual Overlap: 25.15% ✓ PASSED
  Actual Single : 58.96%
  Actual Silence: 15.89%

[OVERLAP_30] - Target: 30%
  Total Meetings: 9
  Total Duration: 4.89 hours
  Actual Overlap: 29.83% ✓ PASSED
  Actual Single : 54.59%

```

VALID set

Figure 32: Actual overlap validation results for the TRAIN and VALID sets.

Speaker selection for each slot is executed by the `_select_speaker()` function using a load-balancing mechanism. At any given moment, the speaker with the lowest total speaking time is given priority. This mechanism ensures that participating speakers are allocated relatively equal speaking time, reflecting conversational fairness in real-world exchanges.

### Overlap Modeling.

Speech overlap slots are categorized into two types to authentically simulate conversational scenarios.

### Turn-Taking.

This accounts for 40% of total overlap events. It occurs when a new speaker begins talking before the current speaker finishes the sentence. Overlap duration ranges from 0.5 s to 2.5 s in Train, or from 0.5 s to 2.0 s in the Val. This is a common phenomenon in natural dialogue, demonstrating continuous and smooth communication among participants.

### Interruption.

This accounts for 60% of total overlap events. It occurs when a speaker abruptly cuts into another person's speech turn. With a longer overlap duration, ranging from 2.0 s to 6.0 s, or from 1.5 s to 5.0 s in the Val configuration, this complex overlap type often poses significant challenges for diarization systems.

For both types, the system imposes a hard constraint: strict maximum of 2 simultaneous speakers at any point in the meeting. This constraint is verified and enforced by the `_validate_max_two_speakers()` function after each timeline generation step.

```
SPEAKER meeting_20260327_110157_89d42a2e 1 0.000 4.212 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 4.539 3.800 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 5.648 4.699 <NA> <NA> VIVOSSPK08 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 10.347 1.794 <NA> <NA> VIVOSSPK08 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 10.706 2.522 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 16.459 6.040 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 22.499 5.225 <NA> <NA> VIVOSSPK08 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 26.084 6.614 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 34.737 5.238 <NA> <NA> VIVOSSPK08 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 36.755 6.801 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 43.967 1.826 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 44.341 2.854 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 48.083 6.743 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 49.973 2.700 <NA> <NA> VIVOSSPK08 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 54.826 3.057 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 59.197 3.207 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 60.078 7.524 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 67.603 4.117 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 69.020 6.376 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 76.933 6.683 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 84.899 6.743 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 91.642 4.478 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 92.137 4.452 <NA> <NA> VIVOSSPK08 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 99.939 10.415 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 102.514 5.839 <NA> <NA> 25226 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 110.653 4.112 <NA> <NA> VIVOSSPK08 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 111.972 6.061 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 119.682 2.641 <NA> <NA> 2F0NM <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 120.702 5.219 <NA> <NA> VIVOSSPK08 <NA> <NA>
SPEAKER meeting_20260327_110157_89d42a2e 1 128.871 6.238 <NA> <NA> 25226 <NA> <NA>
```

Figure 33: Examples of overlapping speaker segments in the RTTM file.

### Analysis of Overlap Segments.

- **Line 2 and Line 3:** Speaker 25226 is talking from 4.539 s to 8.339 s (4.539 + 3.8), but at 5.648 s speaker VIVOSSPK08 jumps in to speak, resulting in an overlap duration of approximately 2.7 s.
- **Line 4 and Line 5:** Speaker VIVOSSPK08 speaks from 10.347 s to 12.141 s, and at 10.706 s speaker 25226 interrupts, producing an overlap of about 1.4 s.
- **Line 13 and Line 14:** Speaker 2F0NM speaks from 48.083 s to 54.826 s, but VIVOSSPK08 suddenly interrupts at 49.973 s.

### Room Acoustic Simulation.

To enhance data realism, the system integrates a simplified room acoustic simulation that approximates room impulse response effects before the signal reaches the microphone [22].

1. **Direct path:** The direct-path component is delayed according to the source-microphone distance, and its amplitude is scaled by distance-based attenuation.
2. **Early reflections:** 20 early reflections are added with delays ranging from 2-50 ms, and their amplitudes decay according to  $0.8^i$ , where  $i$  denotes the reflection order.
3. **Late reverberation:** Gaussian noise with an exponentially decaying envelope starting at 80 ms.

The dry/wet ratio is 0.3, corresponding to 70% original signal and 30% reverb, ensuring that reverb does not overpower the voice.

Room parameters are randomly selected for each meeting following a uniform distribution within the ranges defined in Table 18.

Table 18: Room simulation parameters.

| Parameter                   | Range      | Unit |
|-----------------------------|------------|------|
| Room width                  | 4.0 - 10.0 | m    |
| Room length                 | 5.0 - 12.0 | m    |
| Ceiling height              | 2.5 - 4.0  | m    |
| RT60 (reverb time)          | 0.15 - 0.4 | s    |
| Speaker-microphone radius   | 1.5 - 2.5  | m    |
| Probability of applying RIR | 40%        |      |

Speakers are arranged in a circular formation surrounding the central microphone, with a randomized radius within the above range. This configuration simulates a typical meeting-room setup where the microphone stays in the middle of a table.

Applying Room simulation to 40% of meetings, rather than all meetings. This ensures that the final dataset encompasses both clean samples without reverb and reverberated samples, reflecting the diversity of real-world recording conditions and enhancing the generalization capability of the trained model.

### Background Noise Augmentation.

Background noise is taken from a real-world dataset and mixed into the meeting audio, tightly controlled via the Signal-to-Noise Ratio (SNR) metric.

The noise augmentation process consists of the following steps.

1. Load a random noise file from the library of 947 files.
2. If the noise file is shorter than the meeting audio, load other random noise segments and concatenate them until the resulting signal is long enough.
3. Calculate the RMS energy of the original audio signal and the noise signal.
4. Scale the noise amplitude to achieve the target SNR according to  $SNR_{dB} = 20\log_{10}(RMS_{signal}/RMS_{noise})$ .
5. Add the scaled noise to the meeting audio.
6. Check and normalize the values. If the maximum absolute amplitude exceeds 1.0, the entire signal is divided by the maximum peak value and then multiplied by 0.95. This is anti-clipping that preserves the ratio, rather than hard-clipping individual samples.

Similar to room simulation, noise is only applied to 50% of meetings in order to ensure scenario diversity. The target SNR range is selected as 20-35 dB, corresponding to light-to-moderate noise in actual office environments.

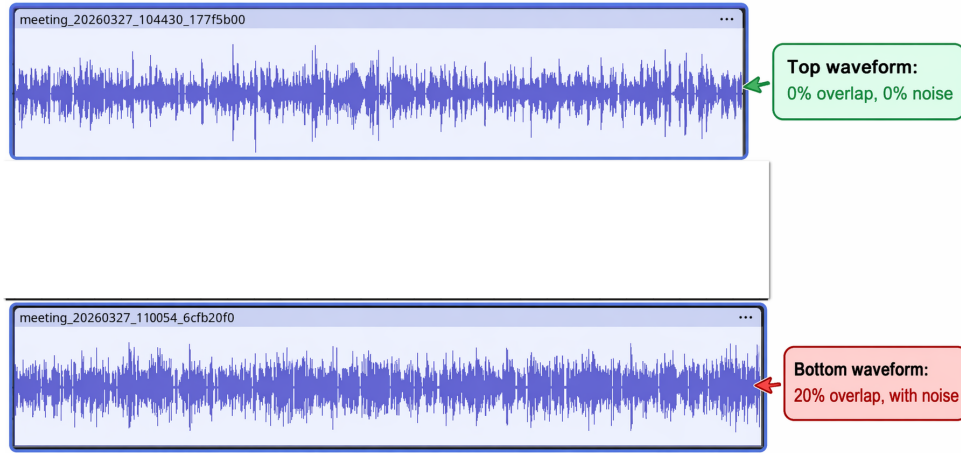


Figure 34: Waveform comparison between a clean meeting and a noisy realistic meeting.

### Audio Signal Analysis.

- **Top waveform (0% overlap, 0% noise):** The signal is visibly thinner, and silent or low-energy regions are extremely clear. Signal peaks stand out sharply against the background.
- **Bottom waveform (20% overlap, with noise):** The signal appears dense and packed almost entirely throughout the duration. There are fewer empty gaps, demonstrating voice overlap. At the same time, the noise floor is significantly higher due to the presence of real-world ambient noise.

#### 4.5.6 Experimental Configurations and Parameters

##### Default Parameters.

Table 19: Default system configuration parameters.

| Parameter Group        | Parameter                    | Value              | Meaning                                 |
|------------------------|------------------------------|--------------------|-----------------------------------------|
| Audio                  | sample_rate                  | 16000 Hz           | Standard sampling rate                  |
| Meeting                | min/max_duration_minutes     | 30 / 35 mins       | Meeting duration                        |
| Meeting                | min/max_speakers             | 2 / 4 people       | Number of participants                  |
| Overlap - Turn-Taking  | turn_taking_ratio            | 0.40 (40%)         | Ratio in total overlap events           |
| Overlap - Turn-Taking  | turn_taking_min/max_sec      | 0.5 / 2.5 s        | Length of turn-taking overlap           |
| Overlap - Interruption | interruption_ratio           | 0.60 (60%)         | Ratio in total overlap events           |
| Overlap - Interruption | interruption_min/max_sec     | 2.0 / 6.0 s        | Length of interruption overlap          |
| Segment                | merge_ratio                  | 0.70 (70%)         | Percentage of segments merged 2-3 times |
| Noise                  | snr_min/max_db               | 20 / 35 dB         | Target SNR range                        |
| Noise                  | noise_probability            | 0.50 (50%)         | Probability of noise in a meeting       |
| Reverb                 | rt60_range                   | 0.15 to 0.40 s     | Reverberation time                      |
| Reverb                 | reverb_probability           | 0.40 (40%)         | Probability of RIR in a meeting         |
| Timeline               | single/silence/overlap_ratio | 0.60 / 0.35 / 0.05 | Time allocation for 3 event types       |

##### Parameters of the Two Data Generation Scripts.

The system employs two separate scripts to generate two distinct datasets with mutually exclusive speakers.

Table 20: Configuration parameters for train.

| Parameter            | Value                                      |
|----------------------|--------------------------------------------|
| Data source          | 18 speakers (6 VIVOS + 6 LSVSC + 6 CV)     |
| Meeting duration     | 30-35 minutes (randomized)                 |
| Overlap levels       | 0%, 5%, 10%, 15%, 20%, 25%, 30% (7 levels) |
| Hours per level      | 30 h per level, 210 h total                |
| Turn-taking min/max  | 0.5 / 2.5 s                                |
| Interruption min/max | 2.0 / 6.0 s                                |
| Noise probability    | 0.50 (50%)                                 |
| Reverb probability   | 0.40 (40%)                                 |

Table 21: Configuration parameters for val.

| Parameter            | Value                                                     |
|----------------------|-----------------------------------------------------------|
| Data source          | 40 VIVOSPK speakers (1-46, except 04, 07, 08, 44, 45, 46) |
| Meeting duration     | 28-35 minutes (randomized)                                |
| Overlap levels       | 0%, 5%, 10%, 15%, 20%, 25%, 30% (7 levels)                |
| Hours per level      | 5 h per level, 33 h total                                 |
| Turn-taking min/max  | 0.5 / 2.0 s                                               |
| Interruption min/max | 1.5 / 5.0 s                                               |
| Noise probability    | 0.50 (50%)                                                |
| Reverb probability   | 0.40 (40%)                                                |

#### 4.5.7 Output Labeling Formats

Each generated meeting comes accompanied by six synchronized label files, catering to various tasks and evaluation tools.

##### WAV.

This is the final audio file, stored in 16 kHz mono PCM 16-bit format. It acts as the direct input for the diarization model.

##### RTTM.

Rich Transcription Time Marks is the NIST standard format for speaker diarization. Each line describes a speaker segment using the syntax `SPEAKER <file_id> 1 <start_time> <duration> <NA> <NA> <speaker_id> <NA> <NA>`. This file is used natively with `dscore` or `pyannote.metrics` to compute Diarization Error Rate (DER).

##### TXT.

This is a word-level transcript mapped as `<start> <end> <speaker_id> [OVERLAP] <word>`. The `[OVERLAP]` tag belongs to words spoken during an overlap segment. This file couples diarization and ASR data, enabling simultaneous evaluation.

##### STM.

Segment Time Marks is the NIST standard format for speech recognition evaluation. In overlapping sections, each speaker is assigned an individual line over the same timespan. It can be used with `sclite` to calculate Word Error Rate (WER).

##### CTM.

Conversation Time Marks provides comprehensive word-level labels, structured as `<file> <channel> <start_time> <duration> <word>`. It is useful for evaluating word-level alignment using `sclite`.

##### JSON.

This file contains complete metadata of the meeting, including the list of speakers, full specifications for every segment, actual single/silence/overlap ratio statistics, noise and reverb flags, room configuration, and the applied random seed for reproducibility.

Table 22: Evaluation purposes and tools for each labeling format.

| Format | Standard | Evaluation Tool          | Target Metric            | Level   |
|--------|----------|--------------------------|--------------------------|---------|
| RTTM   | NIST     | dscore, pyannote.metrics | DER                      | Segment |
| STM    | NIST     | sclite                   | WER                      | Segment |
| CTM    | NIST     | sclite                   | Word alignment           | Word    |
| TXT    | Custom   | None                     | Transcript + Diarization | Word    |
| JSON   | Custom   | None                     | Metadata                 | Meeting |
| WAV    | PCM      | None                     | Audio input              | Meeting |

Example RTTM file content:

```
SPEAKER meeting_20240101_130000_a3f2c1 1 0.000 4.350 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20240101_130000_a3f2c1 1 3.890 2.100 <NA> <NA> VIVOSSPK07 <NA> <NA>
SPEAKER meeting_20240101_130000_a3f2c1 1 6.120 5.230 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20240101_130000_a3f2c1 1 10.950 0.800 <NA> <NA> VIVOSSPK07 <NA> <NA>
```

Lines 1 and 2 overlap: VIVOSPK01 speaks from 0.0 s to 4.35 s, while VIVOSPK07 starts at 3.89 s, resulting in an overlap of 0.46 s.

```
cat data/val/output/overlap_dataset/overlap_10/2speakers/meeting_20260327_112122_ac24f2db.rttm
SPEAKER meeting_20260327_112122_ac24f2db 1 0.000 3.729 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 4.099 5.830 <NA> <NA> VIVOSSPK34 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 7.243 3.213 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 13.459 6.599 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 21.051 10.042 <NA> <NA> VIVOSSPK34 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 29.964 4.450 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 36.664 12.600 <NA> <NA> VIVOSSPK34 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 50.940 9.498 <NA> <NA> VIVOSSPK34 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 59.594 4.844 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 66.105 10.794 <NA> <NA> VIVOSSPK34 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 74.700 5.614 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 81.555 7.556 <NA> <NA> VIVOSSPK34 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 87.706 3.411 <NA> <NA> VIVOSSPK01 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 92.327 7.051 <NA> <NA> VIVOSSPK34 <NA> <NA>
SPEAKER meeting_20260327_112122_ac24f2db 1 101.698 6.555 <NA> <NA> VIVOSSPK01 <NA> <NA>

speakers: ['VIVOSSPK01', 'VIVOSSPK34']
duration: 1748.959875
statistics: {'total_duration': 1748.959875, 'num_segments': 268, 'single_speaker_ratio': 0.7418296267642966,
'silence_ratio': 0.14151275514439202, 'overlap_ratio': 0.11665761809131125, 'speaker_times': {'VIVOSSPK01':
683.9212020892509, 'VIVOSSPK34': 1021.5680355172852}, 'overlap_count': 122}
segments (3 đầu):
{'segment_id': 1, 'start_time': 0.0, 'end_time': 3.729, 'duration': 3.729, 'speaker_id': 'VIVOSSPK01'}
{'segment_id': 2, 'start_time': 4.099, 'end_time': 9.929, 'duration': 5.83, 'speaker_id': 'VIVOSSPK34'}
{'segment_id': 3, 'start_time': 7.243, 'end_time': 10.455, 'duration': 3.213, 'speaker_id': 'VIVOSSPK01'}
```

Figure 35: RTTM label file (top) and JSON metadata statistics (bottom) for a simulated 2-speaker meeting with 10% overlap.

#### 4.5.8 Dataset Splitting Strategy

The dataset is constructed using a stratified by overlap level strategy, serving the research goal of evaluating the impact of overlap ratio on diarization performance. The data is divided into two sets with non-overlapping speakers.

Table 23: Actual statistics for the TRAIN set, 18 speakers.

| Overlap Level | Meetings   | Hours           | Noise              | Reverb             | 2spk       | 3spk       | 4spk       |
|---------------|------------|-----------------|--------------------|--------------------|------------|------------|------------|
| overlap_0     | 57         | 29.90 h         | 34/57 (60%)        | 25/57 (44%)        | 19         | 19         | 19         |
| overlap_5     | 57         | 30.09 h         | 26/57 (46%)        | 24/57 (42%)        | 19         | 19         | 19         |
| overlap_10    | 57         | 30.02 h         | 22/57 (39%)        | 21/57 (37%)        | 19         | 19         | 19         |
| overlap_15    | 57         | 30.20 h         | 28/57 (49%)        | 23/57 (40%)        | 19         | 19         | 19         |
| overlap_20    | 57         | 29.84 h         | 29/57 (51%)        | 22/57 (39%)        | 19         | 19         | 19         |
| overlap_25    | 57         | 29.96 h         | 29/57 (51%)        | 24/57 (42%)        | 19         | 19         | 19         |
| overlap_30    | 57         | 30.34 h         | 34/57 (60%)        | 28/57 (49%)        | 19         | 19         | 19         |
| <b>TOTAL</b>  | <b>399</b> | <b>210.34 h</b> | <b>202 (50.6%)</b> | <b>167 (41.9%)</b> | <b>133</b> | <b>133</b> | <b>133</b> |

#### Train Set Acoustic Conditions Classification.

- Meetings with noise + reverb: approximately 85.
- Meetings with only noise (no reverb): approximately 117.
- Meetings with only reverb (no noise): approximately 82.
- Clean meetings (no noise, no reverb): approximately 115.

Table 24: Actual statistics for the VAL set, 40 speakers.

| Overlap Level | Meetings  | Hours          | Noise             | Reverb            | 2spk      | 3spk      | 4spk      |
|---------------|-----------|----------------|-------------------|-------------------|-----------|-----------|-----------|
| overlap_0     | 9         | 4.68 h         | 4/9 (44%)         | 2/9 (22%)         | 3         | 3         | 3         |
| overlap_5     | 9         | 4.78 h         | 5/9 (56%)         | 4/9 (44%)         | 3         | 3         | 3         |
| overlap_10    | 9         | 4.73 h         | 5/9 (56%)         | 3/9 (33%)         | 3         | 3         | 3         |
| overlap_15    | 9         | 4.79 h         | 6/9 (67%)         | 3/9 (33%)         | 3         | 3         | 3         |
| overlap_20    | 9         | 4.77 h         | 3/9 (33%)         | 5/9 (56%)         | 3         | 3         | 3         |
| overlap_25    | 9         | 4.55 h         | 6/9 (67%)         | 3/9 (33%)         | 3         | 3         | 3         |
| overlap_30    | 9         | 4.89 h         | 3/9 (33%)         | 0/9 (0%)          | 3         | 3         | 3         |
| <b>TOTAL</b>  | <b>63</b> | <b>33.19 h</b> | <b>32 (50.8%)</b> | <b>20 (31.7%)</b> | <b>21</b> | <b>21</b> | <b>21</b> |

#### Val Set Acoustic Conditions Classification.

- Meetings with noise + reverb: approximately 10.
- Meetings with only noise (no reverb): approximately 22.
- Meetings with only reverb (no noise): approximately 10.
- Clean meetings (no noise, no reverb): approximately 21.

In each overlap group, meetings are evenly divided according to the number of participants, namely 2, 3, or 4 speakers, with each category accounting for exactly one-third of the total meetings in that group.

| TRAIN set                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | VAL set                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> overlap_0: 57 meetings   29.90h   noise=34/57   reverb=25/57 2speakers: 19 3speakers: 19 4speakers: 19 overlap_5: 57 meetings   30.09h   noise=26/57   reverb=24/57 2speakers: 19 3speakers: 19 4speakers: 19 overlap_10: 57 meetings   30.02h   noise=22/57   reverb=21/57 2speakers: 19 3speakers: 19 4speakers: 19 overlap_15: 57 meetings   30.20h   noise=28/57   reverb=23/57 2speakers: 19 3speakers: 19 4speakers: 19 overlap_20: 57 meetings   29.84h   noise=29/57   reverb=22/57 2speakers: 19 3speakers: 19 4speakers: 19 overlap_25: 57 meetings   29.96h   noise=29/57   reverb=24/57 2speakers: 19 3speakers: 19 4speakers: 19 overlap_30: 57 meetings   30.34h   noise=34/57   reverb=28/57 2speakers: 19 3speakers: 19 4speakers: 19 GRAND TOTAL MIX: 399 meetings   210.34h With noise: 202   Without: 197 With reverb: 167   Without: 232 </pre> | <pre> VAL DATASET STATS ===== overlap_0: 9 meetings   4.68h   noise=4/9   reverb=2/9 2speakers: 3 3speakers: 3 4speakers: 3 overlap_5: 9 meetings   4.78h   noise=5/9   reverb=4/9 2speakers: 3 3speakers: 3 4speakers: 3 overlap_10: 9 meetings   4.73h   noise=5/9   reverb=3/9 2speakers: 3 3speakers: 3 4speakers: 3 overlap_15: 9 meetings   4.79h   noise=6/9   reverb=3/9 2speakers: 3 3speakers: 3 4speakers: 3 overlap_20: 9 meetings   4.77h   noise=3/9   reverb=5/9 2speakers: 3 3speakers: 3 4speakers: 3 overlap_25: 9 meetings   4.55h   noise=6/9   reverb=3/9 2speakers: 3 3speakers: 3 4speakers: 3 overlap_30: 9 meetings   4.89h   noise=3/9   reverb=0/9 2speakers: 3 3speakers: 3 4speakers: 3 GRAND TOTAL: 63 meetings   33.19h With noise: 32   Without: 31 With reverb: 20   Without: 43 </pre> |

Figure 36: Distribution of the TRAIN and VAL sets across overlap levels and acoustic conditions.

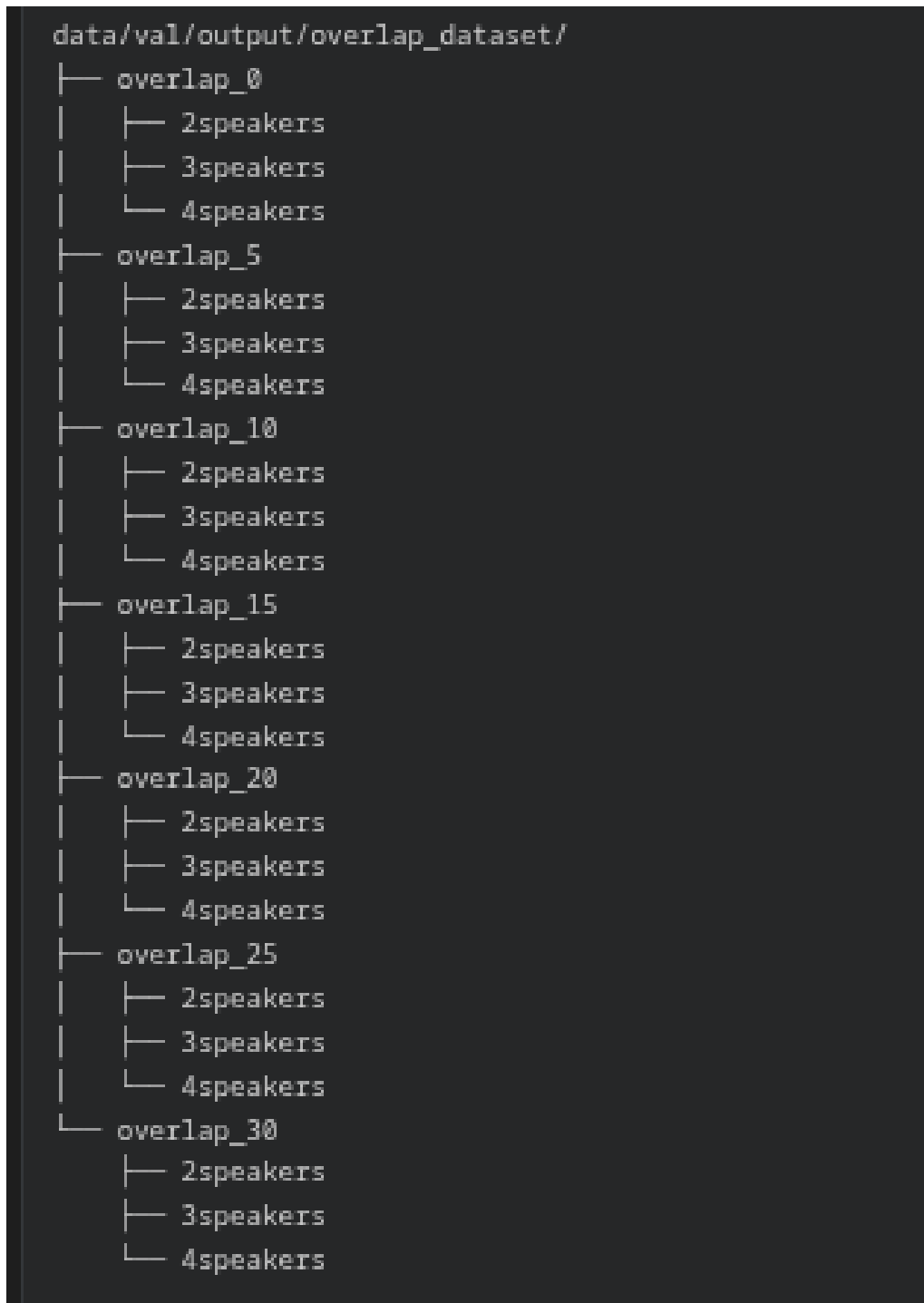


Figure 37: Dataset directory structure.

#### 4.5.9 Dataset Quality Validation

##### Output Consistency Check.

The performs a cross-check among output files of the same meeting.

- It verifies that the WAV audio duration matches the total segment time within the RTTM and JSON files.

- It confirms that the line count in the RTTM file matches the segment count in the JSON file.
- It checks that the TXT file contains the transcript for all segments.
- It detects missing or corrupted files during the writing process.

#### **Actual Overlap Ratio Validation.**

The computes the actual overlap ratio from the JSON file of each meeting and compares it to the target ratio.

- It parses the JSON file and identifies all overlapping temporal segment pairs.
- It calculates total overlap time divided by total meeting duration.
- It compares the measured value to the target overlap level of the respective group.

The evaluation criteria are defined as follows: deviation below 3 percentage points is considered Pass, while deviation greater than or equal to 3 percentage points is considered Fail and requires further investigation.

The actual validation results have already been detailed in Tables 16 and 17. In summary, all 7 overlap levels in both the Train and Val sets achieved the PASS criteria, with deviation below 3 percentage points, and the 0% level reached PERFECT with 0.00% actual overlap.

```

[OVERLAP_0] - Target: 0%
  Total Meetings: 9
  Total Duration: 4.68 hours
  Actual Overlap: 0.00% ✓ PERFECT
  Actual Single : 83.16%
  Actual Silence: 16.84%

[OVERLAP_5] - Target: 5%
  Total Meetings: 9
  Total Duration: 4.78 hours
  Actual Overlap: 4.88% ✓ PASSED
  Actual Single : 79.12%
  Actual Silence: 16.00%

[OVERLAP_10] - Target: 10%
  Total Meetings: 9
  Total Duration: 4.73 hours
  Actual Overlap: 10.36% ✓ PASSED
  Actual Single : 73.44%
  Actual Silence: 16.20%

[OVERLAP_15] - Target: 15%
  Total Meetings: 9
  Total Duration: 4.79 hours
  Actual Overlap: 14.35% ✓ PASSED
  Actual Single : 70.24%
  Actual Silence: 15.40%

[OVERLAP_20] - Target: 20%
  Total Meetings: 9
  Total Duration: 4.77 hours
  Actual Overlap: 21.22% ✓ PASSED
  Actual Single : 62.36%
  Actual Silence: 16.42%

[OVERLAP_25] - Target: 25%
  Total Meetings: 9
  Total Duration: 4.55 hours
  Actual Overlap: 25.15% ✓ PASSED
  Actual Single : 58.96%
  Actual Silence: 15.89%

[OVERLAP_30] - Target: 30%
  Total Meetings: 9
  Total Duration: 4.89 hours
  Actual Overlap: 29.83% ✓ PASSED
  Actual Single : 54.59%
  Actual Silence: 15.59%

```

VAL set

```

[OVERLAP_0] - Target: 0%
  Total Meetings: 57
  Total Duration: 29.90 hours
  Actual Overlap: 0.00% ✓ PERFECT
  Actual Single : 84.37%
  Actual Silence: 15.63%

[OVERLAP_5] - Target: 5%
  Total Meetings: 57
  Total Duration: 30.09 hours
  Actual Overlap: 5.44% ✓ PASSED
  Actual Single : 79.55%
  Actual Silence: 15.00%

[OVERLAP_10] - Target: 10%
  Total Meetings: 57
  Total Duration: 30.02 hours
  Actual Overlap: 10.75% ✓ PASSED
  Actual Single : 74.86%
  Actual Silence: 14.38%

[OVERLAP_15] - Target: 15%
  Total Meetings: 57
  Total Duration: 30.20 hours
  Actual Overlap: 15.62% ✓ PASSED
  Actual Single : 70.06%
  Actual Silence: 14.32%

[OVERLAP_20] - Target: 20%
  Total Meetings: 57
  Total Duration: 29.84 hours
  Actual Overlap: 22.70% ✓ PASSED
  Actual Single : 62.13%
  Actual Silence: 15.17%

[OVERLAP_25] - Target: 25%
  Total Meetings: 57
  Total Duration: 29.96 hours
  Actual Overlap: 26.56% ✓ PASSED
  Actual Single : 57.85%
  Actual Silence: 15.60%

[OVERLAP_30] - Target: 30%
  Total Meetings: 57
  Total Duration: 30.34 hours
  Actual Overlap: 31.67% ✓ PASSED
  Actual Single : 53.25%
  Actual Silence: 15.08%

```

TRAIN set

Figure 38: Output on the VAL and TRAIN sets.

#### 4.5.10 Overall Dataset Statistics

Table 25: General statistics of the training and validation datasets.

| Attribute                     | Train Set                       | Val Set                         |
|-------------------------------|---------------------------------|---------------------------------|
| Total speakers                | 18 (6 VIVOS + 6 LSVSC + 6 CV)   | 40 (VIVOSSPK)                   |
| Total audio duration          | 210.34 hours                    | 33.19 hours                     |
| Number of meetings            | 399 meetings                    | 63 meetings                     |
| Duration per meeting          | 30-35 minutes                   | 28-35 minutes                   |
| Ratio of meetings with noise  | 202/399 (50.6%)                 | 32/63 (50.8%)                   |
| Ratio of meetings with reverb | 167/399 (41.9%)                 | 20/63 (31.7%)                   |
| Noise SNR range               | 20-35 dB                        | 20-35 dB                        |
| Reverb RT60 range             | 0.15-0.4 s                      | 0.15-0.4 s                      |
| Participants per meeting      | 2, 3, or 4                      | 2, 3, or 4                      |
| Overlap levels                | 0%, 5%, 10%, 15%, 20%, 25%, 30% | 0%, 5%, 10%, 15%, 20%, 25%, 30% |
| Sample rate                   | 16000 Hz, mono                  | 16000 Hz, mono                  |
| Label formats                 | WAV, RTTM, TXT, STM, CTM, JSON  | WAV, RTTM, TXT, STM, CTM, JSON  |
| Noise sources                 | 947 WAV files                   | 947 WAV files                   |

#### Pipeline Grand Total.

The complete pipeline produces 462 meetings, 243.53 hours of audio, and 52 independent speakers.

## 4.6 DiariZen Model: Overview and Architecture

### 4.6.1 Overview and Basic Model Information

DiariZen is an open-source toolkit for speaker diarization, developed by the BUT-FIT group (Brno University of Technology). The system is built on the AudioZen platform and integrated with the Pyannote 3.1 library, following the EEND-VC (End-to-End Neural Diarization with Vector Clustering) architectural framework. The base model used in this study is BUT-FIT/diarizen-wavlm-large-s80-md-v2, loaded directly from Hugging Face Model Hub. The notation s80 refers to the training segment size of 80 seconds; md denotes the medium model; and v2 is the second improved version.

Regarding the overall architecture, the EEND module inside DiariZen combines two core components: the WavLM Large feature extractor with a learnable weighted sum mechanism across all Transformer layers, and the Conformer network followed by a classification head using powerset encoding [23]. Representations from each WavLM layer are aggregated, projected through a linear layer with layer normalization, and fed into the Conformer to predict diarization states frame by frame.

WavLM Large is a self-supervised learning (SSL) model pre-trained for comprehensive speech processing, featuring a 24-layer Transformer architecture with approximately 316 million parameters. The model was pre-trained on over 94,000 hours of unlabeled audio and achieves superior performance on many downstream tasks including speaker diarization, speaker verification, speech separation, and ASR. Han et al. [5] confirm that combining WavLM with Conformer in the EEND-VC framework achieves relative DER reductions of 15.8% to 26.6% compared to Pyannote 3.1 baseline on the AMI, AISHELL-4, and AliMeeting benchmarks.

The EEND-VC framework was proposed to combine the overlap-handling ability of EEND with the robustness of clustering-based diarization for long recordings and an arbitrary number of speakers [24]. In EEND-VC, EEND is applied locally on each short window to generate activity probabilities and speaker embeddings, then a global clustering step (Vector Clustering: VC) is performed to map speaker labels consistently throughout the entire recording.

### Modified EEND Module Architecture in DiariZen

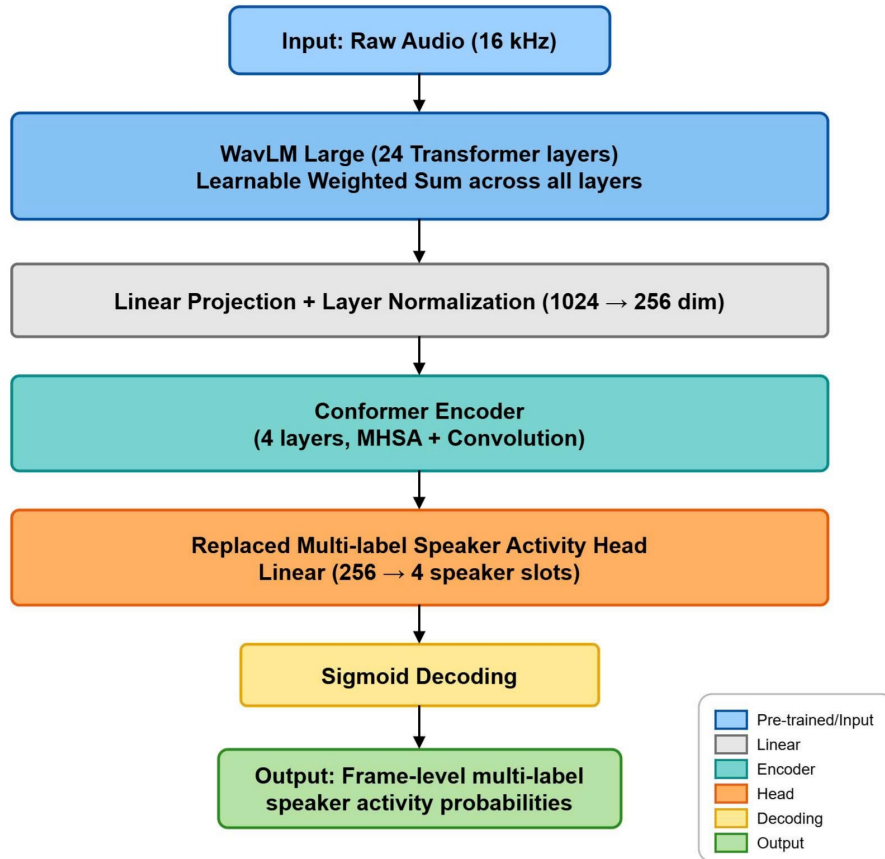


Figure 39: Modified EEND Module Architecture in DiariZen, adapted from [5].

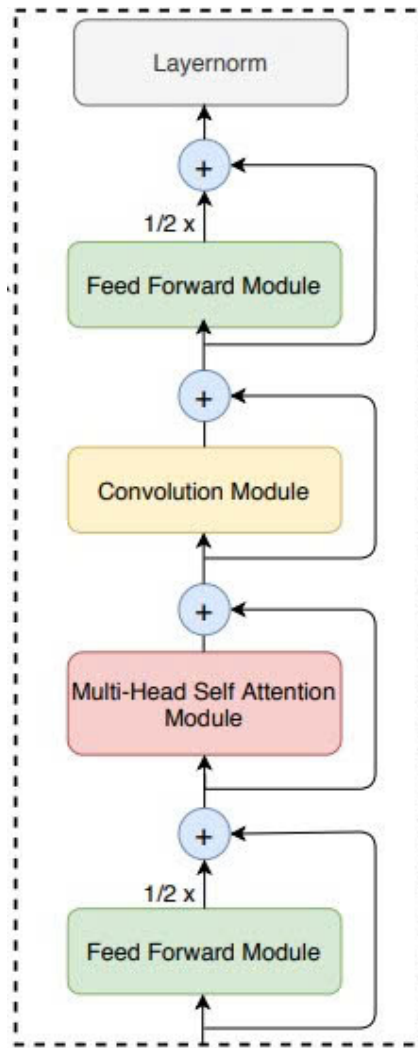


Figure 40: Conformer block architecture, reproduced from [25].

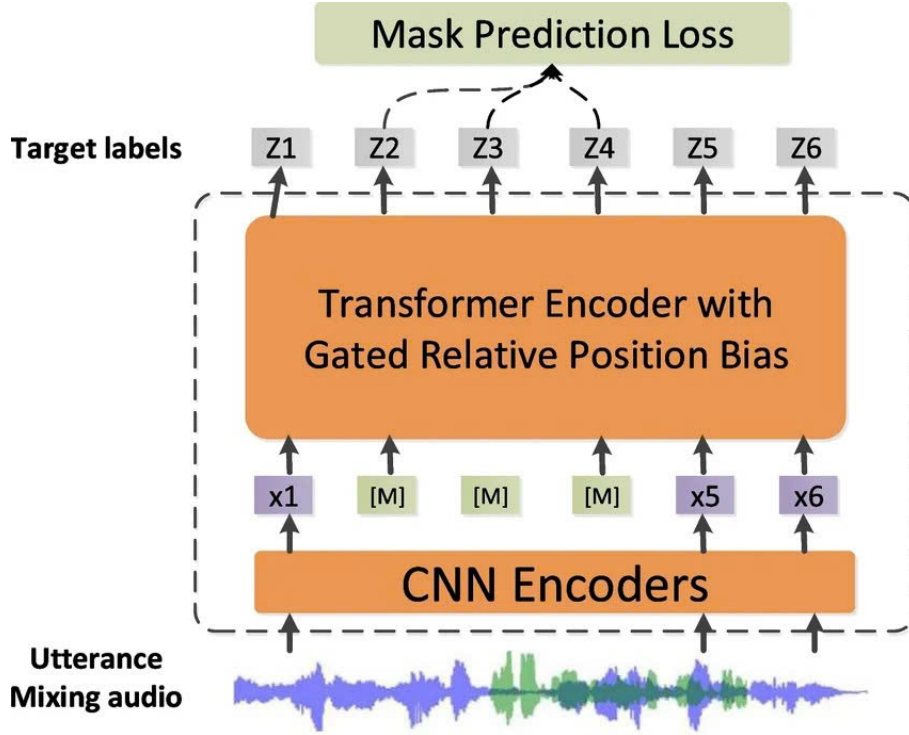


Figure 41: WavLM Architecture, reproduced from [23].

#### 4.6.2 Technical Parameters to Clarify

Regarding its operational nature, the DiariZen-based pipeline is not a simple voice activity detection (VAD) system. It follows a multi-stage diarization process inspired by the EEND-VC framework: generating per-chunk speaker activity probabilities, smoothing frame-level predictions, building speaker centroids from embeddings, assigning local tracks to global speaker identities, filtering silence, and exporting the final RTTM file.

#### 4.6.3 PIT Multilabel BCE: DiariZen

DiariZen is trained with multilabel binary cross-entropy combined with Permutation Invariant Training (PIT). PIT selects the optimal speaker permutation before backpropagation, making the loss permutation-invariant: suited to the inherent label-permutation ambiguity in diarization.

$$\mathcal{L}_{\text{PIT}} = \min_{\sigma \in \text{Perm}(K)} \frac{1}{T \cdot K} \sum_{t=1}^T \sum_{k=1}^K \text{BCE}(\hat{y}_{t,k}, y_{t,\sigma(k)})$$

$K$  = speaker slots |  $\sigma^*$  = best permutation | Gradient only through  $\sigma^*$  | BCE = Binary Cross-Entropy

Figure 42: PIT multilabel BCE loss used in DiariZen, reproduced from [26].

#### 4.6.4 Training and Test Data

The self-labeled data is the primary evaluation set of this study as it reflects real Vietnamese conversational conditions. This set includes multi-speaker recordings (3-13 speakers) from various content genres, manually annotated, and serves as the basis for evaluating real-world deployment capability. The synthetic data from the VIVOS corpus with controlled overlap levels (0%, 5%, 10%), playing a supplementary role in controlled condition experiments: because it is automatically synthesized, parameters such as number of speakers, overlap level, and noise can be precisely adjusted, enabling quantitative analysis along each dimension separately. Both DiariZen and Pyannote branches use the same dataset.

**Note on speaker count distribution:** One clarification is necessary regarding the apparent mismatch between the self-labeled data range (3–13 speakers) and the per-chunk speaker limits of each model (4 for DiariZen, 3 for Pyannote). These limits apply within each short processing window 16s for DiariZen, 10s for Pyannote not to the total number of distinct speakers in the full recording. Files with 8–13 speakers are handled correctly provided that no single window contains more simultaneous speakers than the model’s local capacity, a condition that holds in practice since natural conversation rarely involves more than 2–3 voices overlapping within any given 10–16 second segment.

Table 26: Self-labeled data distribution.

| Data Source                     | Set   | Files     | Total Hours    | Speaker Range |
|---------------------------------|-------|-----------|----------------|---------------|
| <b>TRAIN</b>                    |       |           |                |               |
| Chuyen_Ho_Chuyen_Minh           | Train | 43        | 14.77 h        | 4-13          |
| VIF                             | Train | 20        | 3.37 h         | 3-6           |
| Coi_mo                          | Train | 15        | 4.22 h         | 4-5           |
| Dustin_on_go                    | Train | 8         | 1.74 h         | 4             |
| <b>Total Train self-labeled</b> |       | <b>86</b> | <b>24.09 h</b> | <b>3-13</b>   |
| <b>VALIDATION</b>               |       |           |                |               |
| VIF                             | Val   | 2         | 2.15 h         | 4-5           |
| Dustin_on_go                    | Val   | 1         | 1.19 h         | 4             |
| Chuyen_Ho_Chuyen_Minh           | Val   | 1         | 0.99 h         | 4             |
| Coi_mo                          | Val   | 2         | 0.94 h         | 4-5           |
| <b>Total Val self-labeled</b>   |       | <b>6</b>  | <b>5.27 h</b>  | <b>4-5</b>    |
| <b>TEST</b>                     |       |           |                |               |
| VIF                             | Test  | 2         | 2.52 h         | 4-5           |
| Chuyen_Ho_Chuyen_Minh           | Test  | 1         | 1.21 h         | 4             |
| Dustin_on_go                    | Test  | 1         | 1.19 h         | 4             |
| Coi_mo                          | Test  | 2         | 1.07 h         | 4-5           |
| <b>Total Test self-labeled</b>  |       | <b>6</b>  | <b>6.00 h</b>  | <b>4-5</b>    |
| <b>TOTAL SELF-LABELED</b>       |       | <b>98</b> | <b>35.36 h</b> | <b>3-13</b>   |

#### 4.6.5 Three-Phase Fine-tuning Strategy (Gradual Unfreezing)

The gradual unfreezing method is applied to preserve WavLM Large’s general knowledge while progressively adapting to the Vietnamese data distribution [27]. The reason for applying this method

Table 27: Synthetic Data distribution data\_mix.

| Data Group                       | Set   | Files      | Total Hours    | Speaker Range |
|----------------------------------|-------|------------|----------------|---------------|
| <b>TRAIN</b>                     |       |            |                |               |
| 0% overlap                       | Train | 43         | 26.8 h         | 2-4           |
| 5% overlap                       | Train | 39         | 24.4 h         | 2-4           |
| 10% overlap                      | Train | 39         | 24.3 h         | 2-4           |
| <b>Total Synthetic Data</b>      |       | <b>121</b> | <b>75.54 h</b> | <b>2-4</b>    |
| <b>VALIDATION</b>                |       |            |                |               |
| Synthetic data                   | Val   | 10         | 4.66 h         | 2-4           |
| <b>Total Val Synthetic Data</b>  |       | <b>10</b>  | <b>4.66 h</b>  | <b>2-4</b>    |
| <b>TEST</b>                      |       |            |                |               |
| Synthetic data                   | Test  | 10         | 4.70 h         | 2-4           |
| <b>Total Test Synthetic Data</b> |       | <b>10</b>  | <b>4.70 h</b>  | <b>2-4</b>    |
| <b>TOTAL Synthetic Data</b>      |       | <b>141</b> | <b>84.90 h</b> | <b>2-4</b>    |

instead of full-model fine-tuning stems from the problem characteristics: WavLM Large has already learned rich acoustic representations on a very large unlabeled dataset, and updating all parameters immediately on a smaller target set risks degrading those general features: known as catastrophic forgetting [28]. Gimeno-Gómez et al. [29] report that continual-learning-inspired fine-tuning strategies can reduce forgetting and improve generalization when adapting speech self-supervised representations.

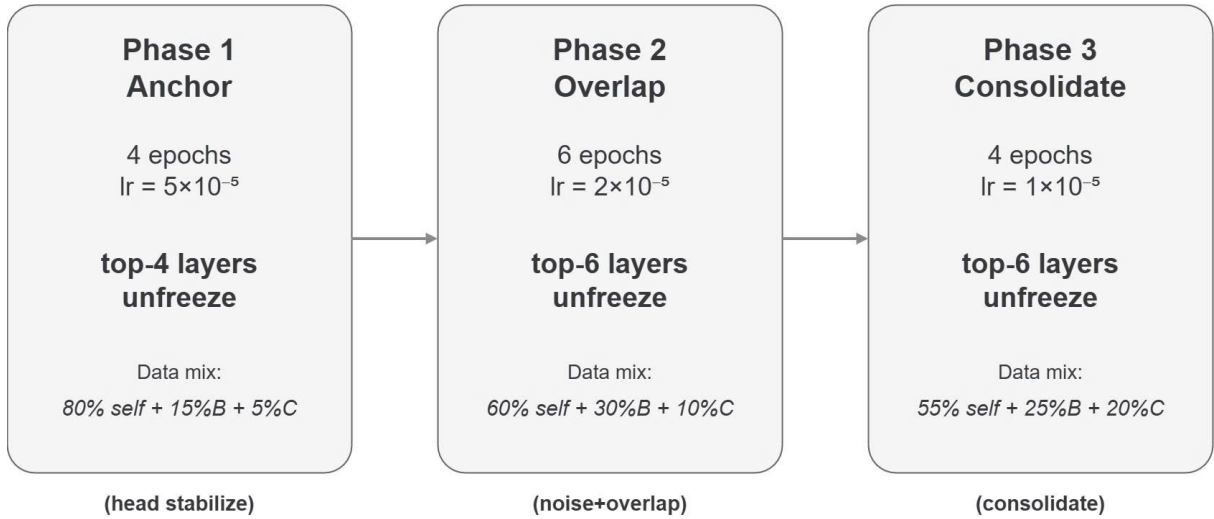


Figure 43: Three-phase gradual unfreezing fine-tuning strategy.

#### 4.6.6 Head Replacement and Multilabel Head Initialization

Before fine-tuning begins, the classifier head of the original model is replaced with a new linear layer (`out_features = max_speakers = 4`) followed by `nn.Identity()` as the activation, effectively switching the output space from powerset softmax to per-speaker sigmoid. The original head produces powerset logits over  $C$  classes, where  $C = 22$  for a 6-speaker configuration or  $C = 11$  for a 4-speaker configuration with maximum pairwise overlap; neither matches the target formulation of

Table 28: Detailed fine-tuning configuration: DiariZen (BUT-FIT/diarizen-wavlm-large-s80-md-v2).

| Parameter                                       | Value                                                                 |
|-------------------------------------------------|-----------------------------------------------------------------------|
| <b>Model &amp; Architecture</b>                 |                                                                       |
| Base Model                                      | BUT-FIT/diarizen-wavlm-large-s80-md-v2                                |
| Backbone                                        | WavLM Large (24 layers, 316M params)                                  |
| Decoder                                         | Conformer + Classification Head                                       |
| Head (after replacement)                        | Linear(out=4) + Sigmoid (multilabel, 4 speakers)                      |
| Head Init                                       | Xavier uniform                                                        |
| <b>Fine-tuning Strategy: Gradual Unfreezing</b> |                                                                       |
| Freeze strategy                                 | top_N (progressively unfreezing from top)                             |
| Phase 1: Unfreeze                               | top-4 layers                                                          |
| Phase 1: LR                                     | 5e-5                                                                  |
| Phase 1: Epochs                                 | 4                                                                     |
| Phase 1: Augmentation                           | False                                                                 |
| Phase 1: Synthetic Data                         | 80% self-labeled; 15% tier B (overlap 5-10%); 5% tier C (overlap 15%) |
| Phase 2: Unfreeze                               | top-6 layers                                                          |
| Phase 2: LR                                     | 2e-5                                                                  |
| Phase 2: Epochs                                 | 6                                                                     |
| Phase 2: Augmentation                           | True                                                                  |
| Phase 2: Synthetic Data                         | 60% self-labeled; 30% tier B; 10% tier C                              |
| Phase 3: Unfreeze                               | top-6 layers                                                          |
| Phase 3: LR                                     | 1e-5                                                                  |
| Phase 3: Epochs                                 | 4                                                                     |
| Phase 3: Augmentation                           | True                                                                  |
| Phase 3: Synthetic Data                         | 55% self-labeled; 25% tier B; 20% tier C                              |
| <b>Loss Function &amp; Weights</b>              |                                                                       |
| Loss function                                   | PIT multilabel BCE (pit_loss)                                         |
| overlap_loss_weight                             | 1.7 (phase 2-3); 1.4 (phase 1)                                        |
| silence_loss_weight                             | 1.0                                                                   |
| triple_loss_weight                              | 0.0: triple overlap frames masked from gradient                       |
| boundary_ignore_sec                             | downweight $\times$ 0.25 for boundary frames                          |
| <b>Optimization</b>                             |                                                                       |
| Batch Size                                      | 6                                                                     |
| Gradient Accumulation                           | 2 (effective batch = 12)                                              |
| Optimizer                                       | AdamW                                                                 |
| Gradient Clipping                               | 0.5                                                                   |
| Weight Decay                                    | 0.01                                                                  |
| Precision                                       | Mixed Precision BFloat16 (bfloat16, AMP)                              |
| <b>Inference Pipeline Configuration</b>         |                                                                       |
| Chunk size                                      | 16.0 s (CHUNK_SEC = 16.0)                                             |
| Sampling rate                                   | 16 kHz                                                                |
| Decode threshold                                | 0.25 (sigmoid multilabel)                                             |
| Median filter window                            | MED_WIN = 11 frames                                                   |
| High centroid threshold                         | HIGH_THR_FIXED = 0.70                                                 |
| Default n_clusters                              | FIXED_N_CLUSTERS = 3                                                  |
| Max speakers                                    | 4                                                                     |
| <b>Validation</b>                               |                                                                       |
| Crop-level validation                           | BCE-PIT loss + PIT-DER on 16 s crops                                  |
| File-level validation                           | DER with collar = 0.25 s (pyannote.metrics)                           |
| Metric for best ckpt                            | file-level DER                                                        |

this study. After the replacement, the model output tensor takes the shape  $[B, T, S]$  with  $S = 4$  independent speaker slots, in place of the original  $[B, T, K]$  powerset class layout, and the decoding decision shifts from a single argmax over  $K$  classes to four independent threshold comparisons over sigmoid-activated logits.

Loss weights are assigned per frame type according to the number of simultaneously active speakers  $n_{\text{act}}$ . Silence frames ( $n_{\text{act}} = 0$ ) are weighted by `silence_loss_weight = 1.0`; two-speaker overlap frames ( $n_{\text{act}} = 2$ ) are upweighted via `overlap_loss_weight = 1.7` to emphasize overlap learning; and triple-overlap frames ( $n_{\text{act}} \geq 3$ ) are assigned `triple_loss_weight = 0.0`, masking them entirely from gradient computation on the grounds that three-way overlap is too acoustically ambiguous to provide a reliable training signal. In addition, frames falling within `boundary_ignore_sec` seconds of the start or end of each chunk are downweighted by a factor of 0.25 to reduce the influence of boundary artifacts introduced by the sliding-window segmentation.

#### 4.6.7 Two-Level Validation: Crop-level and File-level DER

DiariZen performs two separate validation procedures with different meanings. The first is crop-level validation (`run_validation`): computes BCE-PIT loss and PIT-DER on each independent 16-second crop, used to select the best checkpoint during training: this is the DER value shown in training graphs. The second is heavy file-level validation (`heavy_file_level_der_eval`): runs inference on the full file with overlap-aware stitching via a sliding window, applies Hungarian assignment to align speaker order between consecutive chunks, and computes DER with `collar = 0.25s` via `pyannote.metrics`. The DER from this validation is typically lower and more realistic as it reflects performance on the full file rather than individual crop segments.

### 4.7 Pyannote Model: Overview and Architecture

#### 4.7.1 Pipeline Overview and Architecture

The base model in the Pyannote branch is `pyannote/speaker-diarization-community-1`: the open-community version of the `pyannote.audio` toolkit. This pipeline inherits the standard three-step architecture from `pyannote.audio 2.1` [2]. Pyannote is an open-source toolkit built on PyTorch, providing trainable model blocks and pre-trained pipelines for tasks such as speech activity detection, speaker change detection, overlapped speech detection, and speaker embedding.

**(1) Local Speaker Segmentation:** Applies a segmentation model on a short sliding window. In the codebase of this study, the `PyannoteSegWrapper` wrapper class is used to isolate the segmentation model and support both architecture groups: SSL backbone (WavLM/Wav2Vec2/HuBERT) and SincNet + LSTM.

**(2) Speaker Embedding Extraction:** Embeddings are extracted for each detected segment using an independent speaker encoder model. In the current pipeline, embeddings are obtained from `speechbrain/spkrec-ecapa-voxceleb`.

**(3) Global Speaker Clustering:** Agglomerative Hierarchical Clustering (AHC) is applied on the embeddings to group segments belonging to the same speaker throughout the entire recording.

Speaker segmentation in Pyannote applies powerset multi-class cross-entropy loss, replacing traditional multi-label classification. In powerset, each combination of active speaker states is mapped to

a single class, converting the multi-label problem to single-label multi-class classification and eliminating the need for threshold binarization.

### Modified EEND Module Architecture in Pyannote

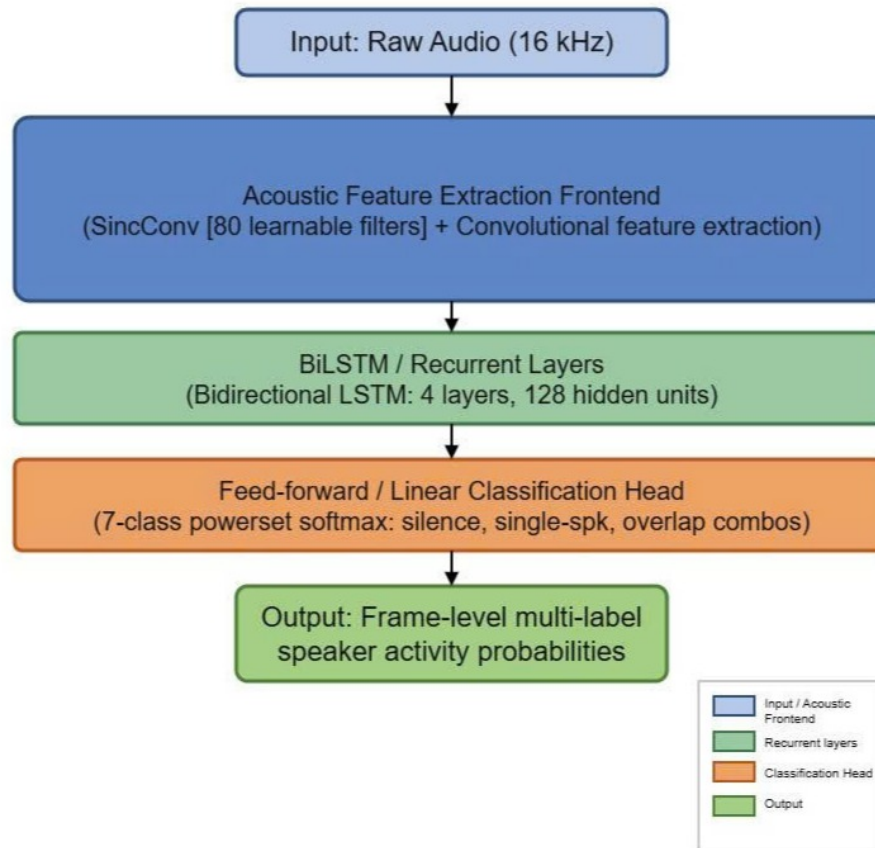


Figure 44: Modified EEND Module Architecture in Pyannote, adapted from [30].

### 4.7.2 Technical Constraints to Clarify

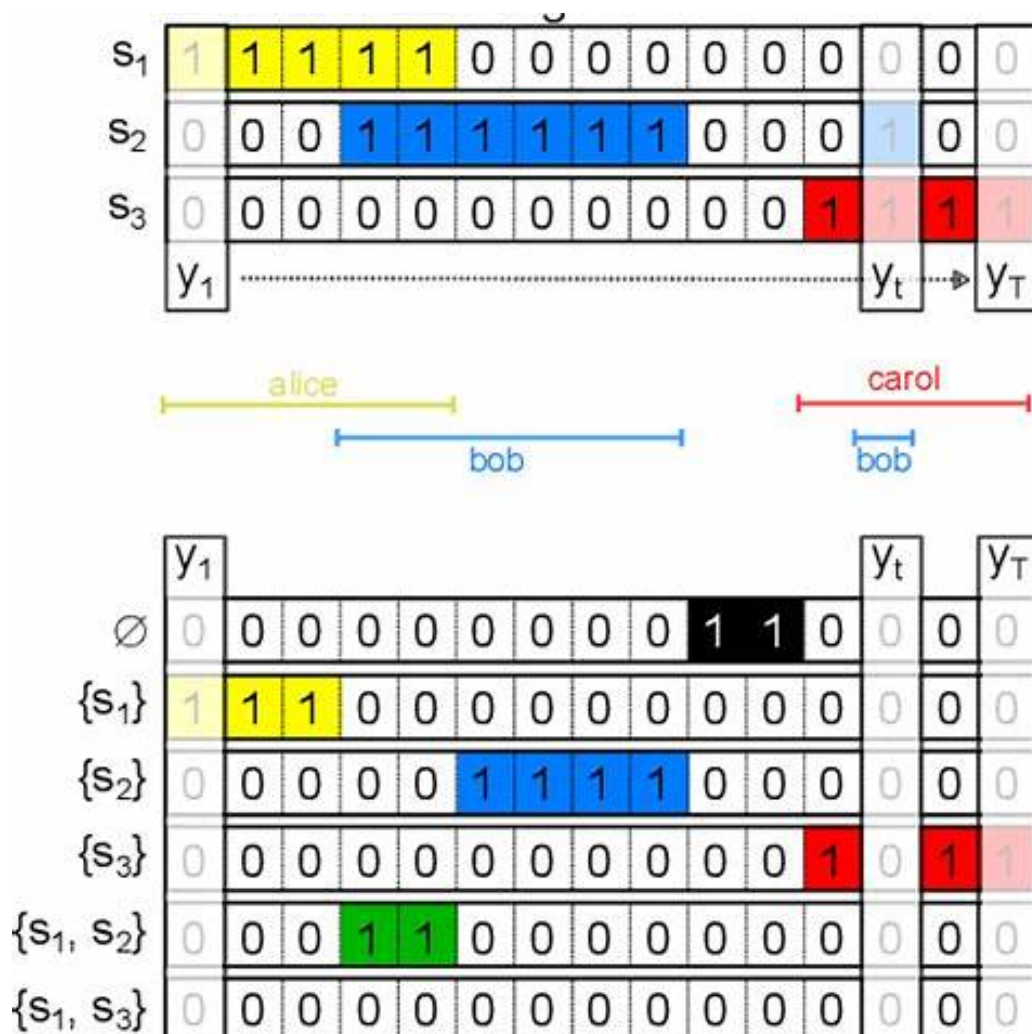


Figure 45: Powerset segments, reproduced from [31].

**model\_max\_speakers\_per\_chunk = 3** is the upper bound of the problem within each 10-second chunk; it does not mean every chunk has exactly 3 speakers.

**model\_max\_speakers\_per\_frame = 2** defines the assumption that each frame has at most 2 overlapping speakers, consistent with the powerset configuration of Plaquet & Bredin (2023).

**max\_speakers = 4** is the scan limit at the file level; **eval\_max\_total\_speakers = 4** is the limit used for DER reporting: DER is only computed on files with no more than 4 speakers.

**Decode mode: argmax\_powerset:** selects the powerset class with the highest probability. This is a key structural difference from DiariZen: external threshold does not intervene in Pyannote's segmentation decision, therefore threshold sweeps have almost no effect on results.

### 4.7.3 Powerset Cross-Entropy: Pyannote

Pyannote uses powerset multi-class cross-entropy, where each class represents a distinct subset of simultaneously active speakers. This reformulates overlap prediction as a single-label classification

over a powerset space; the decision is made by  $\text{argmax}$  with no external threshold required.

$$\mathcal{L}_{\text{PS}} = - \sum_{t=1}^T \sum_{c=1}^C y_{t,c} \log \hat{p}_{t,c} \quad C = 1 + K + \frac{K(K-1)}{2}$$

$C$  = powerset classes (e.g.  $K = 3 \Rightarrow C = 7$ ) | Decision:  $\text{argmax}_c \hat{p}_{t,c}$  | No external threshold needed

Figure 46: Powerset cross-entropy loss used in Pyannote, reproduced from [31].

Loss space:  $[0, \ln C]$  where  $C = 1 + K + K(K-1)/2$  (e.g.,  $K = 3 \rightarrow C = 7$  classes). Because the loss space scale differs fundamentally from PIT BCE, direct numerical comparison of loss values across the two models is meaningless.

Loss values are not comparable across models: read convergence trends within each model only.

#### 4.7.4 Training and Test Data

The Pyannote branch uses the same training, validation, and test datasets as the DiariZen branch: self-labeled conversational recordings spanning 3-13 speakers and Synthetic data with controlled overlap levels across 2-4 speakers. The only difference in data usage arises at evaluation time, where Pyannote applies the filter `eval_max_total_speakers = 4`, restricting DER computation to files whose total speaker count does not exceed this threshold.

Table 29: Detailed fine-tuning configuration: Pyannote (pyannote/speaker-diarization-community-1).

| Parameter                                       | Value                                              |
|-------------------------------------------------|----------------------------------------------------|
| <b>Model &amp; Architecture</b>                 |                                                    |
| Base Pipeline                                   | pyannote/speaker-diarization-community-1           |
| Segmentation model                              | pyannote/segmentation-3.0                          |
| Embedding model                                 | speechbrain/spkrec-ecapa-voxceleb                  |
| Clustering                                      | Agglomerative Hierarchical Clustering (AHC)        |
| model_max_speakers_per_chunk                    | 3 (CHECK-ONLY, policy = warn)                      |
| model_max_speakers_per_frame                    | 2                                                  |
| max_speakers (file level)                       | 4                                                  |
| eval_max_total_speakers                         | 4                                                  |
| <b>Fine-tuning Strategy: Gradual Unfreezing</b> |                                                    |
| Freeze strategy                                 | top_N (progressively unfreezing from top)          |
| Phase 1: Unfreeze                               | top-4 layers (phase1_top4_anchor)                  |
| Phase 1: LR                                     | 5e-5                                               |
| Phase 1: Epochs                                 | 4                                                  |
| Phase 1: Augmentation                           | False                                              |
| Phase 1: Synthetic Data                         | 80% self-labeled 15% tier B 5% tier C              |
| Phase 2: Unfreeze                               | top-6 layers (phase2_top6_overlap)                 |
| Phase 2: LR                                     | 2e-5                                               |
| Phase 2: Epochs                                 | 6                                                  |
| Phase 2: Augmentation                           | True                                               |
| Phase 2: Synthetic Data                         | 60% self-labeled 30% tier B 10% tier C             |
| Phase 3: Unfreeze                               | top-6 layers (phase3_top6_consolidate)             |
| Phase 3: LR                                     | 1e-5                                               |
| Phase 3: Epochs                                 | 4                                                  |
| Phase 3: Augmentation                           | True                                               |
| Phase 3: Synthetic Data                         | 55% self-labeled 25% tier B 20% tier C             |
| <b>Loss Function &amp; Weights</b>              |                                                    |
| Loss function                                   | Powerset cross-entropy + PIT                       |
| overlap_loss_weight                             | 1.7 (phase 2-3); 1.4 (phase 1)                     |
| silence_loss_weight                             | 1.0                                                |
| Decode mode (validation)                        | argmax_powerset (no external threshold used)       |
| <b>Optimization</b>                             |                                                    |
| Batch Size                                      | 128                                                |
| Gradient Accumulation                           | 1                                                  |
| Optimizer                                       | AdamW                                              |
| Gradient Clipping                               | 0.5                                                |
| Weight Decay                                    | 0.01                                               |
| Precision                                       | Mixed Precision BFloat16 (bfloat16, AMP)           |
| <b>Inference Pipeline Configuration</b>         |                                                    |
| Sampling rate                                   | 16 kHz                                             |
| Decode mode                                     | argmax on powerset classes (no external threshold) |
| Threshold sweep                                 | Almost no effect due to argmax-based decode        |
| <b>Validation</b>                               |                                                    |
| Crop-level validation                           | Powerset cross-entropy + PIT-DER on short crops    |
| File-level validation                           | DER with collar = 0.25s (pyannote.metrics)         |
| Metric for best ckpt                            | file-level DER                                     |
| Training history                                | Saved to training_history.json                     |

#### 4.7.5 Pyannote Pipeline During Training and Inference

The Pyannote branch fine-tuning script executes the following steps. First, the script scans and filters data, restricting to files with speaker count not exceeding the evaluation threshold (`eval_max_total_speakers = 4`). Next, the `pyannote/speaker-diarization-community-1` pipeline is loaded from Hugging Face Hub, the segmentation model is extracted and wrapped with `PyannoteSegWrapper`, then enters the phase training loop and saves.

During pipeline loading, several compatibility patches are applied: replacing the segmentation alias with `pyannote/segmentation-3.0`, replacing the embedding alias with `speechbrain/spkrec-ecapa-voxceleb`, and switching clustering to `AgglomerativeClustering` when needed. These are technical implementation notes to ensure library compatibility, not changes to the pipeline's underlying principles.

The loss function used is powerset cross-entropy with PIT (Permutation Invariant Training), combined with weights: `overlap_loss_weight = 1.7` (phases 2-3) and `silence_loss_weight = 1.0`. The decode step during validation uses `argmax_powerset` mode: selecting the powerset class with the highest probability: rather than threshold binarization as in `DiariZen`.

Methodological note: The loss values of `DiariZen` (PIT multilabel BCE) and `Pyannote` (powerset cross-entropy) cannot be directly compared because the two loss functions measure different quantities in different problem spaces. The absolute loss values do not reflect relative quality between the two models. Each training loss graph is only useful for tracking the convergence trend of that individual model.

### 4.8 PhoWhisper-large: Overview and Architecture

#### 4.8.1 Research Background

Automatic Speech Recognition (ASR) for Vietnamese presents specific challenges: a 6-tone system, North-Central-South dialectal diversity, and a large gap between studio-quality and real-world (in-the-wild) audio. This study applies `PhoWhisper-large` which `Whisper` variant pre-trained specifically for Vietnamese by VinAI Research: as the foundation, fine-tuned using a 2-stage strategy on an RTX 3090 Ti (24 GB VRAM).

#### 4.8.2 Model Architecture

`PhoWhisper-large` inherits the Transformer Encoder-Decoder architecture from `Whisper`. The 16 kHz audio signal is converted to an 80-bin log-Mel spectrogram (25 ms window, 10 ms step), processed through 2 Conv1D + GELU layers before entering the Encoder. Both Encoder and Decoder use Multi-Head Attention with Pre-LayerNorm residual blocks, producing rich contextual representations prior to token generation.

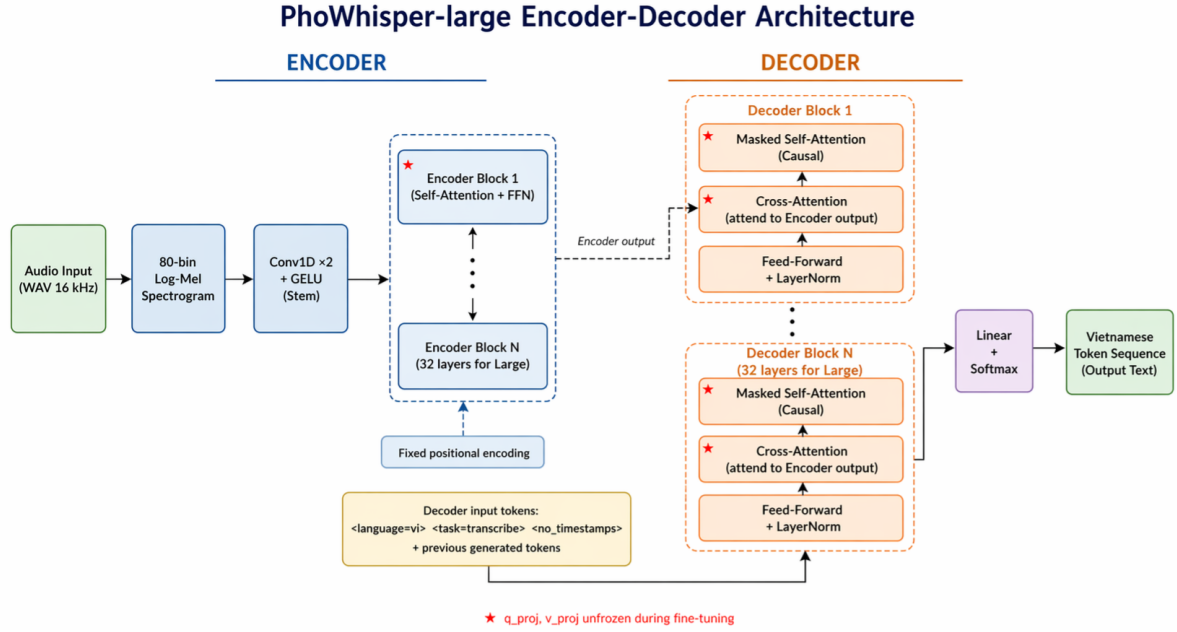


Figure 47: PhoWhisper-large Encoder-Decoder Architecture, adapted from [3].

Table 30: PhoWhisper-large Architecture Components.

| Component                 | Description                                                         |
|---------------------------|---------------------------------------------------------------------|
| Encoder Input             | 80-bin log-Mel spectrogram, 25 ms window, 10 ms step, 16 kHz mono   |
| Conv Stem                 | 2× Conv1D + GELU: reduces time dimension 3000 → 1500 frames         |
| Positional Encoding       | Fixed sinusoidal (non-learnable)                                    |
| Self-Attention            | Multi-Head Attention: Q, K, V projections + Pre-LayerNorm           |
| Cross-Attention (Decoder) | Decoder attends to Encoder output; contains unfrozen q_proj, v_proj |
| Decoder Output            | Vietnamese token sequence, up to 256 tokens                         |
| Sample Rate               | 16 kHz, mono channel                                                |

#### 4.8.3 Fine-tuning Strategy: Partial Unfreezing

Rather than full fine-tuning of the entire model (expensive and prone to overfitting), only two layers are unfrozen: q\_proj and v\_proj across all Self-Attention/Cross-Attention modules: analogous to the LoRA [32] approach: concentrating gradients on projections that directly shape attention patterns. With LoRA rank  $r=16$  and  $=32$  (ratio  $r=2.0$ ), only approximately  $\sim 0.8\%$  of total parameters are updated, enabling fine-tuning on the RTX 3090 Ti without gradient checkpointing.

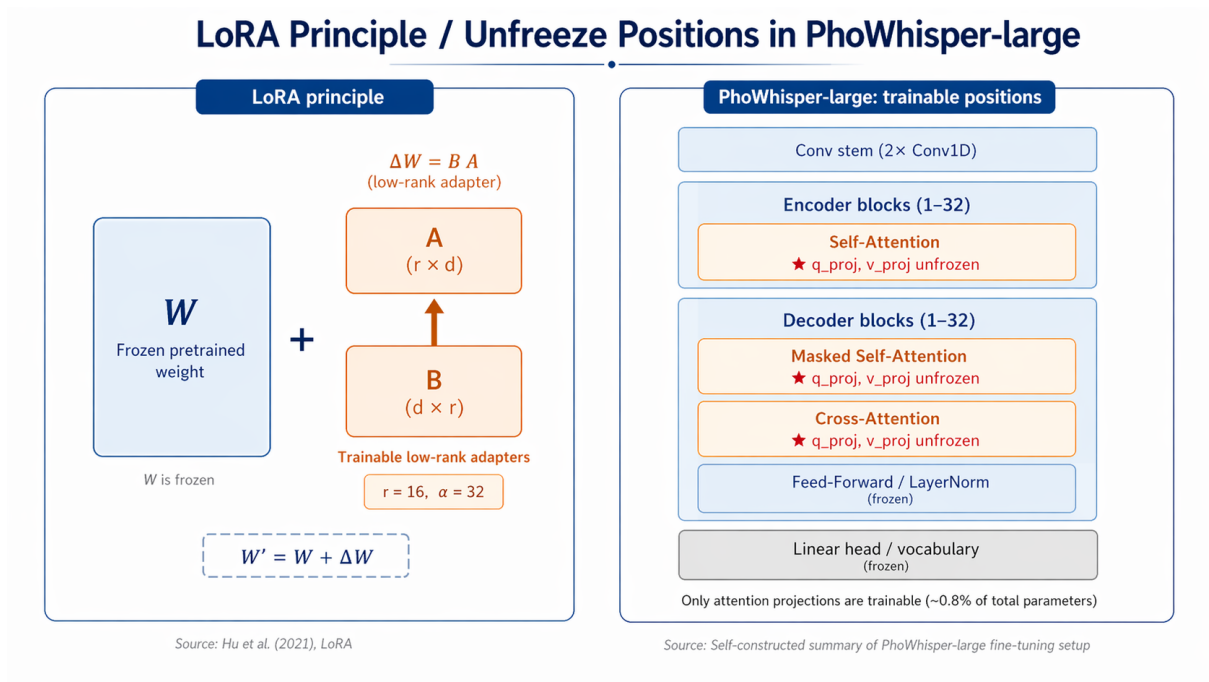


Figure 48: LoRA Principle and Unfreeze Positions in PhoWhisper, adapted from [3, 32].

Table 31: Partial Unfreezing / LoRA Configuration.

| Parameter               | Value          | Notes                                                           |
|-------------------------|----------------|-----------------------------------------------------------------|
| Target Layers           | q_proj, v_proj | Two projections primarily responsible for the attention pattern |
| LoRA Rank ( $r$ )       | 16             | Low-dimensional space to approximate $\Delta W$                 |
| LoRA Alpha ( $\alpha$ ) | 32             | Scaling factor: $\alpha / r = 2.0$                              |
| Dropout                 | 0,05           | Light regularization                                            |
| Trainable ratio         | 0,8%           | Reduces 99% vs. full fine-tuning                                |

#### 4.8.4 Unicode NFC Normalization

All Vietnamese text in the pipeline is normalized to Unicode NFC before tokenization. Reason: Vietnamese characters with diacritics exist in multiple encoding forms (NFC and NFD look identical but have different Unicode code points): if not normalized, the same word can be tokenized differently, artificially increasing WER. The NFC step is fixed in both training and evaluation.

#### 4.8.5 Training Data

Table 32: Data Distribution by Domain and Quality Bucket.

| Domain       | Characteristics                          | Bucket      | Train sets | Training hours |
|--------------|------------------------------------------|-------------|------------|----------------|
| Conan        | Anime dubbing, clear voice, low noise    | silver_real | 25         | 10,43 h        |
| Chuyen Ho    | Natural conversation, fast-paced speech  | silver_real | 15         | 14,76 h        |
| Coi Mo       | Podcast, heavy overlap, slang vocabulary | hard_real   | 5          | 4,33 h         |
| VIF          | Standard broadcast television narration  | gold_real   | 3          | 7,45 h         |
| Dustin On Go | Outdoor vlog, ambient background noise   | silver_real | 1          | 1,21 h         |
| <b>TOTAL</b> |                                          |             | <b>49</b>  | <b>38,2 h</b>  |

Table 33: Self-labeled data distribution.

| Data Source                     | Set          | Files     | Total Hours    | Speaker Range |
|---------------------------------|--------------|-----------|----------------|---------------|
| <b>TRAIN</b>                    |              |           |                |               |
| Chuyen_Ho_Chuyen_Minh           | Train        | 43        | 14.77 h        | 4-13          |
| VIF                             | Train        | 20        | 3.37 h         | 3-6           |
| Coi_mo                          | Train        | 15        | 4.22 h         | 4-5           |
| Dustin_on_go                    | Train        | 8         | 1.74 h         | 4             |
| <b>Total Train self-labeled</b> | <b>Train</b> | <b>86</b> | <b>24.09 h</b> | <b>3-13</b>   |
| <b>VALIDATION</b>               |              |           |                |               |
| VIF                             | Val          | 2         | 2.15 h         | 4-5           |
| Dustin_on_go                    | Val          | 1         | 1.19 h         | 4             |
| Chuyen_Ho_Chuyen_Minh           | Val          | 1         | 0.99 h         | 4             |
| Coi_mo                          | Val          | 2         | 0.94 h         | 4-5           |
| <b>Total Val self-labeled</b>   | <b>Val</b>   | <b>6</b>  | <b>5.27 h</b>  | <b>4-5</b>    |
| <b>TEST</b>                     |              |           |                |               |
| VIF                             | Test         | 2         | 2.52 h         | 4-5           |
| Chuyen_Ho_Chuyen_Minh           | Test         | 1         | 1.21 h         | 4             |
| Dustin_on_go                    | Test         | 1         | 1.19 h         | 4             |
| Coi_mo                          | Test         | 2         | 1.07 h         | 4-5           |
| <b>Total Test self-labeled</b>  | <b>Test</b>  | <b>6</b>  | <b>6.00 h</b>  | <b>4-5</b>    |
| <b>TOTAL SELF-LABELED</b>       |              | <b>98</b> | <b>35.36 h</b> | <b>3-13</b>   |

Table 34: Synthetic Data distribution.

|                                   |       |            |                |                     |
|-----------------------------------|-------|------------|----------------|---------------------|
| <b>Synthetic Data: TRAIN</b>      |       |            |                |                     |
| 0% overlap                        | Train | 43         | 26.8 h         | 2-4 speakers        |
| 5% overlap                        | Train | 39         | 24.4 h         | 2-4 speakers        |
| 10% overlap                       | Train | 39         | 24.3 h         | 2-4 speakers        |
| <b>Total Train Synthetic Data</b> |       | <b>121</b> | <b>75.54 h</b> | <b>2-4 speakers</b> |
| <b>Synthetic Data: VALIDATION</b> |       |            |                |                     |
| <b>Total Val Synthetic Data</b>   |       | <b>10</b>  | <b>4.66 h</b>  | <b>2-4 speakers</b> |
| <b>Synthetic Data: TEST</b>       |       |            |                |                     |
| <b>Total Synthetic Data</b>       |       | <b>10</b>  | <b>4.70 h</b>  | <b>2-4 speakers</b> |
| <b>TOTAL Synthetic Data</b>       |       | <b>141</b> | <b>84.90 h</b> | <b>2-4 speakers</b> |

#### 4.8.6 Fine-tuning PhoWhisper-large

Table 35: Detailed fine-tuning configuration for PhoWhisper-large.

| Parameter                                       | Value                                                                                              |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Model &amp; Architecture</b>                 |                                                                                                    |
| Base model                                      | vinai/PhoWhisper-large                                                                             |
| Architecture                                    | Whisper Encoder-Decoder                                                                            |
| Task                                            | Transcribe (ASR)                                                                                   |
| Language                                        | Vietnamese (vi)                                                                                    |
| Sampling rate                                   | 16,000 Hz                                                                                          |
| <b>Fine-tuning Strategy: Partial Unfreezing</b> |                                                                                                    |
| Freeze strategy                                 | Freeze all → unfreeze only q_proj & v_proj                                                         |
| Trainable parameters                            | q_proj + v_proj across all transformer blocks                                                      |
| <b>Stage 1: Clean Speech Adaptation</b>         |                                                                                                    |
| Stage 1: Epochs                                 | 2                                                                                                  |
| Stage 1: LR                                     | 1e-4                                                                                               |
| Stage 1: Synthetic Data                         | ~91% Self-labeled + dubbed (100%) / ~9% Synthetic data (10% of base hours)                         |
| Stage 1: Overlap filter                         | ≤ 18% overlap ratio per segment                                                                    |
| Stage 1: Augmentation                           | False                                                                                              |
| <b>Stage 2: Robustness Fine-tuning</b>          |                                                                                                    |
| Stage 2: Epochs                                 | 1                                                                                                  |
| Stage 2: LR                                     | 2e-5 (= 1e-4 × scale 0.2)                                                                          |
| Stage 2: Synthetic Data                         | ~95% robust data (Self-labeled + dubbed + robustness 100%) / ~5% Synthetic data (5% of base hours) |
| Stage 2: Overlap filter                         | ≤ 30% overlap ratio per segment                                                                    |
| Stage 2: Augmentation                           | False                                                                                              |
| <b>Loss &amp; Metrics</b>                       |                                                                                                    |
| Loss function                                   | Cross-entropy on token logits                                                                      |
| Metric for best checkpoint                      | Word Error Rate: WER                                                                               |
| WER computation                                 | Custom Levenshtein edit-distance (S+D+I)                                                           |

| Parameter                              | Value                                         |
|----------------------------------------|-----------------------------------------------|
| <b>Optimization</b>                    |                                               |
| Batch size                             | 2 (per device)                                |
| Gradient accumulation                  | 8 steps → effective batch = 16                |
| Optimizer                              | AdamW                                         |
| Warmup ratio                           | 0.05                                          |
| Gradient clipping                      | 1.0                                           |
| Precision                              | FP16 or BF16 (mixed precision, auto-detected) |
| <b>Data Processing</b>                 |                                               |
| Min segment duration                   | 2.5s                                          |
| Max segment duration (preferred)       | 18.0s                                         |
| Max segment duration (hard cap)        | 30.0s                                         |
| Merge gap                              | 0.5s                                          |
| Pad left / right                       | 0.20s / 0.30s                                 |
| Deduplication                          | 2-pass (local per audio + global fingerprint) |
| Synthetic Data reuse cap               | max 2 reuses per base utterance               |
| <b>Validation</b>                      |                                               |
| Eval strategy                          | Per epoch                                     |
| Save strategy                          | Per epoch                                     |
| Save total limit                       | 2 checkpoints                                 |
| Load best model                        | True                                          |
| Max generation length                  | 256 tokens                                    |
| <b>Data Buckets (CLASS_TO_BUCKET)</b>  |                                               |
| vif                                    | gold_real                                     |
| conan, dustin, dustin_on_go, chuyen_ho | silver_real                                   |
| coi_mo, coi_moi                        | hard_real                                     |
| Dubbed sources                         | silver_dubbed                                 |
| Synthetic Data overlap-augmented       | vivos                                         |
| Robustness sources                     | robustness                                    |

| Parameter               | Value                                         |
|-------------------------|-----------------------------------------------|
| <b>Inference</b>        |                                               |
| Decode language         | vi                                            |
| Generation max length   | 256 tokens                                    |
| Forced decoder IDs      | prefix tokens: language + task                |
| Cache                   | Disabled                                      |
| <b>Training History</b> |                                               |
| Logged to               | trainer_state.json (per stage checkpoint dir) |

#### 4.8.7 Loss Function: Cross-Entropy

The model is optimized with Cross-Entropy Loss standard loss for auto-regressive sequence prediction, minimizing the negative log-likelihood of the correct token at each decoding step conditioned on prior context and the audio input:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(y_t | y_{<t}, \mathbf{x})$$

$T$  = sequence length    |     $y_t$  = target token at step  $t$     |     $\mathbf{x}$  = audio input    |     $\theta$  = model parameters

Figure 49: Cross-Entropy Loss, adapted from [3, 33].

#### 4.8.8 Preprocessing Pipeline: Manifest Builder

Processes raw data in 5 automatic steps:

- (1) Scan & Parse RTTM + transcript JSONL.
- (2) Build speech regions and overlap regions.
- (3) Segment-aware Chunking.
- (4) Two-pass deduplication (within-parent and cross-parent).
- (5) Export manifest JSONL separately for Stage 1 and Stage 2.

Table 36: Segment-aware Chunking Rules.

| Condition            | Action                                                                        |
|----------------------|-------------------------------------------------------------------------------|
| Segment < 2,5 s      | Merge with the next segment if the speaker is the same or the gap is < 0,25 s |
| Segment 2,5 s - 18 s | Keep as-is (preferred range)                                                  |
| Segment 18 s - 30 s  | Keep as-is if no natural split point                                          |
| Segment > 30 s       | Mandatory split, distribute words evenly                                      |
| Padding              | Expand boundary +0.20 s (left) and +0.30 s (right) to capture full syllables  |

**Deduplication 2-pass:** Pass 1 (within-parent) removes samples sharing the same text with timestamps differing  $\leq 0.30$  s or IoU  $\geq 85\%$ ; Pass 2 (cross-parent) removes via stable hash over `(audio_path, round(start,2), round(end,2), clean_text)`.

## 4.9 Introduction to Qwen 2.5-1.5B-Instruct

### 4.9.1 Characteristics & Selection Rationale

Qwen/Qwen2.5-1.5B-Instruct: a causal decoder-only Transformer with 1.5 billion parameters. Selection rationale

- (1) Speed: near-instantaneous inference, suitable for real-time pipelines.
- (2) Low VRAM:  $\sim 1,196$  MB with 4-bit NF4, enabling concurrent execution alongside DiariZen and PhoWhisper.
- (3) Vietnamese performance: Qwen 2.5 is recognized as one of the strongest open-source small models for Vietnamese language understanding.

Table 37: Qwen 2.5-1.5B-Instruct Deployment Configuration.

| Parameter                | Value                                    |
|--------------------------|------------------------------------------|
| Model                    | Qwen/Qwen2.5-1.5B-Instruct               |
| Architecture             | Causal Decoder-only Transformer          |
| Quantization             | Not applied (full BF16, no quantization) |
| Compute dtype            | bfloat16                                 |
| Peak VRAM (BF16)         | 3.162 MB                                 |
| Temperature              | 0,1   Top-p: 0,9                         |
| Repetition penalty       | 1,05   No-repeat ngram size: 4           |
| Max input tokens         | 2.304   Normalize max new tokens: 128    |
| Local summary max tokens | 128   Reducer max tokens: 0              |

#### 4.9.2 Qwen Summarization Pipeline

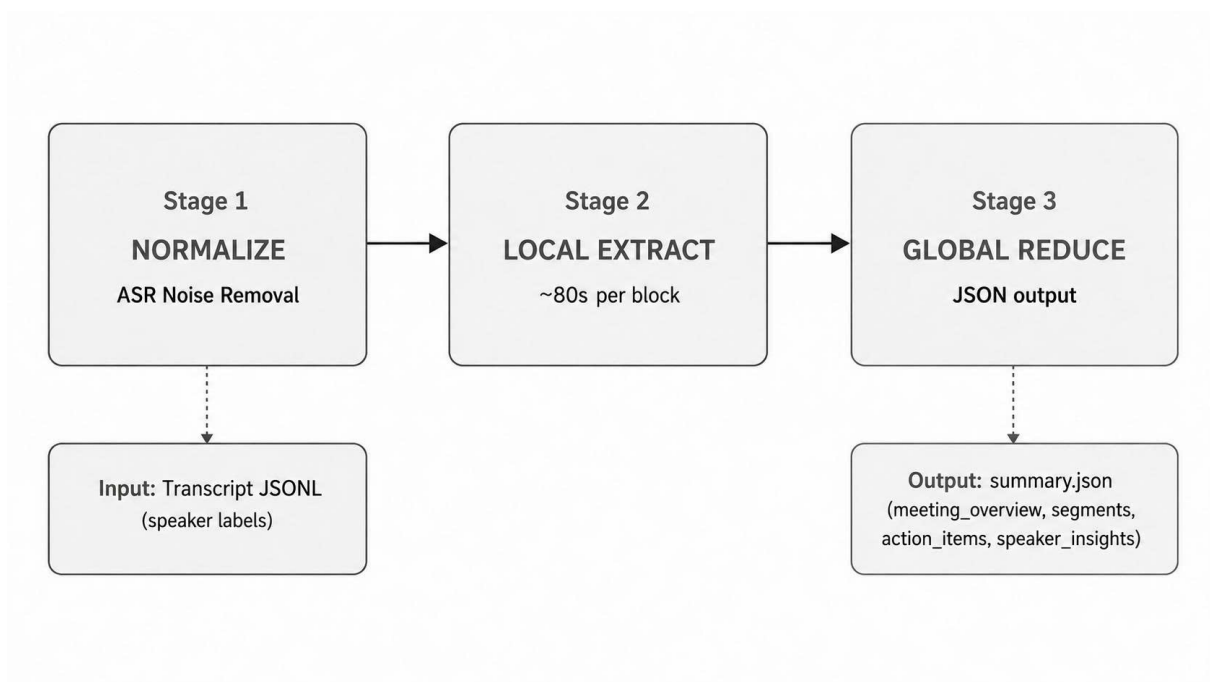


Figure 50: 3-stage pipeline with Smart-Skip mechanism.

Stage 1 addresses a fundamental problem in spoken language processing: ASR output is rarely clean. Disfluencies, spelling inconsistencies, and missing punctuation are pervasive in conversational transcripts, and if left uncorrected they propagate errors into every downstream stage. The normalization step therefore acts as a gating mechanism, filtering segments that fall below a quality threshold before they enter the extraction pipeline.

Stage 2 operates at the local dialogue level, segmenting the normalized transcript into blocks bounded by duration and contextual coherence. Within each block, the model identifies speaker-specific contributions and extracts structured information, technical claims, action items and discussion sum-

maries that preserve local context without yet attempting cross-block coherence.

Stage 3 takes these local extractions as input and produces a single structured JSON report. The global pass is necessary because local summaries are inherently redundant and sometimes contradictory across block boundaries; this stage resolves those inconsistencies, deduplicates action items, and flags potential risks that only become apparent when the full conversation is considered as a whole.

Table 38: Block Segmentation Parameters.

| Parameter                | Value                                                                         |
|--------------------------|-------------------------------------------------------------------------------|
| Max duration per block   | 60.0 seconds                                                                  |
| Max chars per block      | 2.600 characters                                                              |
| Merge gap (same speaker) | $\leq 1.0$ second ( $2 \times \text{merge\_gap}$ if sentence incomplete)      |
| Min text chars (filter)  | $\geq 2$ characters; remove fillers (uh, um, ah, yes, ha...)                  |
| Smart-skip normalize     | Transition block $\leq 35$ s / clean block $\leq 1,800$ chars / noise_score=0 |
| Smart-skip summary       | Transition block / $\leq 3$ segments / $\leq 420$ chars                       |

#### 4.10 Diarization Evaluation Metrics

Unless otherwise noted, all metrics are computed with collar = 0.25 s and overlap-inclusive (overlapping speech regions are not excluded). All computations use pyannote.metrics.

In the per-region formulas below,  $r$  denotes an evaluation region after timeline partitioning,  $\text{dur}(r)$  is the duration of  $r$ ,  $N_{\text{ref}}$  and  $N_{\text{hyp}}$  are the number of active speakers in the reference and hypothesis respectively, and  $N_{\text{cor}}$  is the number of correctly matched speakers after optimal assignment.

##### 4.10.1 Diarization Error Rate (DER)

DER is the primary metric for speaker diarization, measuring the fraction of total reference speaker-time that is incorrectly labeled. It decomposes into three additive components: Missed Speech, False Alarm, and Speaker Confusion.

$$\text{DER} = \frac{T_{\text{miss}} + T_{\text{fa}} + T_{\text{conf}}}{T_{\text{ref}}} \times 100\%$$

$T_{\text{ref}}$  = total reference speech duration   |    $T_{\text{miss}}$  = missed speech   |    $T_{\text{fa}}$  = false alarm   |    $T_{\text{conf}}$  = speaker confusion

Figure 51: General DER Formula, reproduced from [34].

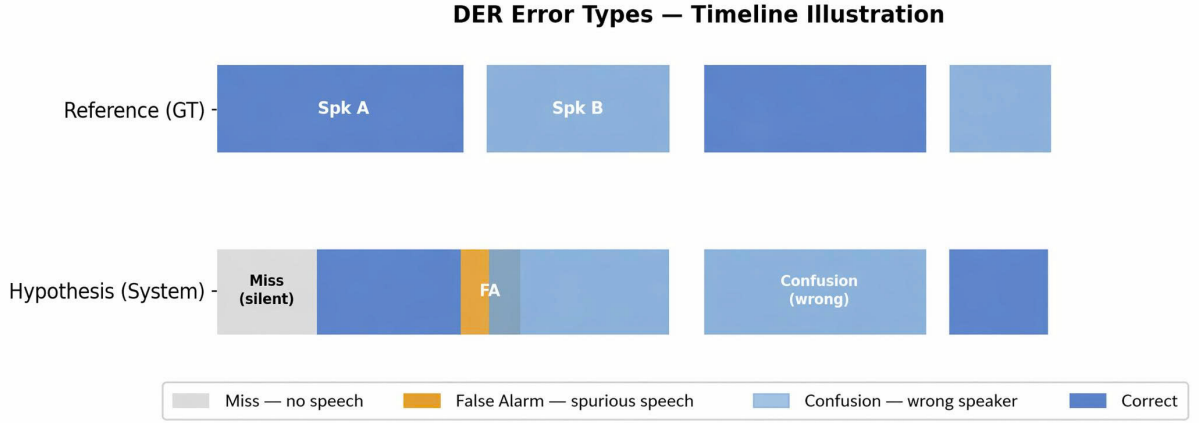


Figure 52: Timeline illustration of the three DER error types, adapted from [35].

#### 4.10.2 Missed Speech (Miss)

Miss measures the total duration of speech present in the reference that the system failed to fully detect. In overlap regions, Miss also increases when the number of active speakers in the hypothesis is smaller than in the reference.

$$\text{Miss\%} = \frac{\sum_r \text{dur}(r) (N_{\text{ref}} - N_{\text{hyp}})^+}{\sum_r \text{dur}(r) N_{\text{ref}}} \times 100\%$$

$N_{\text{ref}}$  = number of reference speakers in region  $r$  |  $N_{\text{hyp}}$  = number of hypothesis speakers

Figure 53: Miss% Formula, reproduced from [35].

Miss is the highest-priority component in this study: missed speech is irreversible. There is nothing for the downstream ASR step to process. No post-processing can recover lost content.

#### 4.10.3 False Alarm (FA)

False Alarm measures the total duration where the system incorrectly outputs speech activity while the reference records silence or no active speaker. It is the inverse error of Miss. The system detects a speaker where no one is speaking.

$$\text{FA\%} = \frac{\sum_r \text{dur}(r) (N_{\text{hyp}} - N_{\text{ref}})^+}{\sum_r \text{dur}(r) N_{\text{ref}}} \times 100\%$$

$N_{\text{ref}}$  = number of reference speakers in region  $r$  |  $N_{\text{hyp}}$  = number of hypothesis speakers

Figure 54: FA% Formula, reproduced from [35].

#### 4.10.4 Speaker Confusion (Conf)

Speaker Confusion measures the total duration where speech is correctly detected but attributed to the wrong speaker. It is computed after applying optimal speaker-to-speaker matching between hypothesis and reference via Hungarian algorithm (linear sum assignment).

$$\text{Conf\%} = \frac{\sum_r \text{dur}(r) (\min(N_{\text{hyp}}, N_{\text{ref}}) - N_{\text{cor}})}{\sum_r \text{dur}(r) N_{\text{ref}}} \times 100\%$$

$N_{\text{cor}}$  = number of correctly matched speakers after optimal Hungarian assignment |  $N_{\text{ref}}$  = number of reference speakers in region  $r$  |  $N_{\text{hyp}}$  = number of hypothesis speakers

Figure 55: Confusion% Formula, reproduced from [35].

In pyannote.metrics, Hungarian matching is performed at file level (global) for file-level DER: this is why file-level DER is typically lower and more stable than crop-level DER.

Impact on ASR pipeline: Confusion is the most dangerous error for conversation analytics: speech is transcribed correctly but attributed to the wrong speaker, corrupting the full dialogue structure. This error cannot be detected or corrected from the transcript alone.

#### 4.10.5 Overlap Detection Metrics: Precision, Recall, F1

These metrics evaluate the ability to detect regions with two or more simultaneously active speakers.  $O_{\text{hyp}}$  and  $O_{\text{ref}}$  denote the predicted and reference overlap regions respectively.

$$\text{Precision} = \frac{|O_{\text{hyp}} \cap O_{\text{ref}}|}{|O_{\text{hyp}}|} \quad \text{Recall} = \frac{|O_{\text{hyp}} \cap O_{\text{ref}}|}{|O_{\text{ref}}|} \quad \text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

$|\cdot|$  = total duration (s) |  $O_{\text{hyp}}$  = predicted overlap |  $O_{\text{ref}}$  = actual overlap

Figure 56: Overlap Precision, Recall, and F1-score Formulas, adapted from [30].

#### 4.10.6 Diagnostic Quantities

##### Predicted Segments and GT Segments.

Predicted Segments (Pred.Seg): Number of discrete speech segments produced by the system in the output RTTM.

GT Segments (GT Seg): Number of segments in the reference RTTM.

The ratio Pred.Seg / GT Seg has no monotonic relationship with DER. Segment count near ground-truth is neither necessary nor sufficient for accurate speaker attribution: speaker assignment quality per segment: not segment count: determines DER.

##### Crop-level DER and File-level DER.

Table 39: Crop-level DER and File-level DER.

|                    | Crop-level DER                                                     | File-level DER                                                   |
|--------------------|--------------------------------------------------------------------|------------------------------------------------------------------|
| Granularity        | 16s / 10s crop (per chunk)                                         | Full audio file                                                  |
| Hungarian matching | Local (per crop)                                                   | Global (per file): more optimal                                  |
| Typical value      | Usually higher                                                     | Usually lower, more realistic                                    |
| Used for           | Monitoring training convergence,<br>tentative checkpoint selection | Final best-checkpoint selection;<br>all results in Tables report |

#### 4.11 Word Error Rate (WER): PhoWhisper

$$\text{WER} = \frac{S + D + I}{N} \times 100\%$$

$S$  = substitutions |  $D$  = deletions |  $I$  = insertions |  $N$  = total reference words

Figure 57: WER Formula, reproduced from [36].

Table 40: Definition of 3 WER Error Types.

| Symbol   | Error Type   | Meaning            | Example                     |
|----------|--------------|--------------------|-----------------------------|
| <b>S</b> | Substitution | Misrecognized word | “xin chào” → “xim chào”     |
| <b>D</b> | Deletion     | Missed word        | “tôi rat vui” → “tôi vui”   |
| <b>I</b> | Insertion    | Spurious insertion | “xin chào” → “xin chào nhé” |

Unicode NFC: normalization, lowercasing, and punctuation removal applied consistently in training & and evaluation: preventing artificial WER inflation due to NFC/NFD discrepancies. WER algorithm: Dynamic Programming (Levenshtein Distance) with back-tracking to accurately count S, D, I.

## 4.12 Qwen Pipeline Evaluation Metrics

Table 41: Automated Evaluation Metrics.

| Metric                       | Good when | Definition                                                                                                                                                     |
|------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Overall Score                | = 1.0     | Aggregate score produced by the evaluation script, combining schema validity, evidence support, speaker-specificity, and confidence quality. Higher is better. |
| Schema Pass Rate             | = 1.0     | Rate of JSON outputs with all required keys present.                                                                                                           |
| Evidence Support Rate        | = 1.0     | Rate of speaker-summary items whose evidence spans are valid and grounded in the transcript timeline.                                                          |
| Speaker Presence Rate        | = 1.0     | Rate of expected speakers that are mentioned in the output.                                                                                                    |
| Generic Speaker Summary Rate | = 0.0     | Rate of generic, non-specific speaker summaries. Lower is better.                                                                                              |
| Low Confidence Rate          | = 0.0     | Rate of speaker summaries marked with low confidence. Lower is better.                                                                                         |

Table 42: LLM Judge Criteria (Qwen 7B).

| Criterion             | Max Score | Definition                                                          |
|-----------------------|-----------|---------------------------------------------------------------------|
| Main Summary Score    | 4         | Accuracy of the main conversation summary.                          |
| Speaker Summary Score | 4         | Accuracy of per-speaker summaries.                                  |
| Faithfulness Score    | 2         | Information is not fabricated; grounded in the original transcript. |
| Conciseness Score     | 2         | Summary is concise, not verbose or repetitive.                      |

## 5 System Design and Implementation

### 5.1 Overview

The user interface of the system is implemented in Streamlit to provide an intuitive and stable way to operate and demonstrate the Vietnamese meeting-processing pipeline. Instead of interacting directly with scripts or terminal commands, the user can upload input data, select the execution mode, monitor the processing status, and review previously generated results through a single interface.

The interface does not replace the backend. It functions as an orchestration and presentation layer above the existing pipeline. Through this design, audio normalization, speaker diarization, ASR, and Qwen-based summarization are integrated into a unified workflow that is suitable for both experimentation and project demonstration.

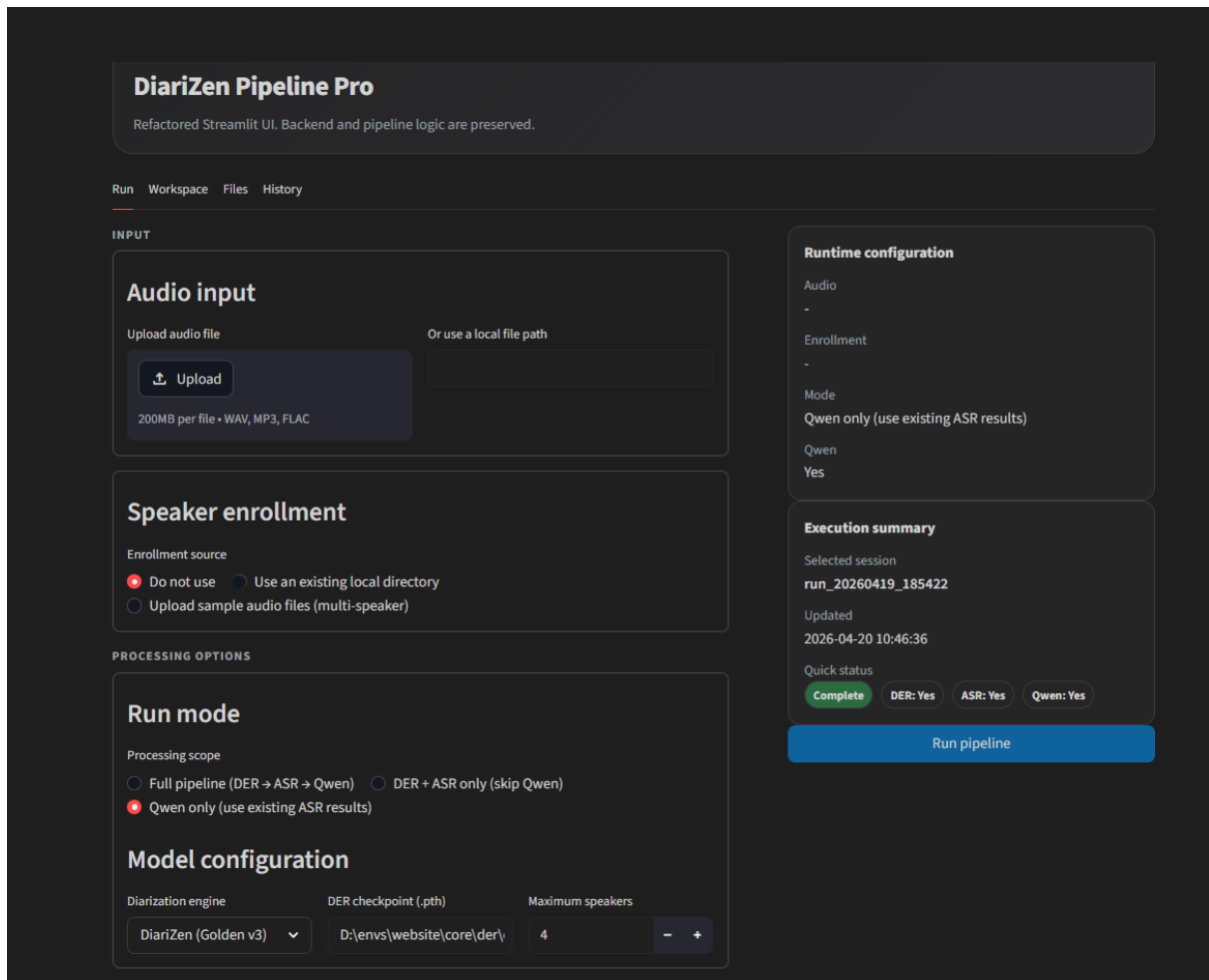


Figure 58: Overall Streamlit interface of the Vietnamese meeting-processing system.

## 5.2 Design Principles and Configurability

The interface was designed according to three main principles. First, it prioritizes operational stability for live demonstration without modifying the logic of the processing pipeline. Second, it preserves a clear separation between frontend and backend responsibilities: the UI is responsible for control and visualization, while DER, ASR, and Qwen inference remain in their dedicated backend modules. Third, the outputs are presented in a readable and verifiable form, especially the transcript, which is displayed as plain text in a structured “mm:ss | Speaker | Text” format rather than as raw HTML.

The interface also supports bilingual display options, dark/light themes, configurable output paths, and basic pipeline parameters. These options make the system more flexible in practical use while maintaining the stability required for an MVP and for project-defense scenarios.

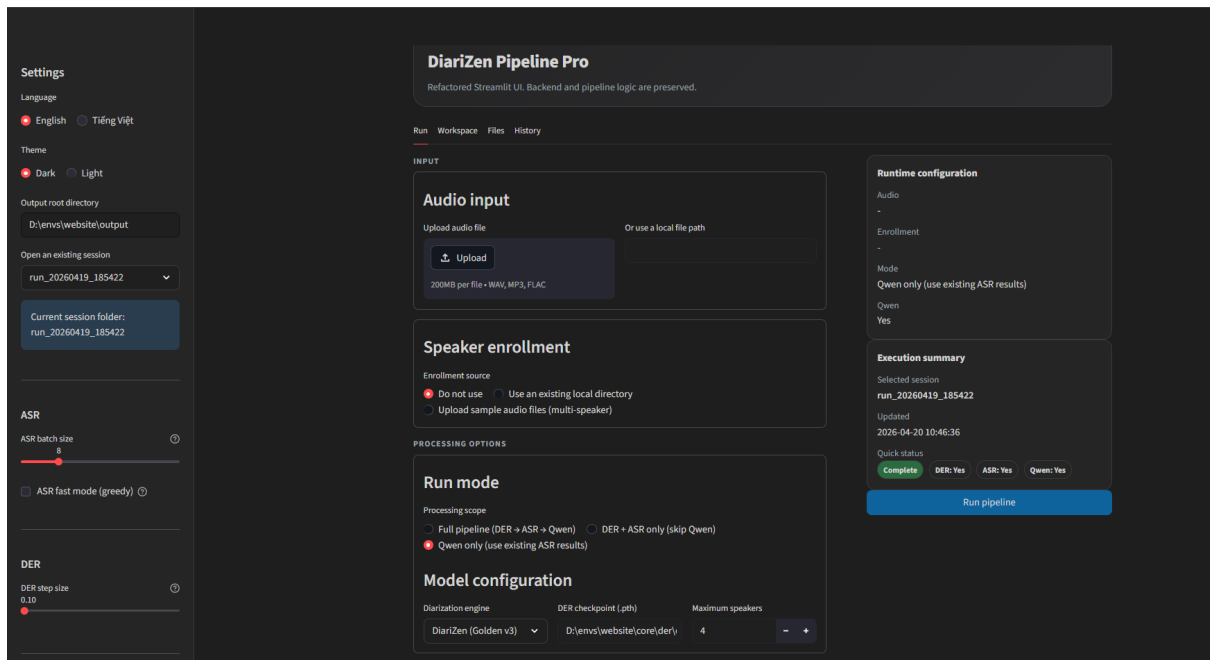


Figure 59: Sidebar configuration panel with language, theme, and pipeline settings.

### 5.3 UI-Backend Integration and Output Organization

At the implementation level, the interface is launched and communicates with the processing modules through intermediate bridge scripts. This organization allows the UI to coordinate execution while avoiding direct intervention in the internal inference logic of each module.

The overall architecture therefore follows a layered design. The Streamlit layer receives user input, exposes configuration options, and forwards execution requests to the backend modules. Each backend block then performs its own specialized task and returns structured outputs to the interface for display.

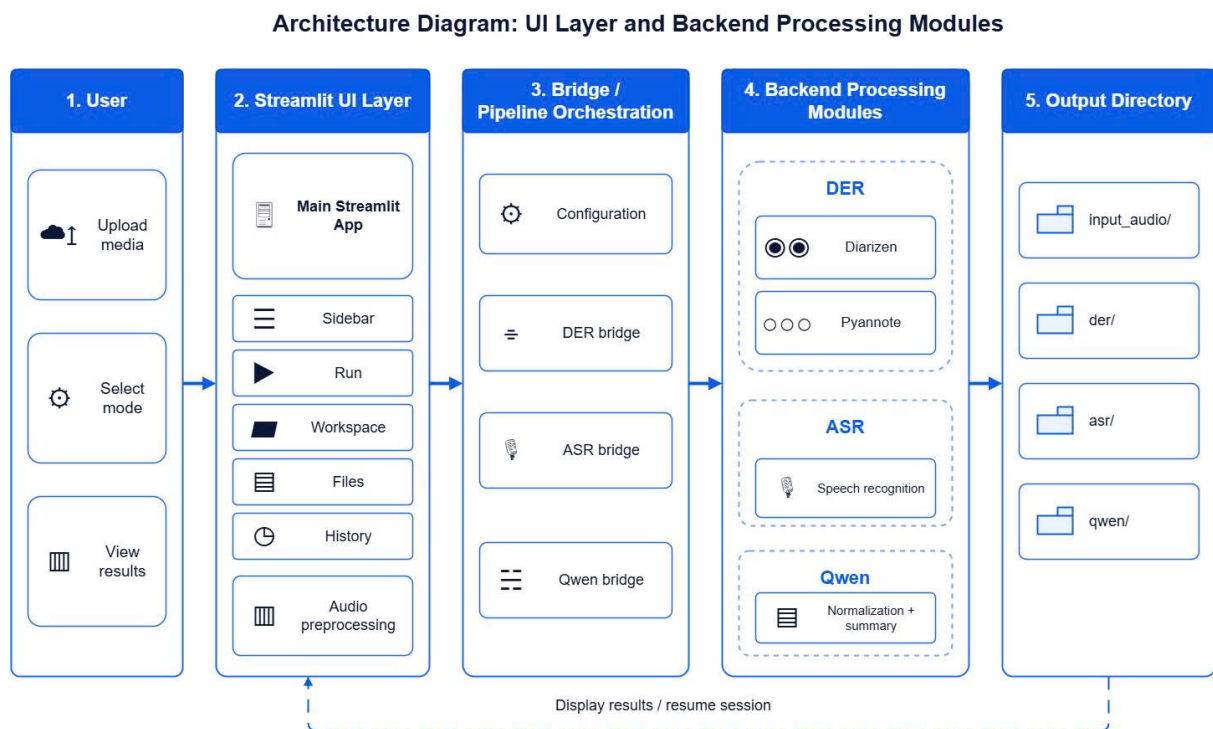


Figure 60: UI-backend architecture of the Streamlit-based meeting-processing system.

The results of each execution are stored in a dedicated session directory following the pattern `output/run_YYYYMMDD_HHMMSS/`. This structure allows the application to detect whether DER, ASR, or Qwen outputs already exist for a given session and makes it possible to reopen and reuse previous runs. It also improves traceability by separating input files, intermediate artifacts, final outputs, and debug logs into distinct subdirectories.

## 5.4 Main Interface Functions

### 5.4.1 Input Area and Execution Modes

The interface allows the user to provide meeting data in the form of audio or video files. Before inference, the input is normalized to mono 16 kHz WAV in order to match the requirements of the downstream DER and ASR stages. The user can also select the most suitable execution mode for the current scenario, such as running the full pipeline, executing DER and ASR only, or running Qwen only on existing ASR outputs.

**INPUT**

### Audio input

Upload audio file

⬆️ Upload

200MB per file • WAV, MP3, FLAC

Or use a local file path

### Speaker enrollment

Enrollment source

☒ Do not use

☐ Use an existing local directory

☐ Upload sample audio files (multi-speaker)

**PROCESSING OPTIONS**

### Run mode

Processing scope

☐ Full pipeline (DER → ASR → Qwen)

☐ DER + ASR only (skip Qwen)

☒ Qwen only (use existing ASR results)

### Model configuration

Diarization engine

DER checkpoint (.pth)

Maximum speakers

DiariZen (Golden v3) ▼

D:\envs\website\core\der\

4 - +

Figure 61: Run tab for selecting the input source and execution mode.

#### 5.4.2 Workspace for Transcript and Summary Inspection

After the pipeline finishes, the application provides a workspace for viewing the transcript and the generated meeting summary. The transcript is presented as plain text with timestamps, speaker labels, and recognized content, which simplifies manual inspection and comparison. The summary panel provides a concise semantic output generated by the Qwen module from the ASR transcript.

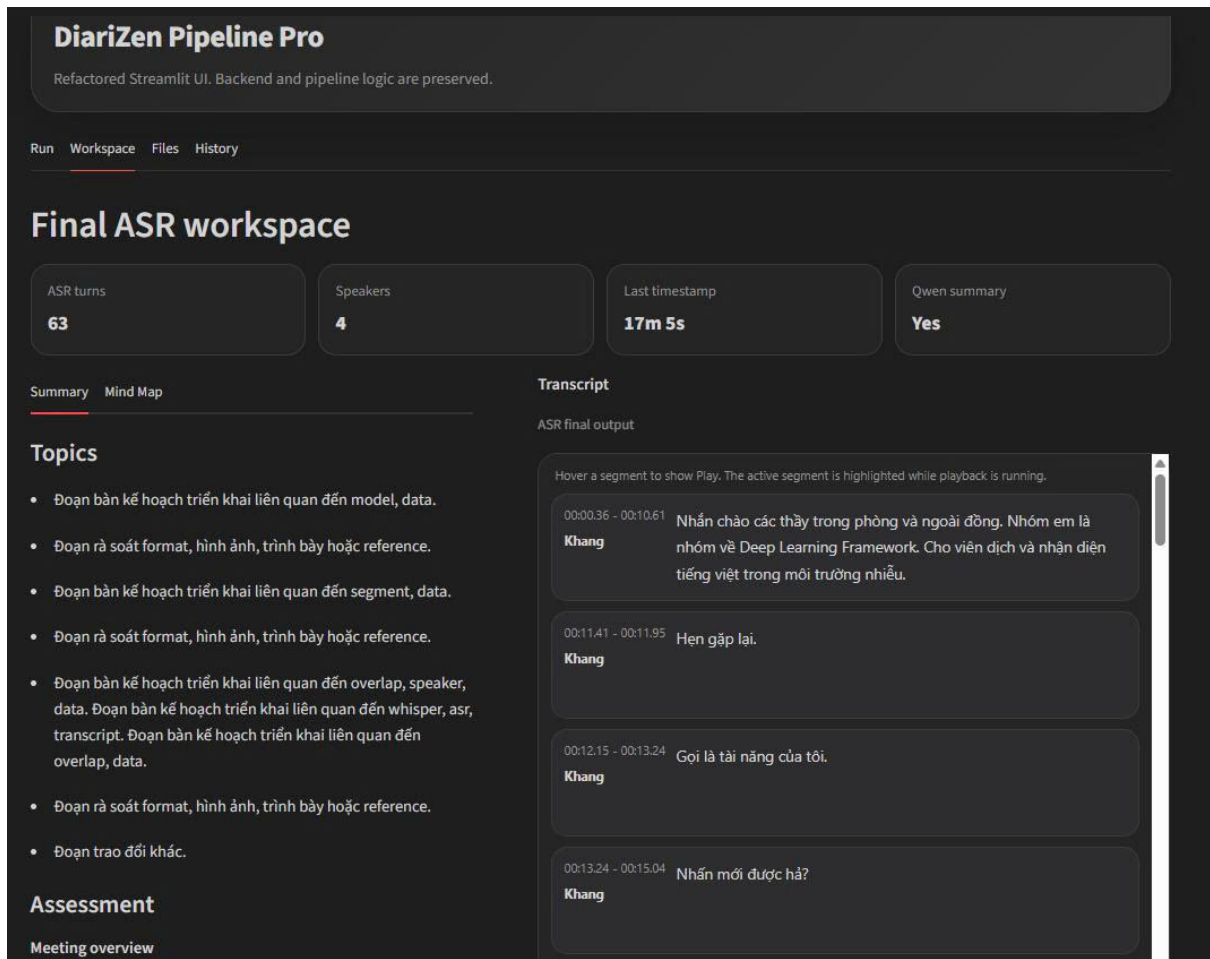


Figure 62: Workspace view for transcript inspection and meeting-summary review.

#### 5.4.3 Run History and Reopening Previous Sessions

Another important function of the interface is session-history management. Previous runs can be filtered, sorted, and reopened in order to inspect transcripts, summaries, or output status without re-running the entire pipeline. This capability is particularly useful for demonstration because prepared sessions can be reused directly when the full processing time is too long for a live presentation.

Run Workspace Files History

Saved sessions

Filter

All

Sort

Newest first

Counts

Complete: 11

Partial: 1

run\_20260419\_185422

2026-04-20 10:46:36

Complete

Yes

Yes

Yes

13

Open session

run\_20260420\_101841

2026-04-20 10:45:47

Complete

Yes

Yes

Yes

13

Open session

run\_20260420\_095428

2026-04-20 10:02:30

Complete

Yes

Yes

Yes

12

Open session

run\_20260419\_184721

2026-04-19 18:48:09

Complete

Yes

Yes

No

9

Open session

Figure 63: Saved-session history for reopening and reviewing previous runs.

## 6 Results and Discussion

### 6.1 DiariZen Results

#### 6.1.1 Training Progress

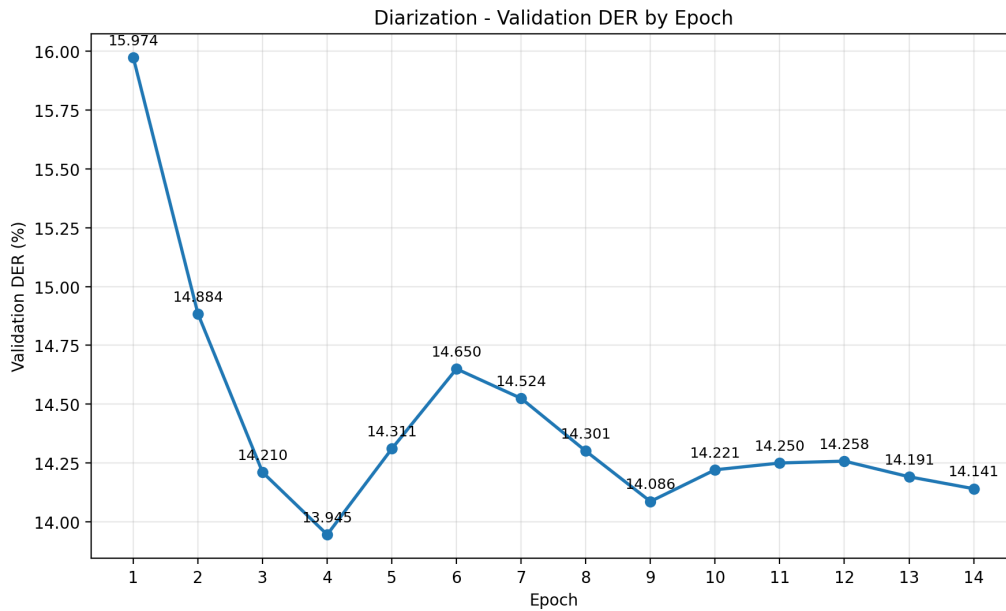


Figure 64: DiariZen Crop-level Validation DER by epoch.

Train loss and validation loss both decrease consistently across the three phases. Phase 1 converges quickly (crop-level DER 15.97%-13.95% at ep004). When augmentation is enabled in Phase 2, crop-level DER increases slightly (14.2-14.7%) as the model learns a harder data distribution: this is an expected effect, not a quality degradation. Phase 3 stabilizes at 14.1%. The best checkpoint according to file-level DER is ep014 (13.8%), different from ep004 which is best for crop-level: consistent with the two-level validation design.

## 6.1.2 Threshold Sweep Results: DiariZen

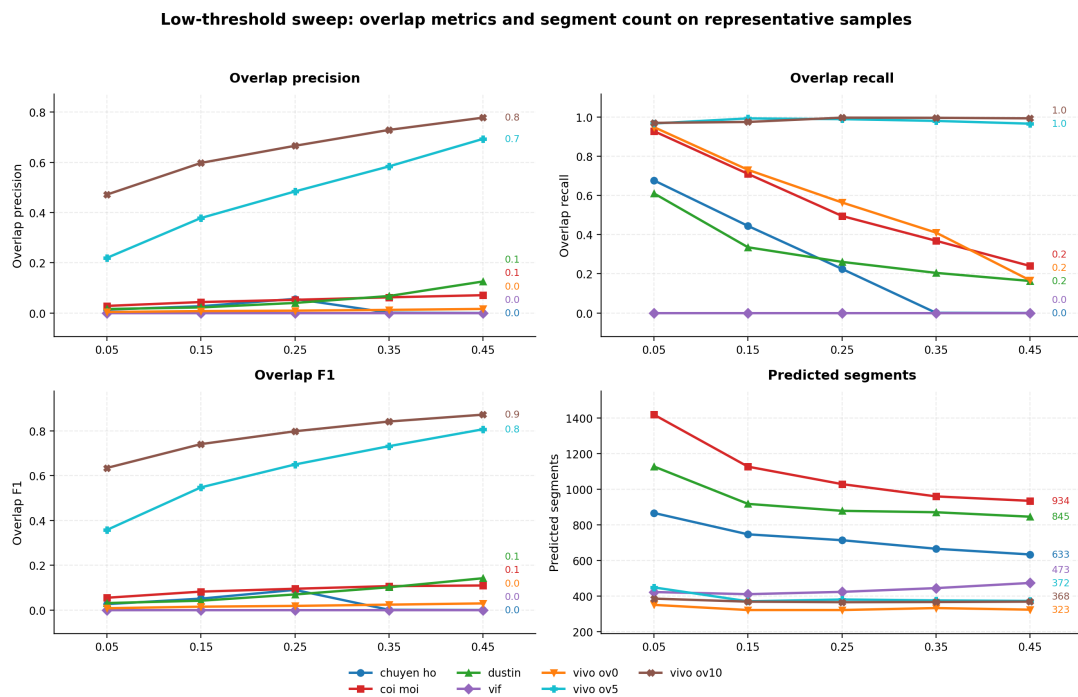


Figure 65: DiariZen fine-tuned: Overlap Precision/Recall/F1 and Predicted Segments by threshold. Precision increases, Recall decreases as threshold increases.

Table 43: DiariZen Fine-tuned DER (%) by threshold on test\_labeled set (★ = selected threshold).

| Dataset          | Metric    | thr=0.05     | thr=0.15 | thr=0.25 ★   | thr=0.35      | thr=0.45      |
|------------------|-----------|--------------|----------|--------------|---------------|---------------|
| <b>chuyen_ho</b> |           |              |          |              |               |               |
|                  | DER (%)   | 54.996       | 38.145   | 32.743       | <b>31.793</b> | 32.142        |
|                  | Miss (%)  | <b>0.202</b> | 1.397    | 3.890        | 7.601         | 11.360        |
|                  | FA (%)    | 52.442       | 34.104   | 26.220       | 22.277        | <b>19.576</b> |
|                  | Conf (%)  | 2.353        | 2.644    | 2.633        | 1.916         | <b>1.206</b>  |
|                  | Ov.Prec.  | 0.013        | 0.027    | <b>0.056</b> | 0.001         | 0.000         |
|                  | Ov.Recall | <b>0.677</b> | 0.444    | 0.225        | 0.001         | 0.000         |
|                  | Ov.F1     | 0.026        | 0.051    | <b>0.090</b> | 0.001         | 0.000         |
|                  | Pred.Seg  | 866          | 746      | 713          | 665           | <b>633</b>    |
|                  | GT Seg    | 632          | 632      | 632          | 632           | 632           |
| <b>coi_moi</b>   |           |              |          |              |               |               |
|                  | DER (%)   | 96.395       | 62.516   | 51.035       | 45.435        | <b>43.496</b> |
|                  | Miss (%)  | <b>0.577</b> | 2.901    | 6.232        | 10.092        | 14.871        |
|                  | FA (%)    | 91.047       | 55.242   | 40.356       | 31.452        | <b>24.623</b> |
|                  | Conf (%)  | 4.771        | 4.373    | 4.447        | <b>3.892</b>  | 4.001         |
|                  | Ov.Prec.  | 0.028        | 0.044    | 0.053        | 0.062         | <b>0.071</b>  |
|                  | Ov.Recall | <b>0.928</b> | 0.710    | 0.495        | 0.369         | 0.240         |
|                  | Ov.F1     | 0.055        | 0.082    | 0.095        | 0.107         | <b>0.110</b>  |
|                  | Pred.Seg  | 1418         | 1126     | 1028         | 959           | <b>934</b>    |
|                  | GT Seg    | 669          | 669      | 669          | 669           | 669           |
| <b>dustin</b>    |           |              |          |              |               |               |
|                  | DER (%)   | 48.716       | 30.264   | 24.509       | <b>22.686</b> | 22.725        |
|                  | Miss (%)  | <b>0.541</b> | 1.632    | 3.398        | 6.144         | 9.345         |
|                  | FA (%)    | 45.112       | 25.436   | 17.707       | 13.505        | <b>10.776</b> |
|                  | Conf (%)  | 3.062        | 3.197    | 3.403        | 3.037         | <b>2.604</b>  |
|                  | Ov.Prec.  | 0.016        | 0.022    | 0.040        | 0.068         | <b>0.126</b>  |
|                  | Ov.Recall | <b>0.611</b> | 0.335    | 0.261        | 0.205         | 0.163         |
|                  | Ov.F1     | 0.031        | 0.042    | 0.070        | 0.102         | <b>0.142</b>  |
|                  | Pred.Seg  | 1127         | 917      | 878          | 870           | <b>845</b>    |
|                  | GT Seg    | 785          | 785      | 785          | 785           | 785           |
| <b>vif</b>       |           |              |          |              |               |               |
|                  | DER (%)   | 8.478        | 6.986    | 6.617        | <b>6.287</b>  | 6.341         |
|                  | Miss (%)  | <b>0.052</b> | 0.224    | 0.468        | 0.743         | 1.136         |
|                  | FA (%)    | 7.968        | 6.402    | 5.795        | 5.308         | <b>4.954</b>  |
|                  | Conf (%)  | 0.458        | 0.360    | 0.355        | <b>0.236</b>  | 0.250         |
|                  | Ov.Prec.  | 0.000        | 0.000    | 0.000        | 0.000         | 0.000         |
|                  | Ov.Recall | 0.000        | 0.000    | 0.000        | 0.000         | 0.000         |
|                  | Ov.F1     | 0.000        | 0.000    | 0.000        | 0.000         | 0.000         |
|                  | Pred.Seg  | 422          | 410      | 423          | 444           | <b>473</b>    |
|                  | GT Seg    | 625          | 625      | 625          | 625           | 625           |

**Remarks.**

Bold values indicate the best result in each metric row across thresholds.

Lower values are better for DER, Miss, FA, and Conf, while higher values are better for Ov.F1.

For Pred.Seg, the highlighted value indicates the predicted segment count closer to GT Seg.

VIF has no overlapping speech; therefore Ov.Prec., Ov.Recall, and Ov.F1 are 0.000.

Table 44: DiariZen Fine-tuned DER / Miss / FA / Conf by threshold on Synthetic Data (★ = selected threshold).

| Dataset                                                   | Metric    | thr=0.05     | thr=0.15     | thr=0.25 ★   | thr=0.35     | thr=0.45     |
|-----------------------------------------------------------|-----------|--------------|--------------|--------------|--------------|--------------|
| <b>Synthetic Data ov0 (GT Seg = 257, no real overlap)</b> |           |              |              |              |              |              |
|                                                           | DER (%)   | 23.902       | 11.818       | 7.563        | 5.351        | <b>3.980</b> |
|                                                           | Miss (%)  | <b>0.011</b> | 0.057        | 0.248        | 0.689        | 1.364        |
|                                                           | FA (%)    | 22.220       | 10.708       | 6.828        | 4.260        | <b>2.170</b> |
|                                                           | Conf (%)  | 1.671        | 1.052        | 0.487        | <b>0.402</b> | 0.446        |
|                                                           | Ov.Prec.  | 0.004        | 0.008        | 0.009        | 0.012        | <b>0.016</b> |
|                                                           | Ov.Recall | <b>0.949</b> | 0.731        | 0.564        | 0.410        | 0.167        |
|                                                           | Ov.F1     | 0.008        | 0.015        | 0.018        | 0.024        | <b>0.030</b> |
|                                                           | Pred.Seg  | 350          | <b>321</b>   | <b>321</b>   | 332          | 323          |
|                                                           | GT Seg    | 257          | 257          | 257          | 257          | 257          |
| <b>Synthetic Data ov5 (GT Seg = 257, overlap 5%)</b>      |           |              |              |              |              |              |
|                                                           | DER (%)   | 28.026       | 15.592       | 11.779       | 9.350        | <b>8.713</b> |
|                                                           | Miss (%)  | 0.188        | <b>0.123</b> | 0.407        | 1.282        | 2.476        |
|                                                           | FA (%)    | 26.344       | 13.610       | 8.980        | 6.203        | <b>4.057</b> |
|                                                           | Conf (%)  | <b>1.493</b> | 1.858        | 2.392        | 1.864        | 2.180        |
|                                                           | Ov.Prec.  | 0.220        | 0.378        | 0.484        | 0.584        | <b>0.694</b> |
|                                                           | Ov.Recall | 0.967        | <b>0.993</b> | 0.989        | 0.980        | 0.966        |
|                                                           | Ov.F1     | 0.358        | 0.548        | 0.650        | 0.732        | <b>0.807</b> |
|                                                           | Pred.Seg  | 448          | <b>370</b>   | 380          | 375          | 372          |
|                                                           | GT Seg    | 257          | 257          | 257          | 257          | 257          |
| <b>Synthetic Data ov10 (GT Seg = 285, overlap 10%)</b>    |           |              |              |              |              |              |
|                                                           | DER (%)   | 18.943       | 11.573       | 8.537        | 6.554        | <b>5.471</b> |
|                                                           | Miss (%)  | 0.305        | 0.276        | <b>0.128</b> | 0.266        | 0.561        |
|                                                           | FA (%)    | 18.096       | 11.166       | 8.355        | 6.221        | <b>4.765</b> |
|                                                           | Conf (%)  | 0.542        | 0.131        | <b>0.054</b> | 0.067        | 0.144        |
|                                                           | Ov.Prec.  | 0.472        | 0.598        | 0.666        | 0.729        | <b>0.778</b> |
|                                                           | Ov.Recall | 0.970        | 0.975        | <b>0.997</b> | 0.996        | 0.994        |
|                                                           | Ov.F1     | 0.635        | 0.741        | 0.799        | 0.842        | <b>0.873</b> |
|                                                           | Pred.Seg  | 385          | 368          | <b>365</b>   | 366          | 368          |
|                                                           | GT Seg    | 285          | 285          | 285          | 285          | 285          |

#### Remarks.

Bold values indicate the best result in each metric row across thresholds.

#### 6.1.3 Overall Results on Synthetic data

Aggregates DER and its components by checkpoint on the Synthetic data set (all noise and overlap conditions), along with Overlap F1 and predicted segment count compared to ground-truth.

Table 45: DiariZen Overall Results on Synthetic Data Set by Checkpoint (thr=0.25).

| Checkpoint    | DER(%)       | Miss(%)      | FA(%)        | Conf(%)      | Ov.F1        | Pred.Seg   | GT Seg |
|---------------|--------------|--------------|--------------|--------------|--------------|------------|--------|
| Baseline      | 30.105       | 15.386       | 7.927        | 6.792        | 0.389        | 487        | 276    |
| <b>Phase1</b> | <b>9.641</b> | <b>0.401</b> | <b>7.192</b> | <b>2.048</b> | <b>0.511</b> | 400        | 276    |
| Phase2        | 13.286       | 0.650        | 10.202       | 2.434        | 0.483        | <b>386</b> | 276    |
| Phase3        | 13.586       | 1.027        | 10.224       | 2.335        | 0.478        | 388        | 276    |

#### Remarks.

Bold values indicate the best result among all checkpoints each condition and metric.

test\_data (vivo) — overall overlap metrics and segment count across checkpoints

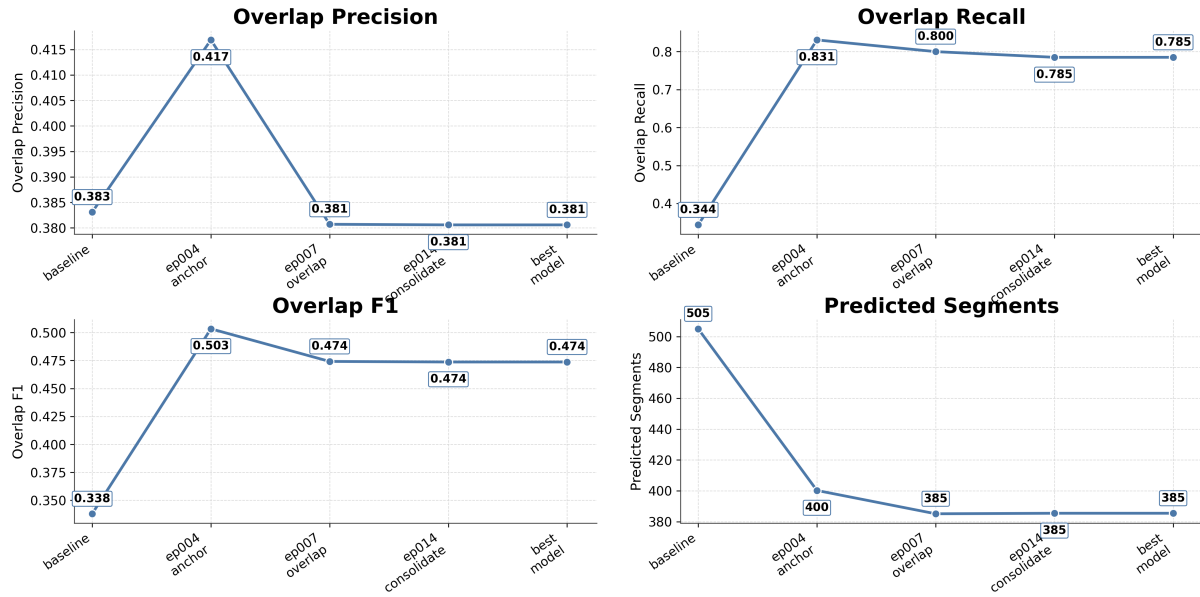


Figure 66: DiariZen: Overlap Precision/Recall/F1 and Predicted Segments overall on Synthetic Data by checkpoint.

#### 6.1.4 Noise Robustness Evaluation: Synthetic Data

Table 46 presents DER and its components (Miss, FA, Conf), Overlap F1, and Predicted Segments for each noise and overlap condition: comparing DiariZen baseline and best model at thr=0.25.

Table 46: DiariZen DER / Miss / FA / Conf by Condition and Model (thr=0.25).

| Condition                    | Model      | DER(%)        | Miss(%)      | FA(%)        | Conf(%)      | Ov.F1        | Pred.Seg   | GT Seg |
|------------------------------|------------|---------------|--------------|--------------|--------------|--------------|------------|--------|
| <b>0% NOISE</b>              |            |               |              |              |              |              |            |        |
| ov0 / 0% noise               | Baseline   | 30.539        | 15.943       | 8.376        | 6.220        | 0.000        | 450        | 257    |
| ov0 / 0% noise               | Best model | <b>7.391</b>  | <b>0.248</b> | <b>6.653</b> | <b>0.490</b> | <b>0.018</b> | <b>326</b> | 257    |
| ov5 / 0% noise               | Baseline   | 27.654        | 15.908       | <b>7.970</b> | 3.777        | 0.594        | 489        | 257    |
| ov5 / 0% noise               | Best model | <b>12.378</b> | <b>0.431</b> | 9.471        | <b>2.475</b> | <b>0.629</b> | <b>388</b> | 257    |
| ov10 / 0% noise              | Baseline   | 26.148        | 14.684       | 8.451        | 3.013        | 0.581        | 533        | 285    |
| ov10 / 0% noise              | Best model | <b>8.482</b>  | <b>0.132</b> | <b>8.295</b> | <b>0.054</b> | <b>0.798</b> | <b>365</b> | 285    |
| <b>WITH BACKGROUND NOISE</b> |            |               |              |              |              |              |            |        |
| ov0 / 23% noise              | Baseline   | 25.882        | 12.235       | <b>7.171</b> | 6.477        | 0.000        | 370        | 237    |
| ov0 / 23% noise              | Best model | <b>15.368</b> | <b>1.776</b> | 11.661       | <b>1.931</b> | <b>0.011</b> | <b>328</b> | 237    |
| ov5 / 23% noise              | Baseline   | 33.674        | 17.683       | <b>7.738</b> | 8.253        | 0.539        | 540        | 294    |
| ov5 / 23% noise              | Best model | <b>14.896</b> | <b>1.360</b> | 11.463       | <b>2.073</b> | <b>0.677</b> | <b>405</b> | 294    |
| ov10 / 17% noise             | Baseline   | 36.731        | 15.865       | <b>7.855</b> | 13.011       | 0.617        | 539        | 324    |
| ov10 / 17% noise             | Best model | <b>23.001</b> | <b>2.215</b> | 13.801       | <b>6.986</b> | <b>0.734</b> | <b>518</b> | 324    |

**Remarks.**

Bold values indicate the better result between the Baseline and Best model for each condition and metric.

**6.1.5 Results by Speaker Count Synthetic Data**

Table 47 breaks down results by number of speakers (2, 3, 4 speakers) on the Synthetic Data, comparing baseline and best model. GT Seg is the actual reference segment count.

Table 47: DiariZen DER / Miss / FA / Conf by Speaker Count on Synthetic Data (thr=0.25).

| Speakers | Checkpoint | DER(%)        | Miss(%)      | FA(%)        | Conf(%)      | Ov.F1        | Pred.Seg   | GT Seg |
|----------|------------|---------------|--------------|--------------|--------------|--------------|------------|--------|
| 2 spk    | Baseline   | 28.114        | 15.512       | 8.266        | 4.337        | 0.392        | 491        | 266    |
| 2 spk    | Best model | <b>9.417</b>  | <b>0.271</b> | <b>8.140</b> | <b>1.006</b> | <b>0.482</b> | <b>360</b> | 266    |
| 3 spk    | Baseline   | 31.307        | 14.050       | <b>7.513</b> | 9.744        | 0.309        | 455        | 281    |
| 3 spk    | Best model | <b>19.185</b> | <b>1.995</b> | 12.731       | <b>4.459</b> | <b>0.373</b> | <b>423</b> | 281    |
| 4 spk    | Baseline   | 33.674        | 17.683       | <b>7.738</b> | 8.253        | 0.539        | 540        | 294    |
| 4 spk    | Best model | <b>14.896</b> | <b>1.360</b> | 11.463       | <b>2.073</b> | <b>0.677</b> | <b>405</b> | 294    |

**Remarks.**

The highlighted values in each column indicate the better result when comparing the Baseline and the Best model within the same speaker-count condition, name 2 speakers, 3 speakers, and 4 speakers.

### 6.1.6 Results on Self-labeled Set (Primary Evaluation Set)

Table 48 presents best model results on each self-labeled class. This is the primary evaluation set reflecting real Vietnamese conversational conditions; baseline DER is included to show relative improvement.

Table 48: DiariZen: DER / Miss / FA / Conf on test\_labeled Set (best model, thr=0.25).

| Dataset (test_labeled) | DER(%) | Miss(%) | FA(%)  | Conf(%) | Ov.F1 | Pred.Seg | GT Seg | Baseline DER |
|------------------------|--------|---------|--------|---------|-------|----------|--------|--------------|
| chuyen_ho              | 32.742 | 3.891   | 26.218 | 2.633   | 0.090 | 713      | 632    | 50.801       |
| coi_moi                | 51.042 | 6.231   | 40.364 | 4.448   | 0.095 | 1028     | 669    | 72.408       |
| dustin_1               | 24.516 | 3.404   | 17.706 | 3.405   | 0.070 | 879      | 785    | 44.078       |
| vif_1                  | 6.617  | 0.468   | 5.795  | 0.355   | 0.000 | 423      | 625    | 18.248       |
| vif_2                  | 7.521  | 0.132   | 7.072  | 0.317   | 0.000 | 318      | 522    | 13.404       |

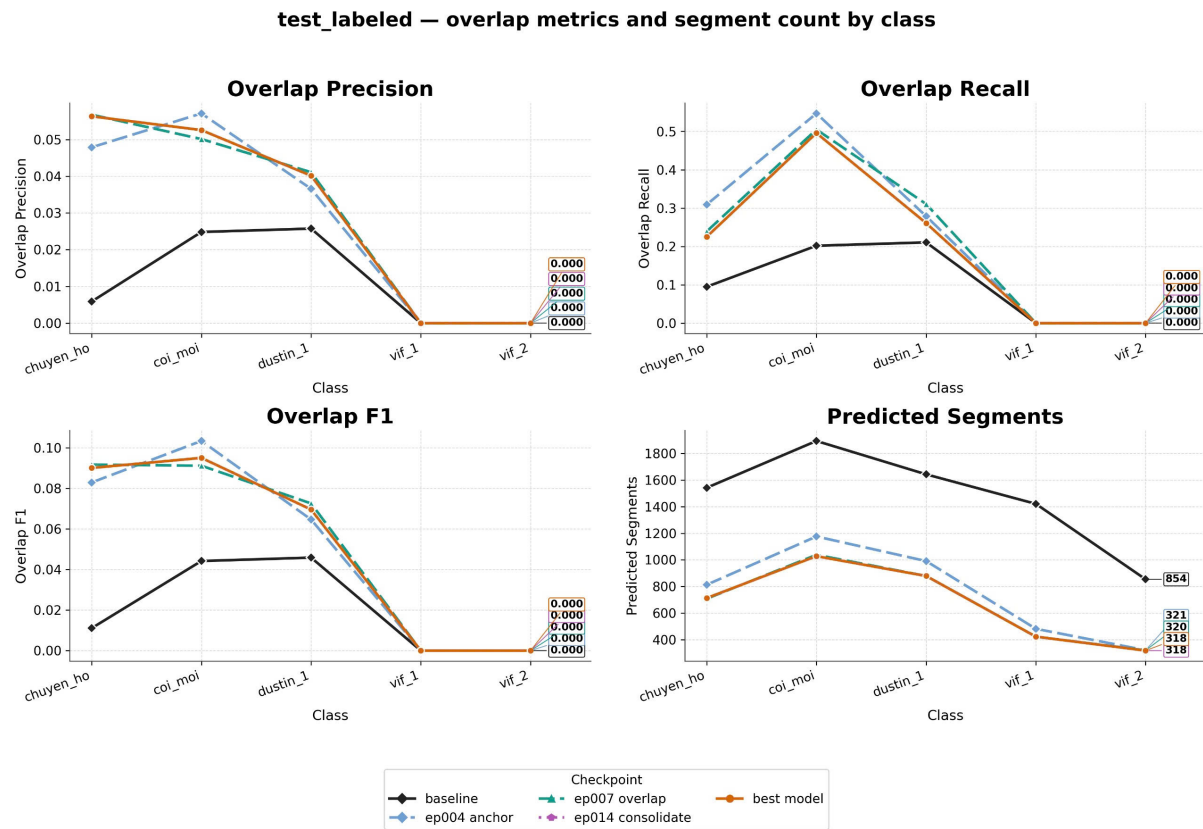


Figure 67: DiariZen Overlap F1 and Predicted Segments by class on test\_labeled .

## 6.2 Pyannote Results

### 6.2.1 Training Progress

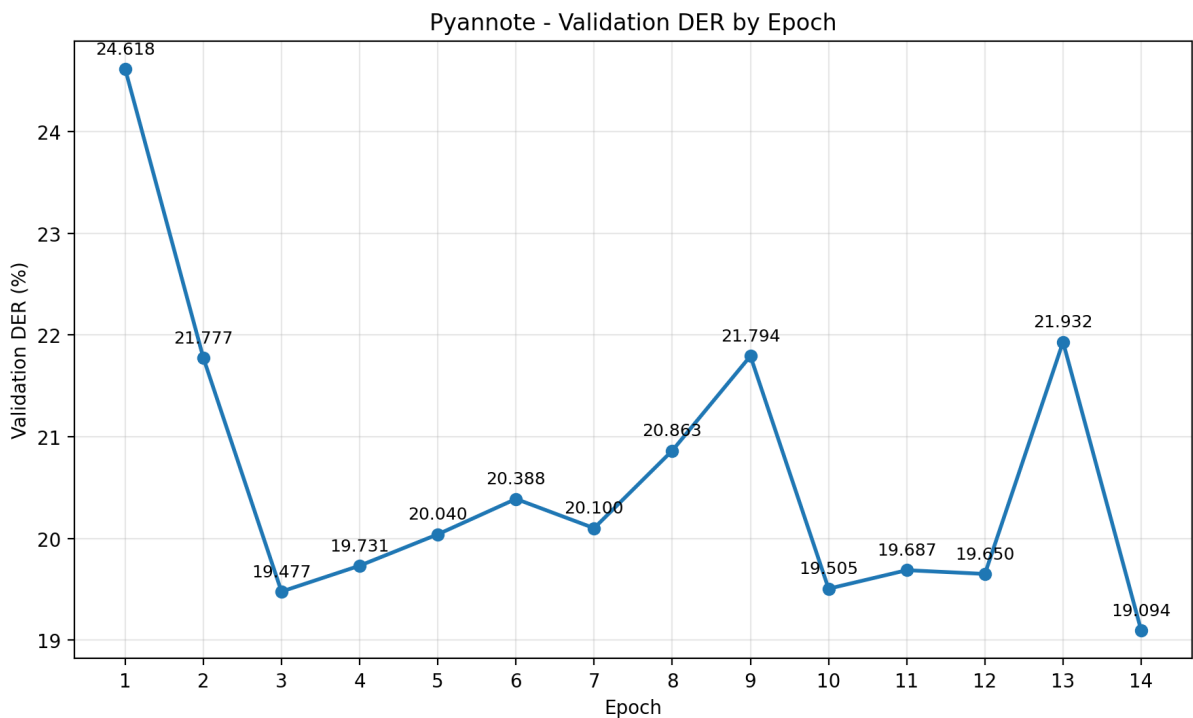


Figure 68: Pyannote Crop-level Validation DER by epoch.

Train loss decreases from 0.87 to 0.71 after 14 epochs; validation DER decreases from 24-25% to 19-21% but still varies between epochs. The model learns signals from Vietnamese data, but convergence stability is lower than DiariZen under the same configuration.

### 6.2.2 Threshold Sweep: Pyannote

Unlike DiariZen, Pyannote's threshold sweep produces nearly flat curves across the entire range: including DER, Miss, FA, and Confusion: on all data classes (chuyen\_ho, coi\_moi, dustin, vif, Synthetic Data ov0/ov5/ov10). This is a direct consequence of the `argmax_powerset` mechanism: segmentation decisions are made by `argmax` over the powerset class distribution; changes to external threshold do not intervene in this decision. Therefore, to optimize Pyannote it is necessary to tune pipeline parameters such as `min_duration_on/off` or clustering threshold, not the sigmoid threshold.

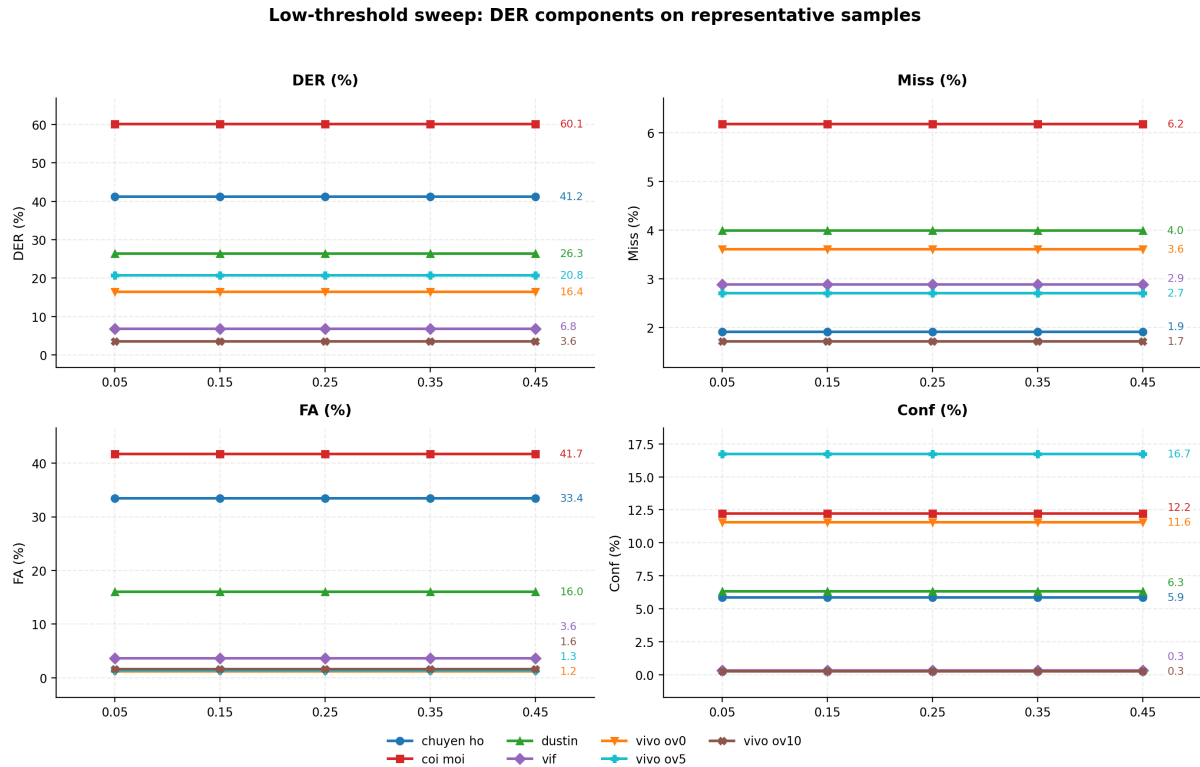


Figure 69: Pyannote fine-tuned Threshold sweep on all representative samples.

### 6.2.3 Overall Results on Synthetic Data

Table 49 aggregates results by checkpoint on the Synthetic Data. The best checkpoint is determined by file-level DER on the validation set: Phase 3 (ep14) is selected even though its overall DER of 17.4% is slightly higher than Phase 2 (17.3%), because file-level DER on individual files gives better results.

Table 49: Pyannote Overall Results on Synthetic Data by Checkpoint.

| Checkpoint     | DER(%)        | Miss(%)      | FA(%)        | Conf(%)       | Ov.F1        | Pred.Seg   | GT Seg |
|----------------|---------------|--------------|--------------|---------------|--------------|------------|--------|
| Baseline       | 62.175        | 20.752       | 19.574       | 21.849        | 0.088        | <b>188</b> | 271    |
| Phase 1        | 17.546        | 5.115        | <b>1.372</b> | <b>11.059</b> | 0.540        | 399        | 271    |
| Phase 2        | <b>17.338</b> | 4.371        | 1.742        | 11.225        | 0.554        | 377        | 271    |
| <b>Phase 3</b> | 17.441        | <b>4.279</b> | 1.761        | 11.401        | <b>0.555</b> | 377        | 271    |

#### Remarks.

Bold values indicate the best result among all checkpoints each condition and metric.

Bold values indicate the best result in each metric row across thresholds.

Lower values are better for DER, Miss, FA, and Conf, while higher values are better for Ov.F1.

For Pred.Seg, the highlighted value indicates the predicted segment count closer to GT Seg.

## VIVOS Test Data — Overall Overlap Metrics and Predicted Segments

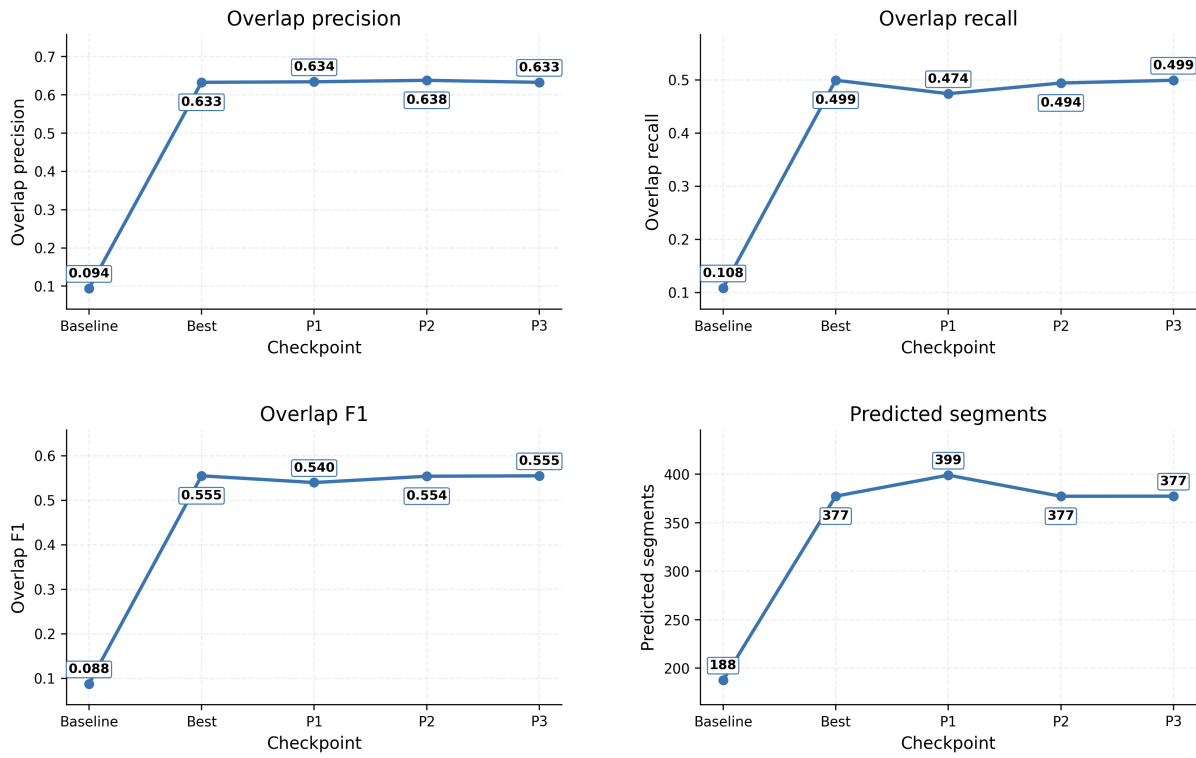


Figure 70: Pyannote Overlap Precision/Recall/F1 and Predicted Segments overall on Synthetic Data . Baseline F1=0.088, fine-tuned F1=0.555.

### 6.2.4 Noise Robustness Evaluation: Synthetic Data

Table 50 presents DER and its components, along with Overlap F1, Predicted Segments, and GT Segments for each noise and overlap condition: comparing baseline and fine-tuned Pyannote.

Table 50: Pyannote DER / Miss / FA / Conf by Condition and Model.

| Condition                    | Model      | DER(%)        | Miss(%)      | FA(%)        | Conf(%)       | Ov.F1        | Pred.Seg   | GT Seg |
|------------------------------|------------|---------------|--------------|--------------|---------------|--------------|------------|--------|
| <b>0% NOISE</b>              |            |               |              |              |               |              |            |        |
| ov0 / 0% noise               | Baseline   | 55.974        | 17.102       | 17.238       | 21.634        | 0.002        | <b>173</b> | 257    |
| ov0 / 0% noise               | Best model | <b>16.121</b> | <b>3.611</b> | <b>1.236</b> | <b>11.274</b> | <b>0.014</b> | 352        | 257    |
| ov5 / 0% noise               | Baseline   | 51.811        | 16.669       | 18.727       | <b>16.415</b> | 0.067        | 159        | 257    |
| ov5 / 0% noise               | Best model | <b>21.385</b> | <b>2.741</b> | <b>1.391</b> | 17.253        | <b>0.885</b> | <b>348</b> | 257    |
| ov10 / 0% noise              | Baseline   | 57.244        | 19.722       | 22.001       | 15.521        | 0.177        | <b>187</b> | 285    |
| ov10 / 0% noise              | Best model | <b>3.552</b>  | <b>1.697</b> | <b>1.594</b> | <b>0.261</b>  | <b>0.940</b> | 402        | 285    |
| <b>WITH BACKGROUND NOISE</b> |            |               |              |              |               |              |            |        |
| ov0 / 23% noise              | Baseline   | 58.493        | 17.310       | 16.001       | 25.182        | 0.006        | <b>174</b> | 237    |
| ov0 / 23% noise              | Best model | <b>10.486</b> | <b>2.409</b> | <b>2.898</b> | <b>5.179</b>  | <b>0.021</b> | 328        | 237    |
| ov5 / 23% noise              | Baseline   | 66.406        | 23.583       | 18.059       | 24.764        | 0.156        | <b>203</b> | 294    |
| ov5 / 23% noise              | Best model | <b>18.384</b> | <b>5.576</b> | <b>1.755</b> | <b>11.053</b> | <b>0.778</b> | 407        | 294    |
| ov10 / 17% noise             | Baseline   | 63.952        | 23.430       | 18.759       | 21.762        | 0.154        | <b>210</b> | 324    |
| ov10 / 17% noise             | Best model | <b>23.171</b> | <b>4.465</b> | <b>1.937</b> | <b>16.769</b> | <b>0.849</b> | 460        | 324    |

**Remarks.**

Bold values indicate the better result between the Baseline and Best model for each condition and metric.

**6.2.5 Results by Speaker Count: Synthetic Data**

Table 51 breaks down best model results by number of speakers. The consistent DER increase from 2 to 4 speakers is a characteristic behavior of Pyannote, unlike DiariZen which has an anomalous DER at the 3-speaker group.

Table 51: Pyannote DER / Miss / FA / Conf by Speaker Count on Synthetic Data.

| Speakers | Checkpoint | DER(%)        | Miss(%)      | FA(%)        | Conf(%)       | Ov.F1        | Pred.Seg   | GT Seg |
|----------|------------|---------------|--------------|--------------|---------------|--------------|------------|--------|
| 2 spk    | Baseline   | 55.010        | 17.831       | 19.322       | 17.857        | 0.082        | <b>173</b> | 266    |
| 2 spk    | Best model | <b>13.672</b> | <b>2.685</b> | <b>1.411</b> | <b>9.575</b>  | <b>0.612</b> | 369        | 266    |
| 3 spk    | Baseline   | 67.065        | 21.226       | 23.364       | 22.474        | 0.087        | <b>215</b> | 282    |
| 3 spk    | Best model | <b>17.008</b> | <b>4.490</b> | <b>2.190</b> | <b>10.327</b> | <b>0.520</b> | 409        | 282    |
| 4 spk    | Baseline   | 64.450        | 23.200       | 16.034       | 25.216        | 0.094        | <b>175</b> | 265    |
| 4 spk    | Best model | <b>21.643</b> | <b>5.661</b> | <b>1.682</b> | <b>14.300</b> | <b>0.533</b> | <b>355</b> | 265    |

**Remarks.**

The highlighted values in each column indicate the better result when comparing the Baseline and the Best model within the same speaker-count condition, name 2 speakers, 3 speakers, and 4 speakers.

### VIVOS Test Data — DER Components by Number of Speakers

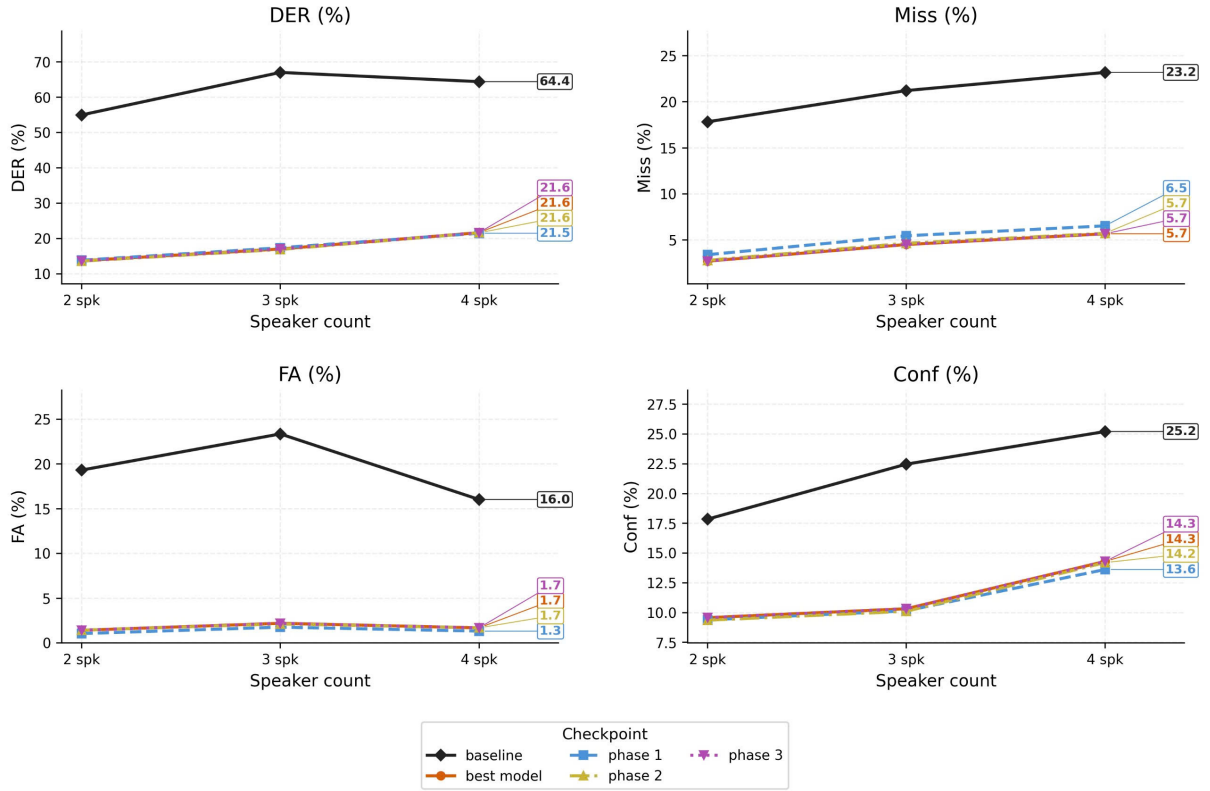


Figure 71: Pyannote DER components by speaker count on Synthetic Data.

#### 6.2.6 Results on Self-labeled Set (Primary Evaluation Set)

Table 52 presents Pyannote best model results on each self-labeled class. The differentiation pattern is similar to DiariZen but Confusion is significantly higher across all classes.

Table 52: Pyannote DER / Miss / FA / Conf on test\_labeled Set (best model).

| Dataset (test_labeled) | DER(%) | Miss(%) | FA(%)  | Conf(%) | Ov.F1 | Pred.Seg | GT Seg | Baseline DER |
|------------------------|--------|---------|--------|---------|-------|----------|--------|--------------|
| chuyen_ho              | 41.281 | 1.895   | 33.413 | 5.974   | 0.066 | 1042     | 632    | 59.164       |
| coi_moi                | 60.186 | 6.084   | 41.661 | 12.441  | 0.128 | 1398     | 669    | 92.307       |
| dustin_1               | 26.412 | 3.958   | 16.018 | 6.436   | 0.106 | 1236     | 785    | 56.185       |
| vif_1                  | 6.793  | 2.875   | 3.618  | 0.301   | 0.000 | 714      | 625    | 19.845       |
| vif_2                  | 6.897  | 0.540   | 5.972  | 0.386   | 0.000 | 375      | 522    | 15.155       |

## Test Labeled — Overlap Metrics and Predicted Segments by Class

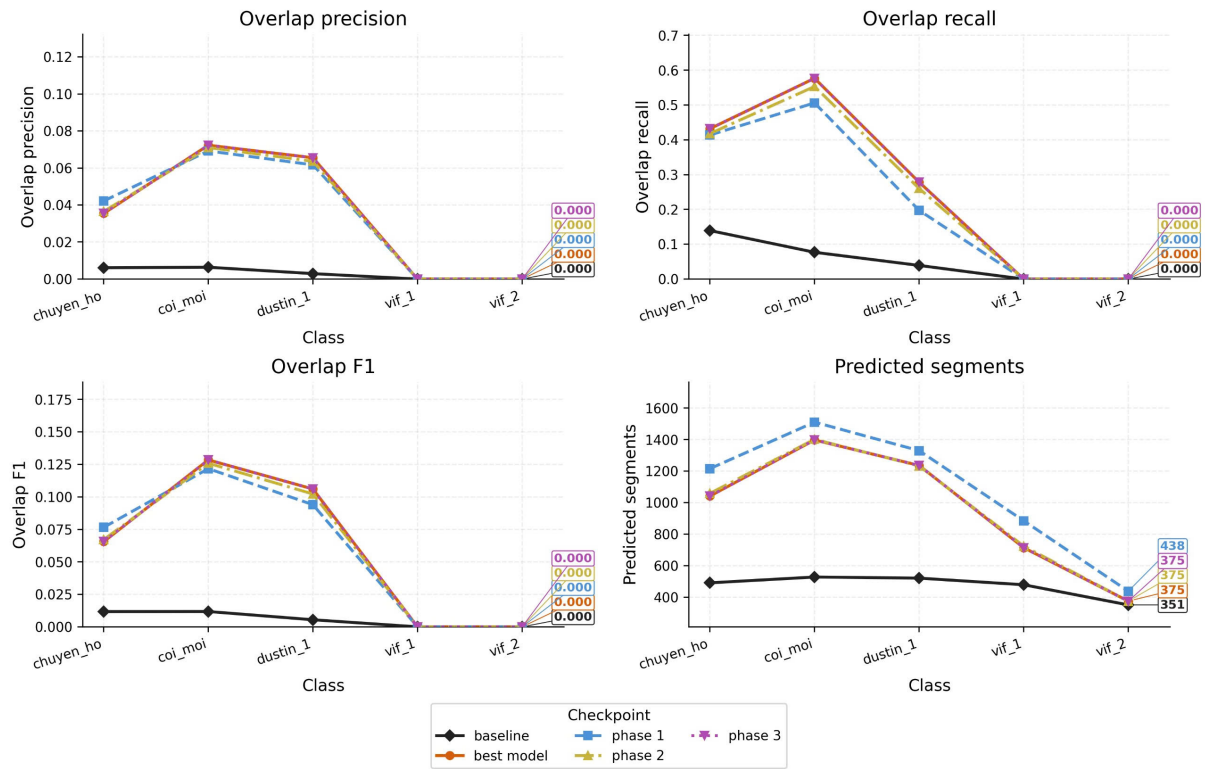


Figure 72: Pyannote Overlap F1 and Predicted Segments by class on test\_labeled.

## 6.3 PhoWhisper Results

### 6.3.1 PhoWhisper: Training Loss Visualization

#### Stage 1: Checkpoint-804 (Epoch 1).

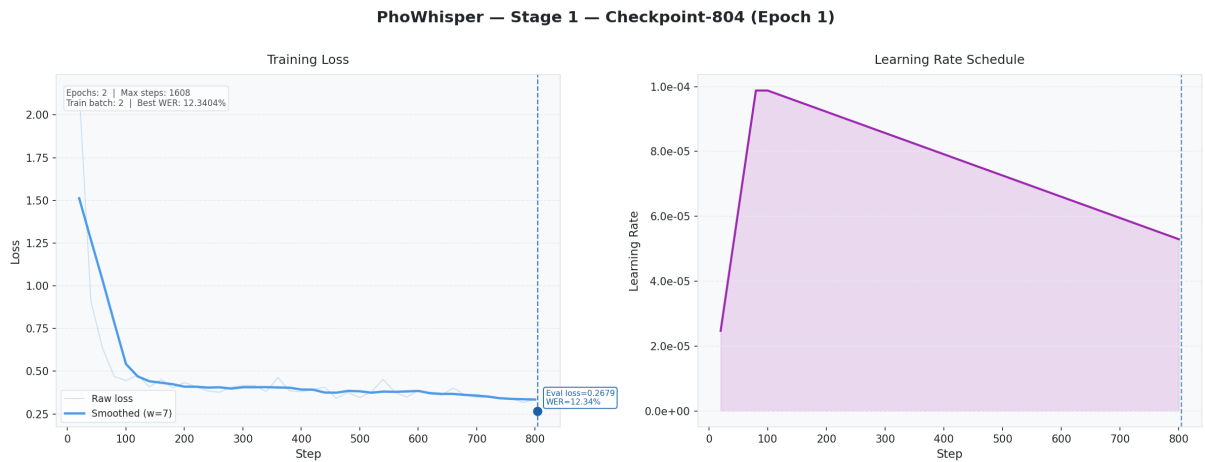


Figure 73: Stage 1 Checkpoint-804 Epoch 1.

## Stage 1: Checkpoint-1608 (Epoch 2 / Final).

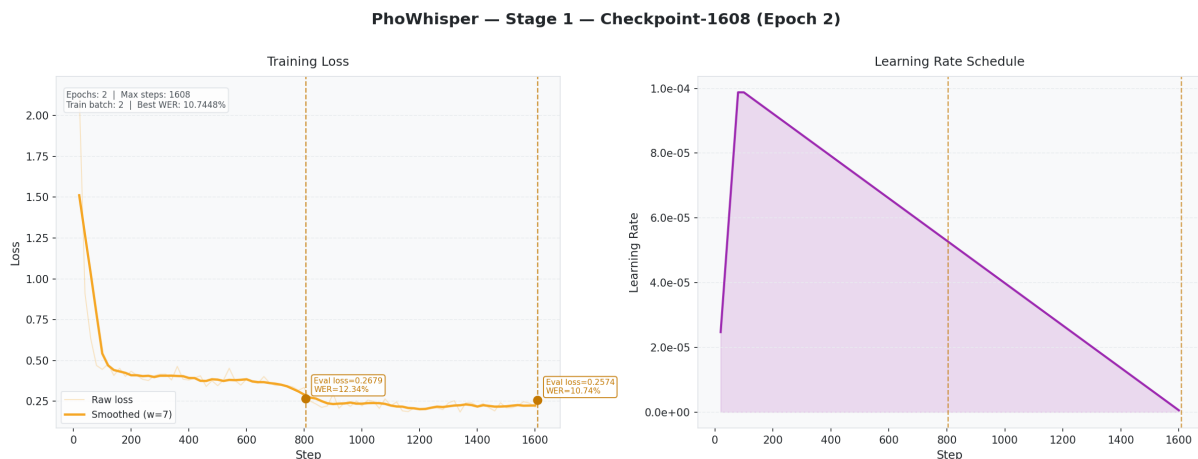


Figure 74: Stage 1 Checkpoint-1608 Epoch 2.

## Stage 1 Observations:

**Training Loss:** Drops sharply from 1.5 to 0.4 within the first 100 steps (epoch 1), then continues to decrease gradually from 0.35 to 0.25 through epoch 2.

**LR Schedule:** Warms up to a peak of  $1e-4$  and linearly decays to 0 at step 1608, completing the full decay schedule by the end of training.

**Eval:** WER improves from 12.34% (epoch 1) to 10.74% (epoch 2), with eval loss decreasing from 0.2679 to 0.2574, indicating continued improvement across both epochs.

## Stage 2: Checkpoint-965 (Epoch 1 / Final).

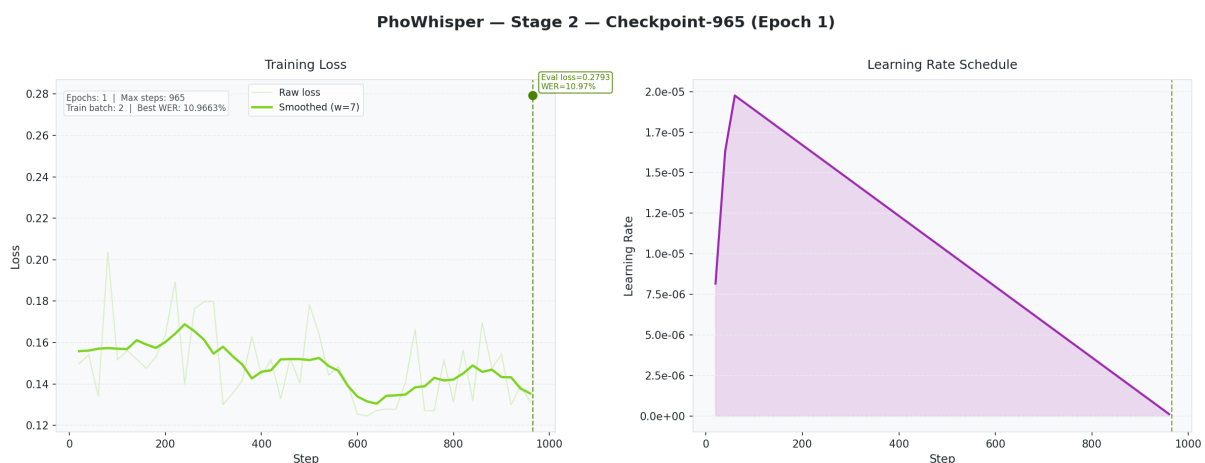


Figure 75: Stage 2 Checkpoint-965 Epoch 1.

## Observations:

**Training Loss:** Starts at a lower level of 0.16 due to weight inheritance from Stage 1, with small fluctuations and a slight downward trend toward 0.14 by the end of the epoch.

LR Schedule: Peak LR is  $2e-5$ , significantly lower than Stage 1 ( $1e-4$ ), consistent with fine-tuning on a new dataset.

Eval: Loss = 0.2793, WER = 10.97%, slightly higher than Stage 1 Final at 10.74%, with no clear improvement observed.

### Combined: All 3 Checkpoints.

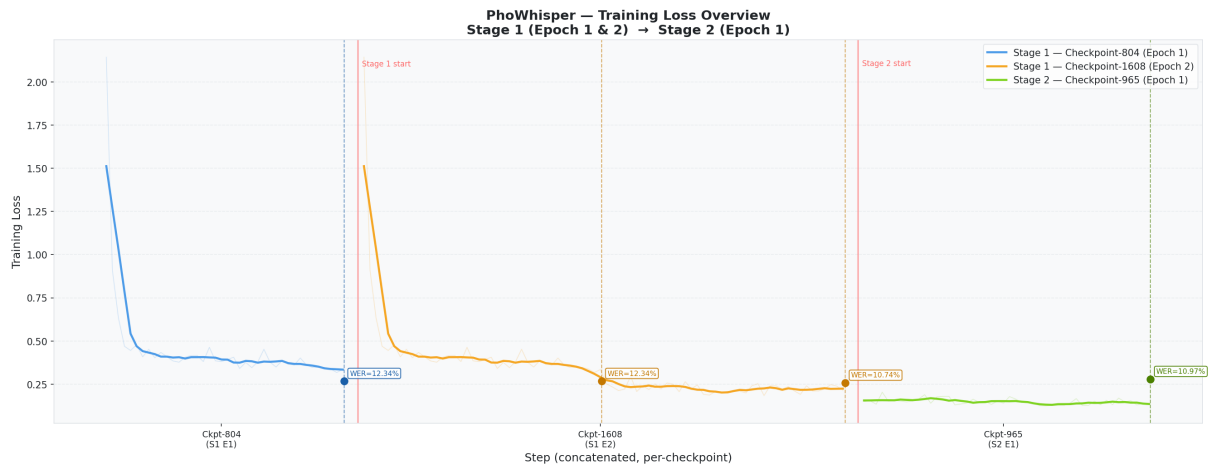


Figure 76: Combined All Stages.

### Observations:

Stage 1 Epoch 1 (blue): The largest loss reduction occurs here, concentrated in the early training steps.

Stage 1 Epoch 2 (orange): Loss continues to decrease with further WER improvement; this checkpoint yields the best result, with WER = 10.74%.

Stage 2 (green): Loss remains low at 0.15, but WER does not improve over Stage 1 Final, possibly due to a distribution shift between the two training datasets.

### 6.3.2 Evaluation Method: Diagnostic Decomposition

Evaluated on the test set (5 datasets, 2.51 hours, 1,655 samples). Rather than reporting only aggregate WER, the system applies Diagnostic Decomposition: decomposing WER into 3 error types S/D/I. All text undergoes Unicode NFC normalization, lowercasing, and punctuation removal before WER computation.

### 6.3.3 Overall Results: Base → Stage 1 → Stage 2

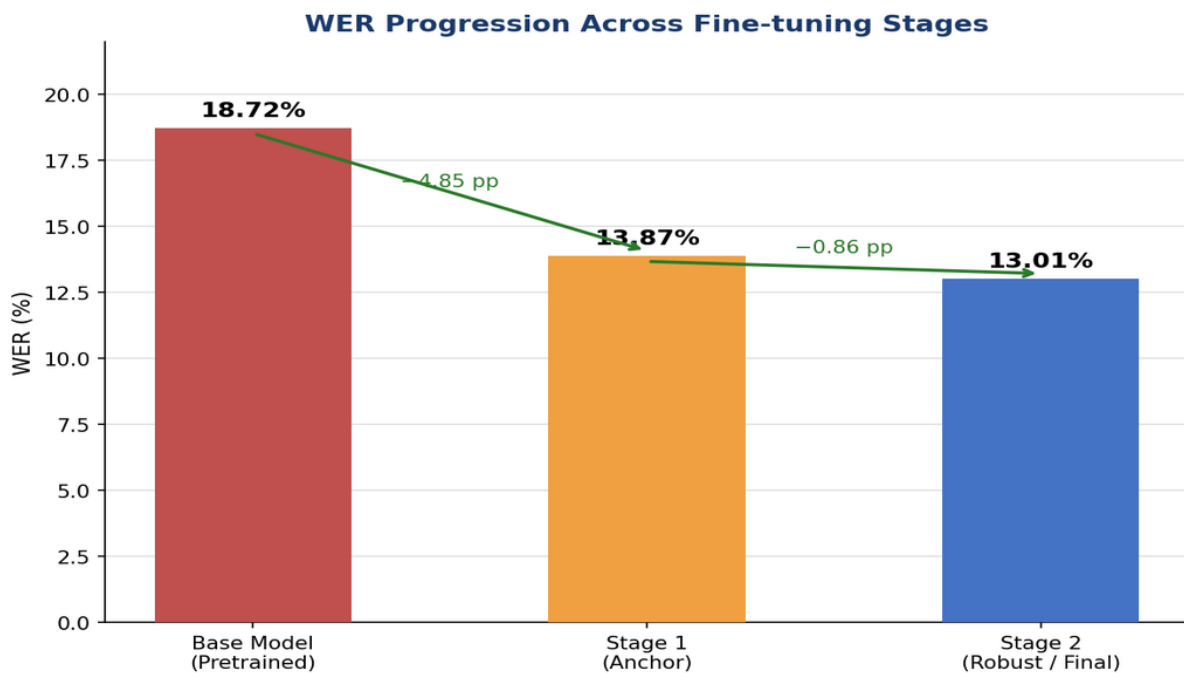


Figure 77: WER progression across fine-tuning stages (test set, 1,655 samples).

Table 53: Global WER Comparison of Base / Stage 1 / Stage 2 (1,655 test samples).

| Metric                  | Base Model | Stage 1  | Stage 2 / Final | Delta   |
|-------------------------|------------|----------|-----------------|---------|
| Overall WER (%)         | 18.72%     | 13.87%   | 3.01%           | −5.71   |
| Substitutions (S)       | 3.691      | 2.743    | 2.803           | −888    |
| Deletions (D)           | 1.936      | 2.185    | 2.110           | +174    |
| Insertions (I)          | 3.589      | 1.897    | 1.490           | −2.099  |
| S (% of total words)    | 7.50%      | 5.57%    | 5.69%           | −1.81   |
| D (% of total words)    | 3.93%      | 4.44%    | 4.29%           | +0.36   |
| I (% of total words)    | 7.29%      | 3.85%    | 3.03%           | −4.26   |
| Runtime / 1.655 samples | 437.68 s   | 441.32 s | 428.12 s        | −9.56   |
| Avg (s/sample)          | 0.2645     | 0.2667   | 0.2587          | −0.0058 |

### 6.3.4 Results by Domain

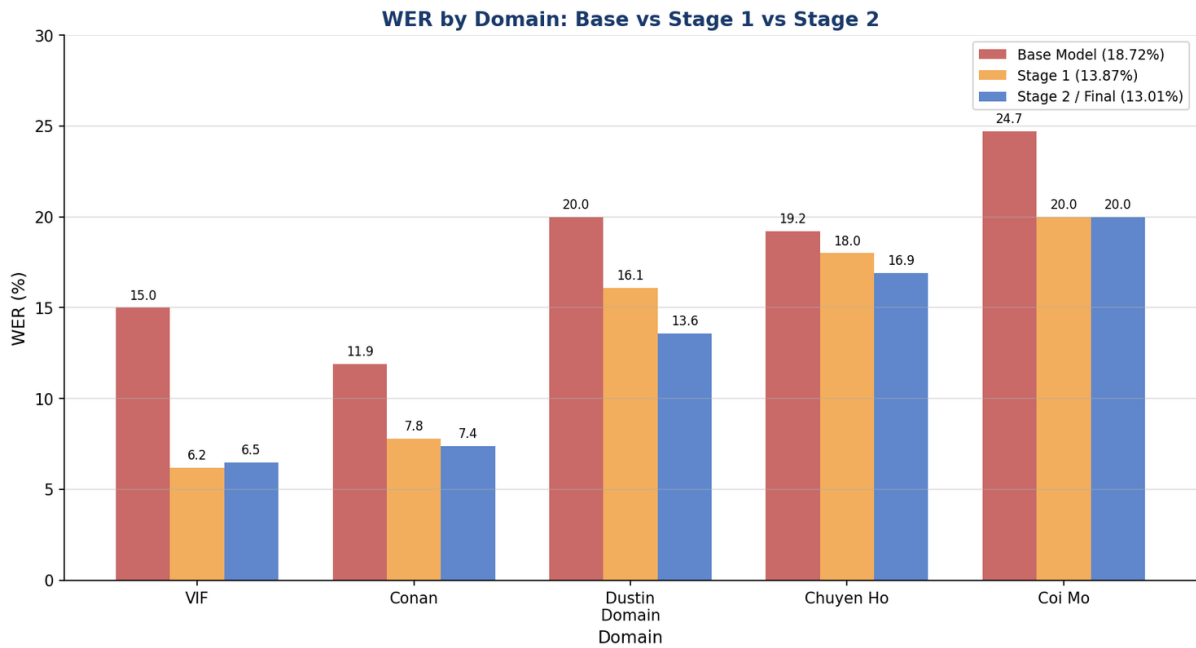


Figure 78: WER by domain: Base / Stage 1 / Stage 2.

Table 54: WER Comparison of base model and checkpoints.

| Domain                | Base WER | Stage 1 WER (S/D/I)     | Stage 2 WER (S/D/I)   | Delta |
|-----------------------|----------|-------------------------|-----------------------|-------|
| VIF                   | 14.97%   | 6.23% (2.38/1.43/2.42)  | 6.48% (2.5/1.5/2.5)   | -8.49 |
| Conan                 | 11.86%   | 7.75% (3.86/2.79/1.10)  | 7.38% (3.8/2.7/0.8)   | -4.48 |
| Dustin                | 20.00%   | 16.12% (5.87/5.09/5.16) | 13.62% (5.9/4.9/2.9)  | -6.38 |
| Chuyen Ho Chuyen Minh | 19.22%   | 18.03% (6.40/6.12/5.51) | 16.93% (6.7/5.9/4.4)  | -2.29 |
| Coi Mo                | 24.73%   | 19.99% (9.77/6.86/3.36) | 19.96% (10.0/6.6/3.4) | -4.77 |

Table 55: Runtime Performance RTX 3090 Ti, FP16.

| Model                       | Total runtime (1,655 samples) | Average (s/sample) |
|-----------------------------|-------------------------------|--------------------|
| Base Model                  | 437,68 s                      | 0,2645 s           |
| Best Stage 1                | 441,32 s                      | 0,2667 s           |
| <b>Best Stage 2 (Final)</b> | 428,12 s                      | 0,2587 s           |

### 6.3.5 Result Analysis

**Stage 1 Leap:** WER drops from 18.72% to 13.87%. The critical gain is Insertions falling from 7.29% to 3.85% (1,692 fewer errors) eliminating the hallucination behaviour of the pre-trained model on silence and background noise.

**Role of Stage 2:** WER further decreases to 13.01%. Insertions continue to fall 1,897 to 1,490 (3.85% to 3.03%). Substitutions increase slightly 5.57% to 5.69% an acceptable trade-off: under noisier data the

model misidentifies words rather than omitting them.

**Data Quality vs. Quantity Relationship:** VIF (gold\_real, 1.48h) achieves WER 6.48% lower than Conan (silver\_real, 12.85h, WER 7.38%). This confirms that audio quality exerts greater influence than quantity within the studied range.

### 6.3.6 Technical Notes

#### VRAM Spike During Evaluation.

When running `predict_with_generate` (auto-regressive), VRAM consumption is higher than in a normal forward pass. On the RTX 3090 Ti, this can cause OOM if the evaluation batch is too large. Solution: `per_device_eval_batch_size=1` and `generation_max_length=256`.

#### Hallucination on Short Segments.

The Whisper model can generate infinitely repeated word sequences on segments < 0,5 seconds or segments containing mostly noise. The pipeline applies: `skip segment < 0.5s`; `generation_max_length=256` as a hard limit; blacklist filter for common hallucination phrases (for example: “bat dau bat dau bat dau...” on a 0.34 s segment).

## 6.4 Qwen Pipeline Evaluation Results

### 6.4.1 Evaluation Methodology

The system is evaluated on 5 separate test sets (Chuyen Ho, Coi Mo, Conan, Dustin, VIF — 5 PhoWhisper domains) using two methods:

- (1) Automatic evaluation — checking schema, evidence support, genericness, confidence.
- (2) LLM Judge using Qwen/Qwen2.5-7B-Instruct as the judge model (available for Chuyen Ho, Coi Mo, VIF).

## 6.4.2 Automated Evaluation Results

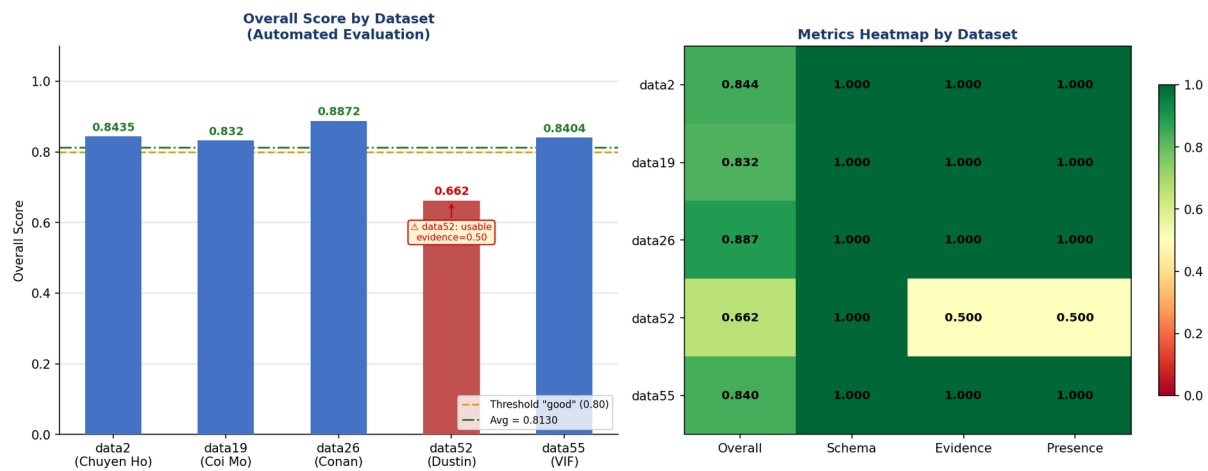


Figure 79: Automated evaluation results across five datasets.

Table 56: Automated Evaluation Results.

| Dataset               | Overall       | Schema       | Evidence     | Speaker Pres. | Generic      | Interp.     |
|-----------------------|---------------|--------------|--------------|---------------|--------------|-------------|
| Chuyen Ho (Podcast)   | <b>0.8435</b> | 1.000        | 1.000        | 1.000         | 1.000        | good        |
| Coi Mo (Conversation) | <b>0.8320</b> | 1.000        | 1.000        | 1.000         | 1.000        | good        |
| Conan (Anime)         | <b>0.8872</b> | 1.000        | 1.000        | 1.000         | 0.615        | good        |
| Dustin (Vlog)         | <b>0.6620</b> | 1.000        | 0.500        | 0.500         | 1.000        | usable      |
| VIF (News broadcast)  | <b>0.8404</b> | 1.000        | 1.000        | 1.000         | 1.000        | good        |
| <b>AVG</b>            | <b>0.8130</b> | <b>1.000</b> | <b>0.900</b> | <b>0.900</b>  | <b>0.923</b> | <b>good</b> |

Table 57: Overall Formula Automated Evaluation.

| Component            | Weight | Calculation               | Sub-total     |
|----------------------|--------|---------------------------|---------------|
| Schema               | 35%    | $0.35 \times 1.0$         | 0.3500        |
| Evidence             | 35%    | $0.35 \times 1.0$         | 0.3500        |
| Generic Score        | 15%    | $0.15 \times (1.0 - 1.0)$ | 0.0000        |
| Compact Score        | 15%    | $0.15 \times 0.9567$      | 0.1435        |
| <b>Overall Score</b> |        |                           | <b>0.8435</b> |

**Note:** Final Formula: Overall = 35%(S) + 35%(E) + 15%(G) + 15%(C) = 0.8435.

Both Generic and Compactness metrics act as penalties, each carrying a 15% weight. A Generic Rate of 1.000 nullifies its respective score entirely, as  $\text{generic\_score} = 1.0 - 1.0 = 0$ . Additionally, a Compactness penalty is applied when the average summary length exceeds the 30-word threshold. This penalty scales linearly using a 50-word tolerance window, meaning the score gradually drops to zero if the summary exceeds the limit by 50 extra words (reaching 80 words in total). For example, an average length of 32.16 words reduces the compactness score to  $1.0 - \frac{32.16 - 30}{50} = 0.9567$ , proportional to its 15% weight.

### 6.4.3 LLM Judge Results (Qwen 7B)

Table 58: LLM Judge Results Using Qwen 7B.

| Criterion             | Chuyen Ho | Coi Mo   | VIF      | DustinGo | Conan     | Max Score |
|-----------------------|-----------|----------|----------|----------|-----------|-----------|
| Main Summary Score    | 3         | 2        | 2        | 2        | 3         | 4         |
| Speaker Summary Score | 2         | 2        | 2        | 1        | 2         | 2         |
| Faithfulness Score    | 2         | 2        | 2        | 3        | 4         | 4         |
| Conciseness Score     | 2         | 2        | 2        | 2        | 2         | 2         |
| <b>Total Score</b>    | <b>9</b>  | <b>8</b> | <b>8</b> | <b>8</b> | <b>11</b> | <b>12</b> |

### 6.4.4 Performance Benchmark

Table 59: VRAM and Tokens/s Benchmark.

| Model Variant        | Precision   | Peak VRAM (MB)  | Tokens/s     | Notes                                 |
|----------------------|-------------|-----------------|--------------|---------------------------------------|
| Qwen 2.5-1.5B        | 4-bit       | 1.196 MB        | 2,81         | Vram less                             |
| <b>Qwen 2.5-1.5B</b> | <b>BF16</b> | <b>3.162 MB</b> | <b>12,39</b> | <b>Optimal performance</b>            |
| Qwen 2.5-7B          | 4-bit       | 5.740 MB        | 4,15         | Requires 5× more VRAM than 1.5B 4-bit |

#### Remarks.

The selected model for deployment is Qwen 2.5-1.5B (BF16), as it provides the best balance between inference speed and memory efficiency.

## 6.5 Discussion

### 6.5.1 General Technical Notes

#### Prioritizing Miss and Confusion in Diarization Evaluation.

In an ASR-integrated pipeline, the three DER components have different practical impacts. Miss (missed speech) causes complete loss of audio content at that segment: there is nothing for ASR to process. Confusion (wrong speaker assignment) corrupts the entire conversation analysis, even when speech is correctly detected. Both Miss and Confusion cannot be corrected at the downstream NLP or ASR step. In contrast, False Alarm (falsely detecting speech where there is none) affects DER but can be partially filtered at the VAD or ASR post-processing step, or may be acceptable in many applications in too much noise audio. Therefore, optimizing Miss and Confusion has higher practical value than optimizing FA alone, even if higher FA increases the overall DER.

#### Distinguishing Crop-level DER and File-level DER.

Crop-level DER (DER computed on each short crop) reflects local segmentation quality but is not equivalent to file-level DER, as the end-to-end pipeline also depends on embedding quality and the

global clustering step. With pyannote.metrics, file-level DER uses global Hungarian mapping with  $\text{collar} = 0.25\text{s}$ : a more realistic metric for the “who spoke when” task across the full file. When interpreting training graphs, it is important to distinguish clearly: the validation DER graphs during fine-tuning are crop-level DER (used to select checkpoints), while the final DER reported in result tables is file-level DER.

### **Loss Functions of the Two Models Cannot Be Directly Compared.**

DiariZen uses PIT multilabel BCE: each speaker slot is optimized independently, gradients only flow through the best permutation. Pyannote uses powerset cross-entropy: all combinations of active states are encoded as a single class and optimized via softmax. The two loss functions measure different quantities in different problem spaces, so absolute loss values do not reflect relative quality between the two models. Training loss graphs are only useful for tracking the convergence trend of each individual model and should not be compared across models.

### **Notes on Evaluation Data and Result Interpretation.**

1. **Synthetic data:** The synthetic data mix set is created by combining single-speaker files from the VIVOS corpus with precisely controlled overlap parameters (0%, 5%, 10%) and noise levels. Advantage: individual factors such as overlap level, noise level, and number of speakers can be isolated for separate quantitative analysis. Limitation: the data does not reflect natural conversation; characteristics such as natural speaker turns, conversational rhythm, and diverse environmental noise are not fully simulated. DER results on synthetic data (7-23%) may be more optimistic than real-world conditions.
2. **Self-labeled, real-world uncontrolled data:** The self-labeled set consists of real Vietnamese conversational recordings with manual annotations. Parameters such as overlap level, number of speakers, or background noise cannot be controlled; these are actual deployment conditions. DER on self-labeled data (7-52%, depending on class) more accurately reflects real performance. In particular, high FA at *chuyen\_ho* (26%) and *coi\_moi* (41%) is not due to weak models but reflects content characteristics: heavy environmental noise, background music, and audience sounds, which are factors absent from synthetic data.
3. **Guidelines for using results:** When evaluating models, the primary metric should be the results on the self-labeled set because it serves as the realistic benchmark. Results on synthetic data are valuable for understanding model behavior under controlled conditions and for quantitative comparison across conditions, but they should not be used directly to predict deployment performance.

## **6.5.2 DiariZen Discussion**

### **Convergence Analysis.**

The DiariZen training process shows clear convergence signals across all three phases, consistent with theoretical predictions when applying gradual unfreezing on domain-specific data. The slight increase in validation DER from ep004 (13.945%) to ep007-ep014 (13-14%) does not reflect quality degradation but is a consequence of enabling data augmentation from Phase 2: the model is exposed

to a harder data distribution (more overlap, more noise), causing crop-level DER to temporarily increase while overall file-level DER is improved.

This is an expected and desirable outcome: the model learns harder conditions from Phase 2 without forgetting the easier conditions from Phase 1, as evidenced by Confusion and Miss remaining stably low (1.03% and 2.34% respectively) throughout Phases 2 and 3.

### **Noise Results: Analysis by Miss and Confusion.**

DiariZen fine-tuned shows strong reductions in both Miss and Confusion across all conditions: Miss drops from 12-18% (baseline) to 0.1-2.2% (fine-tuned), and Confusion from 3-13% to 0.1-7%. FA increases modestly to 6-14% compared to the baseline range of 7-8%, a structural consequence of sigmoid multilabel decoding at low thresholds rather than a model defect.

On synthetic data, absolute DER values (7-15%) should be interpreted with caution: overlap and noise are artificially constructed, so these figures are likely more optimistic than real-world conditions. The self-labeled results (DER 33-51% at `chuyen_ho` and `coi_moi`) are the more informative reference point.

### **Rationale for Threshold 0.25: Analysis on synthetic and Self-labeled Set.**

Threshold 0.25 is the most consequential hyperparameter in the DiariZen pipeline, and its selection rests on two complementary arguments evaluated primarily on the self-labeled set.

The first FA-Miss trade-off. At  $\text{thr}=0.25$ , Miss remains very low across all classes (0.47-6.23%): a critical constraint since missed speech is unrecoverable at the ASR step. Raising the threshold to  $\text{thr}=0.45$  reduces FA from 26.22% to 19.58% (`chuyen_ho`) and from 40.36% to 24.62% (`coi_moi`), while driving Miss up substantially: `chuyen_ho` from 3.89% to 11.36%, `coi_moi` from 6.23% to 14.87%. The asymmetry of this trade-off makes higher thresholds inadvisable.

The second DER improvement. On the `ov5` condition, DER decreases across thresholds as follows:  $\text{thr}=0.05$  (28.0%)  $\rightarrow$  0.15 (15.6%)  $\rightarrow$  0.25 (11.8%)  $\rightarrow$  0.35 (9.35%)  $\rightarrow$  0.45 (8.71%). The reduction from 0.05 to 0.25 yields 16 percentage points; the further reduction from 0.25 to 0.45 yields only 3 points, while Miss rises from 0.41% to 2.48%. On `ov0`, DER decreases from 23.9% to 4.0% across the same range, but Miss increases from 0.01% to 1.36% while FA decreases from 22.2% to 2.17%. Threshold 0.25 therefore sits at the elbow point of the DER curve: the steepest gains have been captured while Miss remains negligible.

### **Overlap Recall, Precision, and Detection Capability.**

**Overlap Recall - clear improvement.** On the overall synthetic data set, Recall increases from 0.344 (baseline) to 0.831 (ep004), stabilizing at 0.785 in the best model. Under controlled conditions the signal is even stronger: `ov5` reaches Recall=0.989 and `ov10` reaches Recall=0.997 at  $\text{thr}=0.25$ , indicating that fine-tuning has effectively resolved the overlap detection deficit in terms of coverage.

**Overlap Precision - no significant improvement.** Precision on the synthetic data set barely moves across checkpoints: 0.383 (baseline)  $\rightarrow$  0.417 (ep004)  $\rightarrow$  0.381 (best model). At  $\text{thr}=0.25$ , fewer than 40% of regions flagged as overlap are genuine overlap. The gap between high Recall and low Precision reflects a systematic over-detection tendency: the model captures nearly all true overlap but also activates on many single-speaker regions. Under controlled conditions the picture is more favorable -

ov5 at thr=0.25 gives Precision=0.484, ov10 gives Precision=0.666 - suggesting that over-detection is partly driven by the acoustic complexity of real-world data rather than a fundamental model limitation.

**Effect on F1.** Overlap F1 increases from 0.39 (baseline) to 0.48 (best model): a meaningful gain, though not yet high. The over-detection pattern is a structural property of the multilabel sigmoid formulation at thr=0.25; raising the threshold to 0.35-0.45 improves Precision substantially (ov5: 0.584→0.694) but at the cost of increased Miss, which is why thr=0.25 is retained as the operating point.

### **Segment Count and Its Relationship to DER.**

A commonly held intuition is that higher-than-GT segment counts imply poor DER, while lower-than-GT counts imply acceptable DER. The experimental data shows a more nuanced picture.

On the side support: the vif class has PredSeg=423, below GT=625 (−32%), and achieves DER=6.6% the best among self-labeled classes. Because vif is a structured talk show with little overlap and few speaker changes, the under-segmentation is benign: segments that exist are correctly attributed, and the missing segments contribute only minor Miss at transition points.

Against the intuition, two cases stand out. The Pyannote baseline creates only 188 segments versus GT=271 (−32%), yet DER reaches 62.2%: here, under-segmentation arises from over-merging distinct speakers into shared segments, producing severe Confusion rather than benign under-coverage. Conversely, coi\_moi with PredSeg=1028 versus GT=669 (+54%) also reaches DER=51.0%: over-segmentation at this scale introduces compounding FA and speaker assignment errors.

**The decisive factor is speaker assignment quality per segment, not segment count alone.** Two systems with identical segment counts can differ dramatically in DER depending on whether the underlying cause is content-driven simplicity or pipeline-driven merging and splitting errors. Segment count should therefore be treated as a diagnostic signal that prompts further investigation, not as a performance predictor in its own right.

### **Results by Speaker Count.**

DER at the 3-speaker group (19.19%) is higher than the 4-speaker group (14.90%), which may appear counterintuitive given that 3-speaker conversations are structurally simpler than 4-speaker ones. The explanation lies in the training data distribution: the majority of self-labeled sessions involve 4 or more speakers, so the model receives disproportionately more exposure to 4+ speaker configurations during fine-tuning. Two-speaker conversations are inherently easier regardless of exposure, while 3-speaker conversations are harder than 2-speaker (9.42%) ones yet underrepresented relative to 4-speaker ones - a combination that elevates DER for this group specifically.

### **Results on Self-labeled Set: Primary Evaluation Set.**

The self-labeled set is the primary evaluation set, reflecting actual deployment conditions. Results differentiate clearly by class acoustic characteristics:

**vif\_1 and vif\_2** (DER 6.62–7.52%): experience-sharing talk shows with a clearly structured conversational format, little background noise, and a well-defined speaker set. This is the most acoustically favorable condition in the self-labeled set. Overlap F1  $\approx 0$  is consistent with the near-absence of genuine simultaneous speech in this format.

**dustin\_1** (DER 24.52%): a talk show combined with outdoor activities and diverse situations, with moderate background noise. FA of 17.71% and Conf of 3.41% are both well-controlled.

**chuyen\_ho** (DER 32.74%, FA 26.22%): a high-activity talk show with diverse environmental noise and ambient sounds. Elevated FA reflects the acoustic difficulty of distinguishing speech from characteristic background sounds in this format; however, Miss of only 3.89% confirms that the model retains strong speech detection despite the noise.

**coi\_moi** (DER 51.04%, FA 40.36%): a gameshow with many participants, background music, sound effects, and sustained studio noise - the acoustically heaviest class in this study. Miss of 6.23% demonstrates that the model detects virtually all speech present, but FA of 40.36% reflects the current pipeline's inability to suppress non-speech activations in this environment.

**Summary.** Miss and Confusion remain well-controlled (<7%) across all classes, which is the primary design priority of this study. High FA at **chuyen\_ho** and **coi\_moi** is a practical limitation rather than a detection failure: the model finds speech that is present but also activates on acoustic events that resemble speech. This is partially addressable through VAD post-processing or targeted inclusion of hard negative training examples.

#### **Overlap Detection synthetic data and on Self-labeled Set.**

On the synthetic data set, the best DiariZen model achieves Overlap F1 = 0.48 compared to baseline 0.39, representing a 23% relative increase. Ep004 peaks at Overlap F1 = 0.51, with a slight decrease at ep007-ep014 consistent with the trade-off introduced by augmentation in later phases. Predicted segment count decreases from 487 (baseline) to 388 (best model), moving toward ground-truth 276 but remaining 41% above it.

On the test\_labeled set, Overlap F1 is near zero at **vif\_1** and **vif\_2**, consistent with the low actual overlap rate in structured talk shows. **coi\_moi** and **dustin\_1** show higher values (0.09-0.10), reflecting greater overlap frequency in those formats. Absolute values nonetheless remain low across the board, indicating that overlap detection on real-world Vietnamese conversational data remains an open problem regardless of model quality. This should not be interpreted as model weakness: the gap between synthetic and real-world Overlap F1 reflects the inherently harder acoustic conditions of in-the-wild recordings rather than a failure to generalize from training data.

### **6.5.3 Pyannote Discussion**

#### **Noise Results: Miss, FA, and Confusion.**

Pyannote fine-tuned shows clear improvement on both the synthetic data set and the self-labeled set. On synthetic data, DER decreases from 51-66% to 3.6-23%, Miss from 17-24% to 1.7-5.6%, and FA from 16-22% to 1.2-2.9%. On the self-labeled set, the improvement is smaller in magnitude but still substantial: **vif** from 20% to 7%, **coi\_moi** from 92% to 60%. The stronger improvement on synthetic data is expected, as controlled overlap and noise conditions more closely match the training distribution.

From the perspective of the individual error components, Miss and FA both decrease substantially after fine-tuning: Miss from 17-24% to 1.7-5.6%, and FA from 16-22% to 1.2-2.9% the latter being significantly lower than DiariZen. Confusion, however, remains high at 0.26-16.8%, with the worst

cases at ov5/0%noise (17.3%) and ov10/17%noise (16.8%). Low FA is a clear structural advantage of Pyannote on synthetic data; the practical value of this advantage is partially offset by high Confusion, since wrong speaker assignment at this stage cannot be corrected by any downstream component. On the self-labeled set, Pyannote’s FA rises to 30-40%, comparable to DiariZen, confirming that elevated FA in noisy conditions reflects the acoustic properties of the data rather than a model-specific weakness.

### **Threshold Sweep: argmax Insensitive to External Threshold.**

Pyannote’s threshold sweep yields flat results across all tested values. This is a design property, not a defect: the `argmax_powerset` decode mode internalizes the segmentation decision into the class probability distribution, leaving no role for an external sigmoid threshold. Consequently, optimizing Pyannote requires tuning pipeline-level parameters `min_duration_on`, `min_duration_off`, and the clustering threshold rather than searching over a scalar threshold as in DiariZen.

### **Overlap Recall, Precision, and Detection Capability.**

**Overlap Recall improvement but still limited.** On the overall synthetic data set, Recall increases from 0.108 (baseline) to 0.499 (best model). While the absolute gain is large, Recall=0.499 remains substantially below DiariZen (0.785), meaning the best Pyannote model still misses approximately 50% of actual overlap regions. Under controlled noise conditions the picture improves: ov5/0%noise achieves Recall=0.8 and ov10/0%noise achieves Recall=0.9, suggesting that Pyannote’s overlap detection is more reliable when acoustic conditions are clean and overlap boundaries are unambiguous.

**Overlap Precision very high, characteristic of powerset.** Precision on the synthetic data set increases from 0.094 (baseline) to 0.633 (best model). On noise data, fine-tuned Pyannote achieves Precision  $\approx 1.0$  for ov5 and ov10: almost no false positive overlap detections. For ov0 where no actual overlap exists, Precision=0.0, which is the correct outcome. This pattern is the structural inverse of DiariZen: Pyannote is conservative it rarely reports overlap unless the evidence is strong, but consequently misses many genuine overlap regions. DiariZen is aggressive it detects nearly all true overlap but activates on many single-speaker regions as well. The mechanism behind Pyannote’s conservatism is that powerset argmax only fires when one class combination clearly dominates the probability distribution; weak or ambiguous overlap signals are absorbed into the single-speaker class instead.

**Net effect on F1.** Pyannote fine-tuned achieves Overlap F1=0.555 on the overall synthetic data set, higher than DiariZen (0.478), and a large improvement from baseline (0.088). The practical interpretation depends on the application: when precise overlap localization without false positives is the priority, Pyannote is more suitable; when complete overlap coverage including faint regions is needed, DiariZen is more suitable. In the context of an ASR pipeline, missing overlap (low Recall) is generally less damaging than falsely detecting it (low Precision), because missed segments are still processed as single-speaker audio, while false positive overlap can cause incorrect stream routing in the downstream transcription stage.

### **Segment Count and Its Relationship to DER.**

At baseline, Pyannote creates only 188 segments on the synthetic data set versus GT=271 (−32%), with DER=62.2%. On the individual noise conditions, segment counts are similarly low: ov0=174, ov5=203, ov10=210, all below GT 257-285. This under-segmentation does not reflect acoustically simple content; rather, it reflects the pipeline merging multiple distinct speakers into shared segments

(over-merging), as evidenced by Confusion=21.8% and Miss=20.8% at baseline both extremely high.

After fine-tuning, the best model produces 377 segments and DER drops to 17.4%: an absolute improvement of 44.7 percentage points. On the ov10 condition, fine-tuned Pyannote creates 460 segments, above GT=285 (+61%), yet DER remains at a reasonable level. The transition from under-segmentation to mild over-segmentation, accompanied by a large DER reduction, confirms that the baseline failure was caused by segmentation quality (over-merging) rather than segment count.

This finding connects directly to the DiariZen analysis in the preceding section. Two under-segmentation cases appear in this experiment: Pyannote baseline (188 < GT=271, DER=62.2%) and vif in DiariZen (423 < GT=625, DER=6.6%). Both produce fewer segments than ground-truth, yet their DER outcomes are opposite because the underlying causes differ: Pyannote under-segments by incorrectly merging speakers, while vif under-segments because the content genuinely requires fewer boundaries. This reinforces the conclusion established earlier: segment count is a diagnostic signal, not a performance predictor, and speaker assignment quality per segment is the factor that determines DER.

### **Results by Speaker Count.**

Unlike DiariZen, Pyannote (Best model) shows a consistent and monotonic DER increase across speaker counts: 13.67%(2 speakers) → 17.01%(3 speakers) → 21.64% (4 speakers). This is the expected behavior as speaker count increases the difficulty of both segmentation and clustering. FA remains remarkably stable across all groups (1.41-2.19%), confirming that speech detection is not the bottleneck. Confusion, by contrast, rises consistently from 9.58% → 10.33% → 14.30% as speaker count increases, identifying speaker discrimination as Pyannote’s primary weakness.

The root cause of this Confusion pattern is structural. Pyannote’s short sliding window (10s) produces more segments than DiariZen’s 16s chunks (Best model: 377 vs GT=271), and shorter segments yield less acoustic context per embedding, making ECAPA-TDNN representations less discriminative. AHC clustering then operates on these weaker embeddings, producing higher assignment errors as the number of speakers to distinguish increases.

### **Results on Self-labeled Set.**

On the self-labeled set, Pyannote follows a differentiation pattern similar to DiariZen, with performance stratified by acoustic complexity: vif\_1/vif\_2 achieves DER 6.8-6.9%; dustin\_1 achieves 26.41%; chuyen\_ho 41.28%; coi\_moi 60.19%. Compared to baseline, DER is reduced by 60-70% relatively across all classes a substantial improvement. However, Confusion remains the dominant limitation: across all classes, Pyannote’s Confusion is significantly higher than DiariZen’s (coi\_moi: 12.4% vs 4.4%), consistent with the over-segmentation and non-adapted embedding model analysis above.

On the synthetic data set, Pyannote best model achieves Overlap F1=0.555, higher than DiariZen (0.478). This advantage stems from the powerset formulation’s explicit representation of simultaneous speaker combinations, which enables more precise overlap class discrimination. Overlap Precision increases from 0.094 to 0.633-0.638; Recall from 0.108 to 0.474-0.499. The best model’s segment count (377) is approximately 39.1% above ground-truth (271), a significant improvement over the baseline (188 segments). When evaluated by speaker count, both Overlap F1 (0.52-0.61) and segment predictions (355-409) are stable, indicating consistent behavior regardless of the number of speakers in the session.

**Direct comparison.** Pyannote achieves higher Overlap F1 than DiariZen (0.555 vs 0.478) but substantially higher Confusion (11.4% vs 2.9%). The two results are not contradictory: overlap detection and speaker attribution are separate problems. Pyannote is more accurate at identifying where overlap occurs; DiariZen is more accurate at assigning the correct speaker to each region. Which model is more suitable depends on which error type is more costly for the target application.

### Comprehensive Root Cause Analysis.

The gap between Pyannote’s performance on synthetic data and on real-world self-labeled recordings, as well as the persistently high Confusion across all conditions, can be traced to five interacting causes.

**Domain mismatch.** The base Pyannote model was trained primarily on English data under acoustic conditions different from Vietnamese conversational speech. Fine-tuning on Vietnamese data partially compensates for this, but the mismatch is not fully resolved within the scope of this study.

**Narrow optimization space.** The `argmax_powerset` decode mode renders external threshold tuning ineffective. Meaningful optimization requires adjusting `min_duration_on`, `min_duration_off`, and the clustering threshold parameters not swept in this study.

**Formulation sensitivity to domain mismatch.** Powerset cross-entropy is theoretically well-suited to overlap modeling when the training data covers the full range of class combinations. In Vietnamese conversational conditions, however, overlap distribution is uneven and domain shift is significant, making the powerset formulation more sensitive to these factors than DiariZen’s multilabel BCE, which places independent sigmoid decisions on each speaker slot and degrades more gracefully under distribution shift.

#### 6.5.4 Comparison DiariZen and Pyannote

##### Training Process.

The two models use completely different loss functions (PIT multilabel BCE and powerset cross-entropy) so absolute values cannot be compared. Convergence is assessed based on train/val loss trends and validation DER for each model separately.

DiariZen: stable convergence: Train loss decreases steadily 0.172→0.105. Validation loss decreases 0.106→0.094 with very small variation after epoch 5. Validation DER converges quickly in Phase 1, slightly increases when augmentation is enabled (Phase 2), then stabilizes in Phase 3: good convergence behavior, no overfitting.

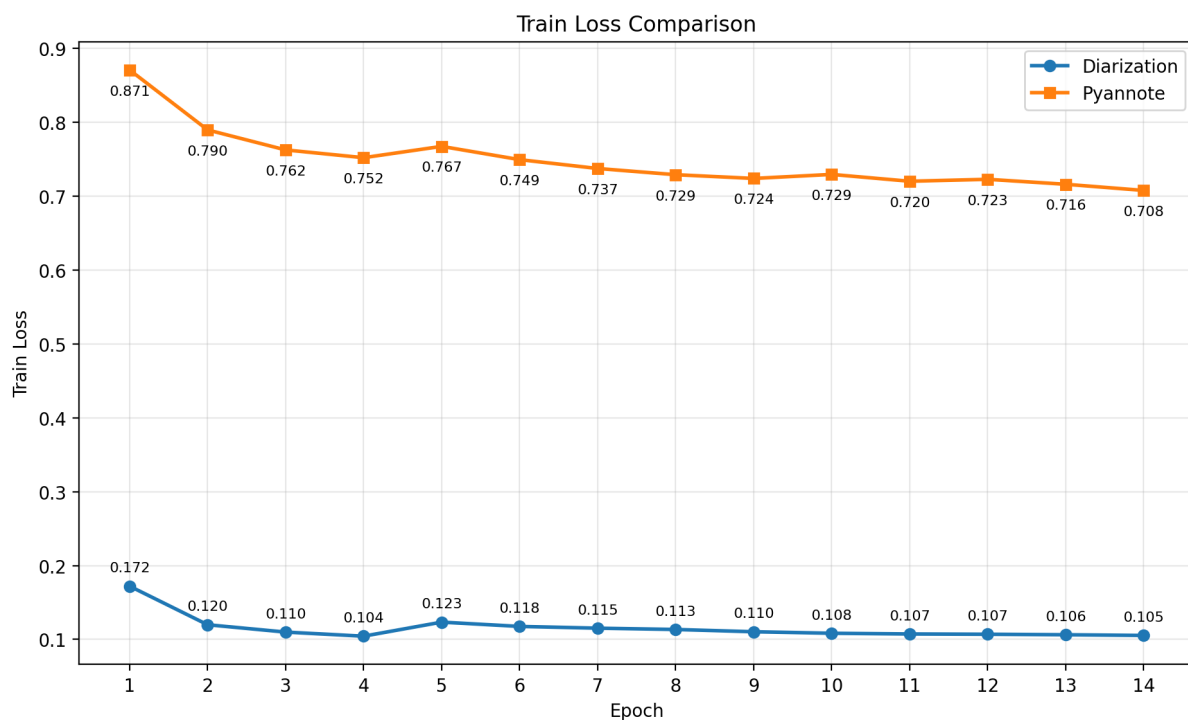


Figure 80: Train loss comparison of two models.

**Pyannote: less stable convergence:** Train loss decreases steadily 0.871→0.708, showing the model learns signals from the data. However, validation loss fluctuates non-monotonically (peak 0.561 at epoch 13, decreasing to 0.494 at epoch 14). Validation DER fluctuates in the 19-22% range throughout Phases 2-3 instead of converging flatly like DiariZen: a sign of greater sensitivity to domain mismatch.

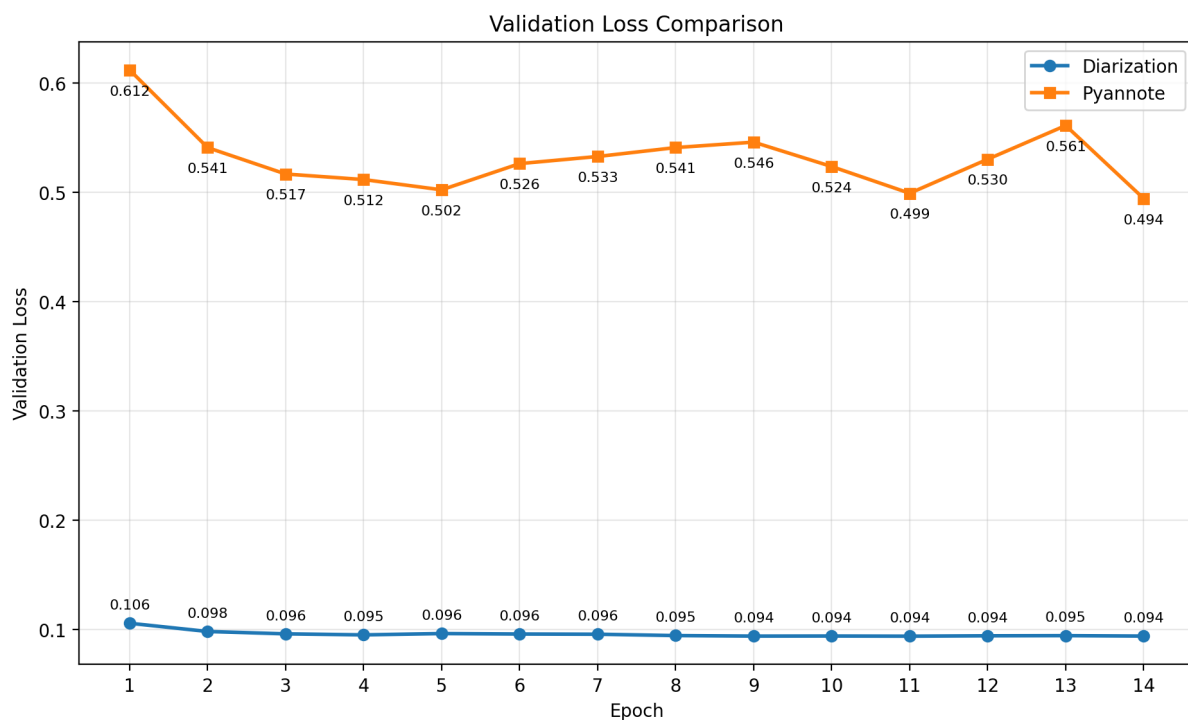


Figure 81: Validation loss comparison of two models .

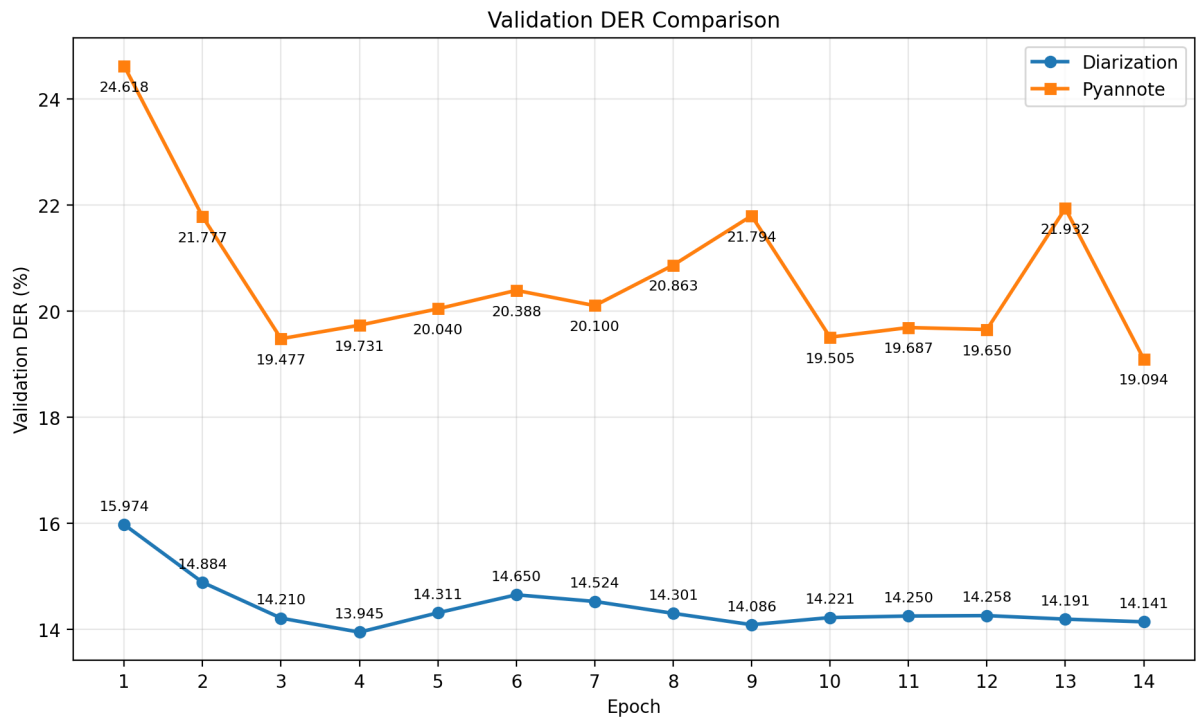


Figure 82: Crop-level validation DER comparison of two models.

### FA-like Surplus Diagnosis Report.

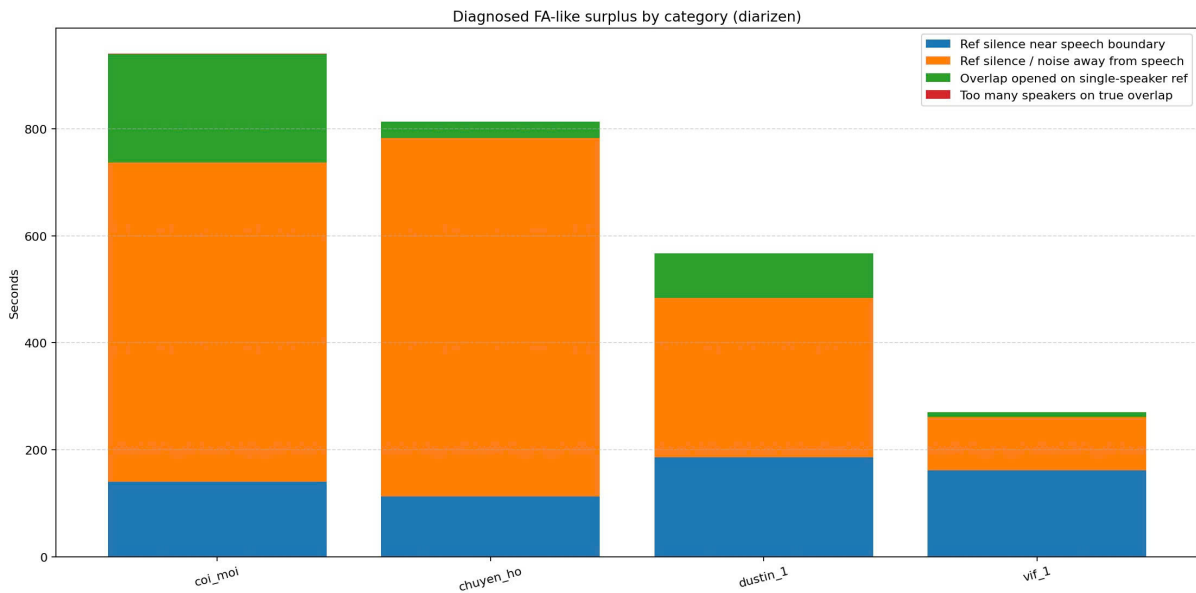


Figure 83: Pyannote (Best model).

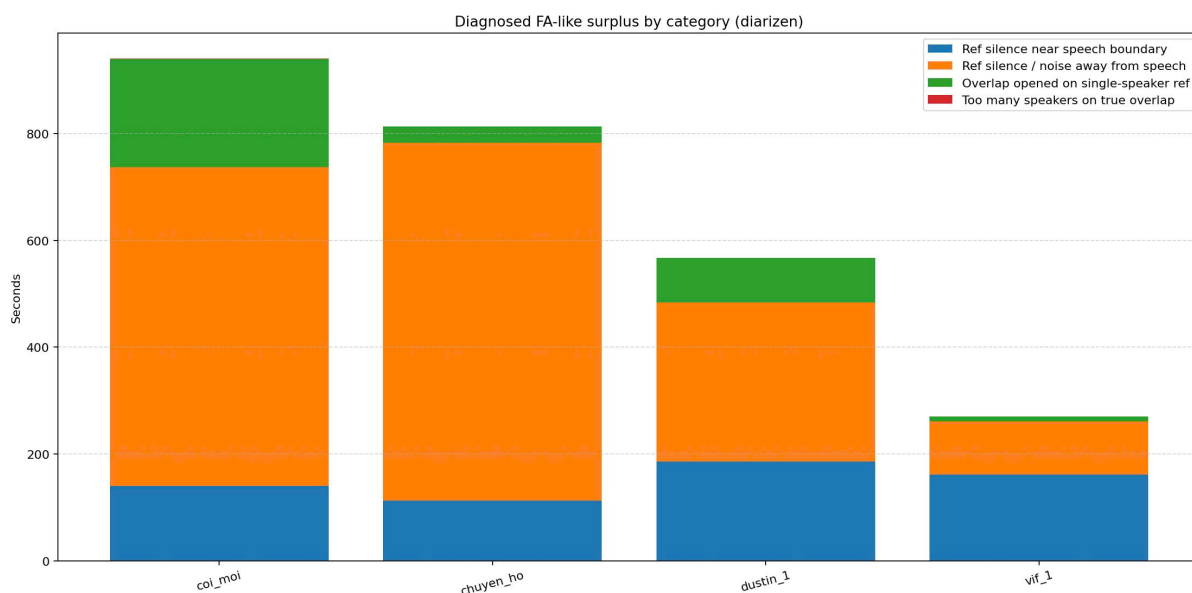


Figure 84: DiariZen (Best model).

### FA-like Surplus: Root Cause Analysis.

The stacked bar charts decompose the FA-like surplus of each model into four categories. Across both systems and all four test files, the dominant contributor is Ref silence/noise away from speech (orange): regions where the reference records silence or background noise that is acoustically distant from any speech boundary, yet the model activates a speaker. This is the primary source of FA in both pipelines and is directly tied to the noise characteristics of each recording: coi\_moi and chuyen\_ho, which contain sustained ambient crowd noise, show the largest orange bars, while vif\_1 the talk show shows the smallest. The implication is that the FA problem in this study is fundamentally a noise suppression problem

The secondary contributor is Ref silence near speech boundary (blue), which is relatively consistent across files and models (approximately 120-190s per file). This component reflects onset and offset timing errors: the model activates slightly before a speaker begins or continues slightly after a speaker ends. Unlike the orange component, this error is not driven by noise content but by the inherent imprecision of sliding-window segmentation at segment boundaries, and its consistency across files confirms it is a structural property of the pipeline rather than a data-dependent artifact.

The remaining two components FA from overlap on single-speaker reference regions (green) and too many speakers on true overlap (red) are minor contributors. The red component is negligible across all files and both models.

**Comparing the two models:** Pyannote’s total FA-like surplus is higher than DiariZen’s on coi\_moi and chuyen\_ho, driven entirely by a larger orange component. On dustin\_1 and vif\_1, the two models are comparable. This confirms that the FA gap between Pyannote and DiariZen on noisy files is not a systematic model weakness but a consequence of how each model’s activation sensitivity interacts with sustained non-speech noise: both models share the same root cause, but Pyannote activates more aggressively on those conditions.

### Final Results.

Table 60: Comparison DiariZen vs Pyannote (Best model).

| Criterion                            | Advantage | DiariZen (Best FT) | Pyannote (Best FT)     |
|--------------------------------------|-----------|--------------------|------------------------|
| DER: Synthetic Data (overall)        | DiariZen  | <b>13.793%</b>     | 17.441%                |
| Miss: Synthetic Data (overall)       | DiariZen  | <b>1.138%</b>      | 4.279%                 |
| FA: Synthetic Data (overall)         | Pyannote  | 9.731%             | <b>1.761%</b>          |
| Confusion: Synthetic Data (overall)  | DiariZen  | <b>2.924%</b>      | 11.401%                |
| Overlap F1: Synthetic Data (overall) | Pyannote  | 0.478              | <b>0.555</b>           |
| Pred. Seg count: Synthetic Data      | Pyannote  | 385                | <b>377</b>             |
| DER: 2 speakers                      | DiariZen  | <b>9.438%</b>      | 13.672%                |
| DER: 3 speakers                      | Pyannote  | 17.858%            | <b>17.008%</b>         |
| DER: 4 speakers                      | DiariZen  | <b>14.084%</b>     | 21.643%                |
| Overlap F1: 4 speakers               | DiariZen  | <b>0.677</b>       | 0.533                  |
| Conf: ov0/0% noise                   | DiariZen  | <b>0.490%</b>      | 11.274%                |
| FA: ov0/0% noise                     | Pyannote  | 6.653%             | <b>1.236%</b>          |
| DER: ov0/23% noise                   | Pyannote  | 15.368%            | <b>10.486%</b>         |
| Loss function                        | N/A       | PIT multilabel BCE | Powerset cross-entropy |

DiariZen outperforms Pyannote on the majority of criteria in Table 60. The two most critical advantages are Confusion (2.9% vs 11.4%) and Miss (1.1% vs 4.3%): both are unrecoverable errors at the downstream ASR step, and their combined effect produces the overall DER gap of 13.8% vs 17.4%.

Pyannote holds two clear advantages. First, FA is substantially lower (1.8% vs 9.7%), meaning the pipeline produces far fewer spurious speech activations on silent or noisy regions. Second, Overlap Precision is higher (0.63 vs 0.38): when Pyannote reports overlap, it is almost always genuine overlap, whereas DiariZen detects more overlap overall but at the cost of more false positives. Both advantages are structural consequences of the powerset argmax formulation, which is more conservative than sigmoid multilabel decoding.

**Conclusion.** DiariZen is the more suitable choice for Vietnamese conversational ASR pipelines where low Confusion and low Miss are the primary requirements. Pyannote is preferable in applications where low FA or high Overlap Precision is the priority.

### 6.5.5 PhoWhisper: Discussion

#### Objective.

The Vietnamese Whisper fine-tuning pipeline targets two objectives:

1. Reducing hallucination: errors where the model generates words that are not present in the audio, measured by Insertion % in WER.
2. Deduplication: removing duplicated samples in the training set, which is an indirect cause of overfitting and repetitive degeneration.

#### Reducing Hallucination.

The Whisper problem is an autoregressive model: each next token depends on the previous token.

When the audio is unclear, silent, or contains heavy overlapping speech, the model does not know when to stop and may repeat endlessly. A real example: a 0.34-second segment generated “subscribe subscribe kenh youtube...”.

### Five implemented mechanisms.

Table 61: Five implemented mechanisms for reducing hallucination.

| # | Mechanism                                                 | Effect                                                                                                                                                                                                |
|---|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Partial unfreeze of q_proj / v_proj (0.8% params)         | Only adapts the attention pattern for Vietnamese. The decoder prior (frozen) preserves the ability to recognize sentence stopping points, preventing overfitting to repetitive patterns.              |
| 2 | Hard cap<br>generation_max_length = 256                   | Strictly blocks infinite loops. 256 tokens $\approx$ 180-200 Vietnamese words, sufficient for all segments $\leq$ 30 seconds.                                                                         |
| 3 | Attention mask for padding frames                         | The encoder ignores the zero-padded part of audio shorter than 30 seconds. Without this fix, the decoder “sees” artificial signals from zero-padding and generates words that are not in the audio.   |
| 4 | Overlap filter: Stage 1 $\leq$ 18%,<br>Stage 2 $\leq$ 30% | Stage 1 uses only clean data ( $\leq$ 18% overlap) to build a “Linguistic Anchor.” Stage 2 expands to $\leq$ 30% from a stable checkpoint, avoiding learning incorrect mappings from ambiguous audio. |
| 5 | LR scale $\times 0.2$ (Stage 1 $\rightarrow$ Stage 2)     | Stage 2 trains with a learning rate 5 times smaller, avoiding catastrophic forgetting and preserving the learned attention pattern.                                                                   |

### Measured results.

Stage 1 provides the largest reduction (47%): partial unfreeze + attention mask + overlap filter  $\leq$  18% work together. Stage 2 further reduces it by 21% thanks to robustness data controlled by a smaller learning rate.

### Why dedup affects hallucination.

When the same audio segment appears N times in the training set, its gradient is accumulated N times, causing the model to overfit to that pattern. At inference time, when it encounters similar audio, the model generates based on its memorized pattern instead of performing real transcription: this is hallucination in the form of repetitive degeneration. Dedup ensures that each audio-text pair appears only once.

### Two-layer dedup (2-pass).

1. **Pass 1: Within-parent:** same audio file, same text (normalized), timestamp difference  $\leq$  0.30s OR IoU  $\geq$  85% to remove. This catches duplicate chunks created by chunking with overlapping

left/right padding.

2. **Pass 2: Cross-parent:** `fingerprint = hash(audio_path, start, end, text)`. This catches the same file appearing in multiple datasets or being re-scanned from multiple directories. Two identical sentences from two different audio files are not deduplicated: that is genuinely different data.

Dedup is run twice: once when scanning each folder and once when merging data from multiple sources.

### Supporting segment chunking.

Segments < 2.5s are merged (a major hallucination source). Segments > 30s are forcibly split. Padding of +/-0.10-0.20s at both ends ensures sufficient acoustic context.

**Insertion / Hallucination Control:** Insertions drop from 7.29% (base) to 3.03% (Stage 2) a reduction of 4.26% and 2,099 absolute errors. Stage 1 resolves the majority through clean-data training; Stage 2 continues the reduction via robustness data.

**Robustness vs. Accuracy Trade-off:** Stage 2 slightly increases Substitution (5.57% to 5.69%) while both Insertion and Deletion decrease: this is an inevitable trade-off when the model is exposed to robustness data with higher overlap ( $\leq 30\%$ ) and heavier noise. Specifically, when the input acoustic spectrum is distorted by background noise or overlapping signals, the model tends to infer the nearest contextual vocabulary item (substitution) rather than omit it completely (deletion). This behavior reflects that the model has learned to make informed guesses rather than remain silent on difficult audio, and this is considered an acceptable trade-off because substitutions are easier to detect and correct than deletions in the practical post-processing pipeline.

**Data Quality vs. Quantity:** VIF (gold\_real, 1.48h) achieves WER 6.48%, lower than Conan (silver\_real, 12.85h, 7.38%). Coi Mo (hard\_real): despite 2.57h of training data: still records the highest WER at 19.96%, reflecting the intrinsic difficulty of the high-overlap conversational domain.

**Summary:** A total of 7 technical mechanisms were implemented together, focusing on three layers: input data (dedup, overlap filter, segment chunking), training architecture (partial unfreeze, attention mask, LR scale), and inference (generation hard cap). As a result, Insertion % decreased from 7.29% to 3.03%: equivalent to a 58.5% reduction in hallucination errors.

### 6.5.6 Qwen Discussion

The pipeline outputs. Key observations across 5 datasets:

1. **Schema:** 5/5 datasets pass `schema_pass_rate = 1.000`: JSON output always structurally valid.
2. **Evidence:** 4/5 datasets achieve `evidence_support_rate = 1.000`; Dustin (Vlog) only reaches 0.500 due to `bridge_utils` speaker matching issue.
3. **Generic summary:** Chuyen Ho, Coi Mo, Dustin, VIF all have `generic_speaker_summary_rate = 1.000`: all speaker summaries are generic. Only Conan (13 speakers) achieves 0.615.

4. **LLM Judge:** Five datasets were judged with total scores of 8-11/12, verdict “review”. Common issues: the main summary does not state the main point correctly; speaker roles are mixed up.
5. **Meeting type:** Chuyen Ho, Coi Mo, Conan, Dustin are classified as “talkshow”; VIF is “discussion”. risk\_flags = “The transcript has many segments/topics” appears in all 5 datasets.

## 7 Conclusion

Table 62: Summary of achievements and limitations by system component.

| Component                 | Achievements                                                                                         | Limitations                                                                                                                                                                                                              |
|---------------------------|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Diarization</b>        | Handles noisy multi-speaker audio. Reduces missed speech and keeps speaker confusion relatively low. | when audio has too much noise False alarm is still high. Over-segmentation remains.                                                                                                                                      |
| <b>ASR</b>                | Reduces hallucination after fine-tuning. Produces usable Vietnamese transcripts.                     | Overlapped speech is not fully handled. The system still struggles with English words in Vietnamese conversations.                                                                                                       |
| <b>Qwen Summarization</b> | Generates local meeting summaries. Works with diarized and transcribed content.                      | Summaries are still general, and some details may be missed. Because a relatively small local Qwen model is used, its ability to identify the role, responsibility, or contribution of each participant remains limited. |
| <b>Overall Pipeline</b>   | Combines diarization, ASR, and summarization into one framework for Vietnamese meeting analysis.     | Needs further improvement in overlap handling, false alarm reduction, ASR robustness, summary detail, and participant-role understanding.                                                                                |

## References

- [1] X. Anguera Miró, S. Bozonnet, N. Evans, C. Fredouille, G. Friedland, O. Vinyals, Speaker diarization: A review of recent research, *IEEE Transactions on Audio, Speech, and Language Processing* 20 (2012) 356–370. doi:10.1109/TASL.2011.2125954.
- [2] H. Bredin, pyannote.audio 2.1 speaker diarization pipeline: Principle, benchmark, and recipe, in: *Proc. INTERSPEECH*, 2023, pp. 1983–1987. doi:10.21437/Interspeech.2023-105.
- [3] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, I. Sutskever, Robust speech recognition via large-scale weak supervision, 2022. arXiv:2212.04356.
- [4] T.-T. Le, L. T. Nguyen, D. Q. Nguyen, PhoWhisper: Automatic speech recognition for vietnamese, in: *Proceedings of the ICLR 2024 Tiny Papers Track*, 2024. arXiv:2406.02555.
- [5] J. Han, F. Landini, J. Rohdin, A. Silnova, M. Diez, L. Burget, Leveraging self-supervised learning for speaker diarization, 2024. arXiv:2409.09408.
- [6] H.-T. Luong, H.-Q. Vu, VIVOS: Vietnamese speech corpus for ASR, [Online]. Available: <https://zenodo.org/records/7068130>, 2016. doi:10.5281/zenodo.7068130, accessed: Apr. 8, 2026.
- [7] L. T. T. Tran, H.-G. Kim, H. M. La, S. V. Pham, Automatic speech recognition of vietnamese for a new large-scale corpus, *Electronics* 13 (2024) 977. URL: <https://www.mdpi.com/2079-9292/13/5/977>. doi:10.3390/electronics13050977.
- [8] R. Ardila, M. Branson, K. Davis, M. Henretty, M. Kohler, J. Meyer, R. Morais, L. Saunders, F. M. Tyers, G. Weber, Common voice: A massively-multilingual speech corpus, in: *Proceedings of the Twelfth Language Resources and Evaluation Conference*, European Language Resources Association, Marseille, France, 2020, pp. 4218–4222. URL: <https://aclanthology.org/2020.lrec-1.520/>.

- [9] Haydar Kadioglu, Speech Noise Dataset, [Online]. Available: <https://www.kaggle.com/datasets/abdullahhaydarkadolu/speech-noise-dataset>, 2025. Accessed: Apr. 8, 2026.
- [10] BUT-FIT, BUT-FIT/diarizen-wavlm-large-s80-md-v2, [Online]. Available: <https://huggingface.co/BUT-FIT/diarizen-wavlm-large-s80-md-v2>, 2025. Accessed: Apr. 8, 2026.
- [11] P. Singh, S. Ganapathy, Deep self-supervised hierarchical clustering for speaker diarization, in: Proc. INTERSPEECH, 2020, pp. 294–298. doi:10.21437/Interspeech.2020-2297.
- [12] S. Horiguchi, Y. Fujita, S. Watanabe, Y. Xue, K. Nagamatsu, End-to-end speaker diarization for an unknown number of speakers with encoder-decoder based attractors, in: Proc. INTERSPEECH, 2020, pp. 269–273. doi:10.21437/Interspeech.2020-1022.
- [13] N. Ryant, dscore: Rich transcription time marked (RTTM) format documentation, [Online]. Available: <https://github.com/nryant/dscore>, 2019. Accessed: Apr. 8, 2026.
- [14] J. Bai, S. Bai, Y. Chu, et al., Qwen technical report, 2023. URL: <https://arxiv.org/abs/2309.16609>. arXiv:2309.16609.
- [15] SYSTRAN, faster-whisper: Faster whisper transcription with CTranslate2, [Online]. Available: <https://github.com/SYSTRAN/faster-whisper>, n.d. Accessed: Apr. 8, 2026.
- [16] Facebook Research, Demucs: Music source separation, [Online]. Available: <https://github.com/facebookresearch/demucs>, n.d. Accessed: Apr. 8, 2026.
- [17] yt-dlp, yt-dlp: A feature-rich command-line audio/video downloader, [Online]. Available: <https://github.com/yt-dlp/yt-dlp>, n.d. Accessed: Apr. 8, 2026.
- [18] Ajay Ramachandran, SponsorBlock: Skip youtube video sponsors, [Online]. Available: <https://github.com/ajayyy/SponsorBlock>, n.d. Accessed: Apr. 8, 2026.
- [19] A. Baeovski, H. Zhou, A. Mohamed, M. Auli, wav2vec 2.0: A framework for self-supervised

- learning of speech representations, 2020. URL: <https://arxiv.org/abs/2006.11477>. arXiv:2006.11477.
- [20] Thai Binh Nguyen, Vietnamese End-to-End Speech Recognition Using wav2vec 2.0, [Online]. Available: <https://huggingface.co/nguyenvulebinh/wav2vec2-base-vietnamese-250h>, 2021. doi:10.5281/zenodo.5356039, accessed: Apr. 8, 2026.
- [21] A. Graves, S. Fernández, F. Gomez, J. Schmidhuber, Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks, in: Proceedings of the 23rd International Conference on Machine Learning, 2006, pp. 369–376. doi:10.1145/1143844.1143891.
- [22] R. Scheibler, E. Bezzam, I. Dokmanić, Pyroomacoustics: A python package for audio room simulation and array processing algorithms, in: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing, 2018, pp. 351–355. doi:10.1109/ICASSP.2018.8461310.
- [23] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao, J. Wu, L. Zhou, S. Ren, Y. Qian, Y. Qian, M. Zeng, X. Yu, F. Wei, WavLM: Large-scale self-supervised pre-training for full stack speech processing, IEEE Journal of Selected Topics in Signal Processing 16 (2022) 1505–1518. doi:10.1109/JSTSP.2022.3188113.
- [24] K. Kinoshita, M. Delcroix, N. Tawara, Integrating end-to-end neural and clustering-based diarization: Getting the best of both worlds, in: Proc. IEEE International Conference on Acoustics, Speech and Signal Processing, 2021, pp. 7198–7202. doi:10.1109/ICASSP39728.2021.9414333.
- [25] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, R. Pang, Conformer: Convolution-augmented transformer for speech recognition, in: Proc. INTERSPEECH, Shanghai, China, 2020, pp. 5036–5040. doi:10.21437/Interspeech.2020-3015.

- [26] Y. Fujita, N. Kanda, S. Horiguchi, K. Nagamatsu, S. Watanabe, End-to-end neural speaker diarization with permutation-free objectives, in: Proc. INTERSPEECH, Graz, Austria, 2019, pp. 4300–4304. doi:10.21437/Interspeech.2019-2899.
- [27] J. Howard, S. Ruder, Universal language model fine-tuning for text classification, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, Association for Computational Linguistics, 2018, pp. 328–339. URL: <https://aclanthology.org/P18-1031/>. doi:10.18653/v1/P18-1031.
- [28] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, R. Hadsell, Overcoming catastrophic forgetting in neural networks, Proceedings of the National Academy of Sciences 114 (2017) 3521–3526. doi:10.1073/pnas.1611835114.
- [29] S. Zaiem, T. Parcollet, S. Essid, Less forgetting for better generalization: Exploring continual-learning fine-tuning methods for speech self-supervised representations, 2024. URL: <https://arxiv.org/abs/2407.00756>. arXiv:2407.00756.
- [30] H. Bredin, A. Laurent, End-to-end speaker segmentation for overlap-aware resegmentation, in: Proc. INTERSPEECH, Brno, Czech Republic, 2021, pp. 3111–3115. doi:10.21437/Interspeech.2021-560.
- [31] A. Plaquet, H. Bredin, Powerset multi-class cross entropy loss for neural speaker diarization, in: Proc. INTERSPEECH, 2023, pp. 3222–3226. URL: [https://www.isca-archive.org/interspeech\\_2023/plaquet23\\_interspeech.html](https://www.isca-archive.org/interspeech_2023/plaquet23_interspeech.html). doi:10.21437/Interspeech.2023-205.
- [32] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations, 2022. URL: <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [33] I. Sutskever, O. Vinyals, Q. V. Le, Sequence to sequence learning with neural networks, in:

- Advances in Neural Information Processing Systems, volume 27, 2014, pp. 3104–3112. URL: <https://papers.nips.cc/paper/5346-sequence-to-sequence-learning-with-neural-networks>.
- [34] T. J. Park, N. Kanda, D. Dimitriadis, K. J. Han, S. Watanabe, S. Narayanan, A review of speaker diarization: Recent advances with deep learning, *Computer Speech & Language* 72 (2022) 101317. doi:10.1016/j.csl.2021.101317.
- [35] X. Anguera Miró, Robust Speaker Diarization for Meetings, Ph.d. dissertation, Universitat Politècnica de Catalunya, Barcelona, Spain, 2006. URL: <https://xavieranguera.com/phdthesis/>.
- [36] A. C. Morris, V. Maier, P. Green, From WER and RIL to MER and WIL: Improved evaluation measures for connected speech recognition, in: *Proc. INTERSPEECH*, Jeju Island, Korea, 2004, pp. 2765–2768. doi:10.21437/Interspeech.2004-668.
- [37] BUT Speech@FIT, DiariZen: Speaker diarization toolkit, [Online]. Available: <https://github.com/BUTSpeechFIT/DiariZen>, n.d. Accessed: Apr. 8, 2026.